PROJECT REPORT

Design and Verification of a RISC-V Multiply-Accumulate (MAC) Unit for Digital Signal Processing

Hardware Description Language: Verilog (IEEE Std 1364-2005)
Simulator: Icarus Verilog 12 Waveforms: GTKWave
Synthesis: Yosys 0.70 Place & route: nextpnr-ecp5
Integrated in a 5-stage pipelined RV32IM processor

Submitted by

Mohsin Mushtaq	2025MECECS001
Mubashir Karim Ganaie	2025MECECS005

Submitted to

Dr. Afshan Amin Khan

Department of Electronics and Communications
National Institute of Technology Srinagar

Code: `https://github.com/mohsin2240/rv32-booth-mac`

November 10, 2026

Abstract

Multiply-accumulate (MAC) is the basic operation behind digital signal processing (DSP) kernels such as FIR filters, dot products and matrix multiplication. This project designs and verifies a MAC unit for RISC-V and integrates it into a complete 5-stage pipelined RISC-V processor. The unit implements eight R-type instructions in the RISC-V `custom-0` opcode space. It provides a dedicated 64-bit accumulator with signed saturation and a sticky overflow flag, together with unsigned modular accumulation, a Q15 fixed-point read-out matching the ARM CMSIS-DSP convention, a parameterised radix-4 Modified Booth multiplier, and a two-stage pipeline that accepts one instruction per cycle. Within the processor (IF/ID/EX/MEM/WB, RV32I plus RV32M multiply, full forwarding and hazard detection), the unit's two stages coincide with EX and MEM, so MAC instructions flow through the pipeline like ordinary instructions.

At unit level, two self-checking testbenches with an independent reference model perform 88,896 checks with zero errors; these include an exhaustive 8-bit multiplier test, analytical DSP benchmarks and a 3,000-instruction random regression. At system level, real RISC-V programs produced by a purpose-written assembler run on the processor, comprising an 18-part ISA and hazard self-test and a 16-tap Q15 low-pass FIR filter over 128 samples. All programs pass. The FIR output is bit-identical to the golden model and attenuates the unwanted tone by 60.1 dB. With the MAC instructions, the filter runs 1.59× faster (looped) and 2.67× faster (unrolled) than the same filter in plain RV32IM code, and pipeline diagrams reconstructed from the simulation trace and VCD waveforms document the pipeline behaviour.

Finally, Yosys produces a latch-free MAC netlist of 9,532 gates and 179 flip-flops, which passes the full testbench at gate level. After place-and-route with nextpnr on a Lattice ECP5 FPGA, the MAC unit runs at 56.4 MHz (78.8 MHz using hard DSP blocks), and the complete processor with the MAC unit runs at 34.6 MHz using 9 % of the device. RTL schematics, floorplans, routing-density maps and critical-path analyses are included.

Contents

1 Introduction **4**
- 1.1 Motivation . . . 4
- 1.2 Objectives . . . 4
- 1.3 Report organisation . . . 4

2 Background and Theory **6**
- 2.1 Multiply-accumulate in digital signal processing . . . 6
- 2.2 The RISC-V ISA and custom extensions . . . 6
- 2.3 Fixed-point arithmetic and the Q15 format . . . 6
- 2.4 Accumulator width, guard bits and saturation . . . 6
- 2.5 Radix-4 Modified Booth multiplication . . . 7
- 2.6 Pipelining . . . 8

3 Design Specification **9**
- 3.1 Instruction encoding . . . 9
- 3.2 Instruction set . . . 9
- 3.3 Module interface . . . 10

4 Implementation **11**
- 4.1 Source files . . . 11
- 4.2 Micro-architecture . . . 11
- 4.3 Booth multiplier . . . 12
- 4.4 Stage 1: decode and multiply . . . 12
- 4.5 Stage 2: accumulate, saturate and read out . . . 12
- 4.6 Coding guidelines followed . . . 13

5 RISC-V Pipelined Processor Integration **14**
- 5.1 Pipeline organisation . . . 14
- 5.2 Hazard handling . . . 15
- 5.3 Instruction set and system . . . 16
- 5.4 Software toolchain . . . 16
- 5.5 DSP test case: low-pass FIR filter . . . 17

6 Verification Methodology **18**
- 6.1 Testbench architecture . . . 18
- 6.2 Test plan . . . 19
- 6.3 Multiplier unit test . . . 19
- 6.4 Constrained-random stimulus . . . 19
- 6.5 Mutation test (checking the checker) . . . 19
- 6.6 System-level verification . . . 20

7 MAC Unit Simulation Results **21**
- 7.1 Summary . . . 21
- 7.2 Directed test results . . . 21
- 7.3 Simulation waveforms . . . 21
- 7.4 DSP benchmark 1: moving-average FIR . . . 23
- 7.5 DSP benchmark 2: triangular low-pass FIR . . . 23
- 7.6 Functional coverage . . . 24

7.7 Performance analysis 24

8 System-Level Simulation Results **26**
8.1 Results summary 26
8.2 Pipeline behaviour 26
8.3 Processor waveform 29
8.4 DSP results: the FIR filter on the processor 31
8.5 Performance: MAC unit vs. plain RV32IM 32

9 Logic Synthesis and FPGA Implementation **34**
9.1 RTL schematics generated by Yosys 34
9.2 Generic gate-level synthesis of the MAC unit 36
9.3 Area breakdown: MAC unit and complete processor 38
9.4 FPGA implementation on the Lattice ECP5 39
9.5 Post-synthesis gate-level simulation 43

10 Discussion **44**
10.1 Design trade-offs 44
10.2 Limitations 44
10.3 Future work 44

11 Conclusion **45**

References **46**

A Source Code **47**
A.1 `rtl/mac_unit.v` (main module) 47
A.2 `rtl/booth_multiplier.v` 50
A.3 `tb/tb_mac_unit.v` (testbench) 52
A.4 `tb/tb_booth_multiplier.v` 61
A.5 `rtl/rv32_core.v` (5-stage RISC-V processor) 63
A.6 `rtl/riscv_soc.v` 71
A.7 `tb/tb_riscv_soc.v` (system testbench) 73

B RISC-V Programs **77**
B.1 `sw/isa_test.s` 77
B.2 `sw/fir_mac_loop.s` 81
B.3 `sw/fir_rv32im_loop.s` 82

C Synthesis Scripts **84**
C.1 `synth/synth_generic.ys` 84
C.2 `synth/synth_ecp5.ys` 84
C.3 `synth/fmax_wrapper.v` 85
C.4 `synth/native_multiplier.v` (comparison only) 86
C.5 `synth/synth_core.ys` 86
C.6 `synth/schematics.ys` 87

D Simulation Logs **88**
D.1 Booth multiplier testbench 88
D.2 MAC unit testbench 88
D.3 Processor: `isa_test` 90
D.4 Processor: `fir_mac_loop` 90
D.5 Processor: `fir_rv32im_loop` 90

List of Figures

3.1 R-type encoding of the MAC instructions (custom-0 major opcode). 9

4.1 Two-stage MAC unit datapath. Grey bars are pipeline registers. 11

5.1 5-stage RV32IM pipeline with the integrated MAC unit. Grey bars are pipeline registers; dashed green blocks are the hazard and forwarding units; the MAC unit (orange) spans EX and MEM. 14

6.1 Self-checking testbench architecture. 18

7.1 Simulation waveform for T2: back-to-back MAC instructions. 22
7.2 Simulation waveform for T5: positive saturation and the sticky overflow flag. . . 23
7.3 FIR benchmark results computed by the MAC unit. 24

8.1 `isa_test` T12 executing: `mul.acc`, `mac` and the dependent `addi t3,t2,1`. The `addi` is held in ID for one cycle (MAC-use stall) and then receives `t2` from WB; the result is 85 as expected. 27
8.2 `fir_mac_loop`: two iterations of the inner loop. Every iteration ends with a taken `bnez`, which squashes the two instructions fetched behind it (2-cycle penalty). . . 28
8.3 `fir_mac_unrolled`: the unrolled `lh/lh/lh/lh/mac/mac` pattern issues one instruction per cycle with no stalls or flushes. 28
8.4 Simulation waveform of the 5-stage core executing the FIR program 30
8.5 FIR filtering executed on the RISC-V core: input, CPU output and golden model (top), and the error against the golden model and against an ideal floating-point filter (bottom). 31
8.6 Spectrum of the input and of the output computed by the processor. 31
8.7 Quantised Q15 filter taps and frequency response. 32
8.8 Measured FIR execution time (128 outputs × 16 taps) and speed-up. 32
8.9 Cycle breakdown and CPI of every program. 33

9.1 Yosys RTL schematic of `booth_multiplier` with $W = 6$ (word-level cells: `$eq` group decoders, Booth-recoding `$pmux`, partial-product `$macc`/`$alu`). 35
9.2 The same 6-bit Booth multiplier after gate-level synthesis (216 gates). 36
9.3 Yosys RTL schematic of the complete `mac_unit` (Booth multiplier shown as a single block). 37
9.4 Cell-type distribution of the synthesised MAC unit. 38
9.5 Generic-gate area breakdown of the MAC unit and of the complete processor. . . 39
9.6 ECP5 resource usage (log scale) and post-route $F_{\max}$ of the three designs. 40
9.7 Cumulative delay along the critical register-to-register path of each design. . . . 40
9.8 Slack histograms of all timing endpoints (100 MHz target for A/B, 50 MHz for C). 41
9.9 Die render of the MAC unit (design A) on the ECP5: placement by block, routing density and critical path. 42
9.10 Die render of the complete RV32IM core with the MAC unit (design C). The register file (purple) sits next to the MAC unit; the critical path runs through the RV32M multiplier on the DSP blocks. 43

List of Tables

2.1 Common DSP kernels expressed as multiply-accumulate operations. 6
2.2 Radix-4 Booth recoding table. 7

3.1 MAC unit instruction set. ACC is the 64-bit accumulator and OVF the sticky overflow flag. 9
3.2 Port list of `mac_unit`. 10

4.1 Project source files. 11

5.1 Work done in each pipeline stage. 15
5.2 Instructions implemented by `rv32_core`. 16
5.3 Programs executed on the processor. 16

6.1 Test plan of the system testbench. 19

7.1 Verification summary. 21
7.2 Selected directed test results (DUT output = expected). 21
7.3 Moving-average FIR step response. 23
7.4 Low-pass FIR response to DC + Nyquist input. 24
7.5 Functional coverage collected over the full regression. 24
7.6 Arithmetic instruction count per output of an 8-tap FIR (estimate). 25

8.1 Programs executed on the 5-stage core (all results from the hardware performance counters). 26
8.2 Processor mutation tests. 26
8.3 FIR benchmark: MAC instructions vs. RV32IM software (measured). 32

9.1 Generic synthesis result (flattened `mac_unit`). 38
9.2 ECP5 LFE5U-85F implementation results (post place & route). 39

1 Introduction

1.1 Motivation

Signal-processing algorithms spend most of their run time evaluating sums of products. An N-tap finite impulse response (FIR) filter, for example, computes

$$y[n] = \sum_{k=0}^{N-1} h[k]\, x[n-k], \tag{1.1}$$

which needs N multiplications and N additions for every output sample. Dedicated DSP processors therefore provide a single-cycle *multiply-accumulate* instruction ACC $\leftarrow$ ACC $+ a \cdot b$ together with a wide accumulator, so that long sums do not overflow.

RISC-V is an open, modular instruction-set architecture (ISA) [8]. The base RV32I ISA and the standard `M` extension provide `mul`, `mulh` and `add`, but no fused multiply-accumulate and no 64-bit accumulator. On RV32IM, accumulating one 32×32-bit product into a 64-bit sum takes about six instructions. RISC-V explicitly reserves opcode space for custom extensions [8], and DSP-oriented RISC-V cores such as PULP RI5CY / OpenHW CV32E40P use this approach to add MAC instructions [4, 5].

1.2 Objectives

1. Define a small, consistent RISC-V instruction-set extension for multiply-accumulate operations in the `custom-0` opcode space.
2. Implement a synthesizable, pipelined MAC unit in Verilog with: a 64-bit accumulator, signed saturation, unsigned accumulation, and Q15 fixed-point support.
3. Implement the multiplier using the radix-4 Modified Booth algorithm.
4. Verify the design with self-checking testbenches, using directed, benchmark and constrained-random tests, and compare the results against an independent reference model and analytically known values.
5. Document the results using terminal output, logs and simulation waveforms.
6. Integrate the MAC unit into a 5-stage pipelined RISC-V processor and run real RISC-V DSP programs on it, measuring the speed-up over plain RV32IM code.
7. Synthesise the MAC unit and the processor with Yosys, place and route them on an FPGA, and re-verify the synthesised netlist.

1.3 Report organisation

The remainder of this report is organised as follows. Chapter 2 presents the theory (MAC operations in DSP, RISC-V extensions, fixed-point arithmetic, Booth multiplication and pipelining), and Chapter 3 gives the instruction-set and interface specification. Chapter 4 describes the MAC unit RTL implementation, while Chapter 5 covers its integration into the 5-stage RISC-V processor. Chapter 6 sets out the verification methodology, Chapter 7 reports the MAC unit simulation results, and Chapter 8 reports the processor-level simulation results (pipeline diagrams, waveforms, DSP results and performance). Chapter 9 presents the synthesis and FPGA implementation (schematics, area, timing and floorplans), and Chapters 10 and 11 give the

discussion and conclusion. The appendices contain the full source code, scripts, programs and logs.

2 Background and Theory

2.1 Multiply-accumulate in digital signal processing

Table 2.1 shows that common DSP kernels reduce to repeated MAC operations. The MAC rate of a processor therefore largely sets its DSP throughput, which is why DSP architectures are traditionally rated in MAC operations per second.

Table 2.1: Common DSP kernels expressed as multiply-accumulate operations.

Kernel	Expression	MACs per output
FIR filter	$y[n] = \sum_k h[k]x[n-k]$	N (taps)
Dot product	$s = \sum_i a_i b_i$	N
Correlation	$r[l] = \sum_n x[n]\, y[n+l]$	N per lag
Matrix multiply	$c_{ij} = \sum_k a_{ik} b_{kj}$	K
Energy	$E = \sum_n x[n]^2$	N

2.2 The RISC-V ISA and custom extensions

The RISC-V Unprivileged ISA [8] uses fixed 32-bit instructions with a 7-bit major opcode in bits [6:0]. In the base opcode map, the major opcodes `custom-0` (`0001011`) and `custom-1` (`0101011`) are reserved for non-standard extensions and are guaranteed never to be used by standard extensions. The R-type format, with fields `funct7`, `rs2`, `rs1`, `funct3`, `rd` and `opcode`, has two source registers and one destination register, which suits binary arithmetic operations. This project places all of its instructions in `custom-0` and uses `funct3` to select the operation.

Related work: the PULP RI5CY core [4] and its successor OpenHW CV32E40P [5] add Xpulp instructions such as `p.mac` and `p.msu`, which accumulate into a general-purpose register. This design instead uses a dedicated 64-bit accumulator, in the style of classic DSPs, so that 32×32-bit products can be accumulated at full precision.

2.3 Fixed-point arithmetic and the Q15 format

DSP code often uses fixed-point fractional formats instead of floating point. In the Q15 format, a 16-bit two's-complement integer X represents the real value $X/2^{15}$, which lies in $[-1,\, 1-2^{-15}]$. Multiplying two Q15 numbers gives a Q30 product:

$$\frac{X_1}{2^{15}} \cdot \frac{X_2}{2^{15}} = \frac{X_1 X_2}{2^{30}}. \tag{2.1}$$

To convert an accumulated Q30 sum back to Q15, shift it right arithmetically by 15 bits and saturate it to the 16-bit range. This is exactly what the ARM CMSIS-DSP library does in `arm_fir_q15`, using `__SSAT(acc >> 15, 16)` on a 64-bit accumulator [6]. One important corner case is $(-1.0) \times (-1.0) = +1.0$, which cannot be represented in Q15 and must saturate to 32767/32768.

2.4 Accumulator width, guard bits and saturation

A signed 32×32-bit product needs at most 64 bits; its magnitude is at most 2^{62}, reached by $(-2^{31})^2$. Summing N such products needs a further $\lceil \log_2 N \rceil$ *guard bits* to rule out overflow

completely. A 64-bit accumulator is enough for typical Q15 workloads (products below 2^{30}, leaving more than 30 guard bits), but it can overflow with full-scale 32-bit operands. Two's-complement wrap-around would turn a small overflow into a large error with the wrong sign, so DSP hardware uses *saturation arithmetic*: the result is clamped to the largest or smallest representable value [7].

Signed overflow of $S = A+P$, where A and P are 64-bit two's-complement values, is detected by sign-extending both operands to 65 bits:

$$S_{65} = \{A_{63}, A\} + \{P_{63}, P\}, \qquad ovf = S_{65}[64] \oplus S_{65}[63], \tag{2.2}$$

$$\mathrm{sat}(S) = \begin{cases} \texttt{0x7FFF_FFFF_FFFF_FFFF} & \text{if } ovf \wedge S_{65}[64] = 0, \\ \texttt{0x8000_0000_0000_0000} & \text{if } ovf \wedge S_{65}[64] = 1, \\ S_{65}[63:0] & \text{otherwise.} \end{cases} \tag{2.3}$$

2.5 Radix-4 Modified Booth multiplication

Booth's algorithm [1] recodes the multiplier so that runs of ones need fewer additions. MacSorley's radix-4 ("modified") form [2] scans the multiplier B in overlapping 3-bit groups $\{b_{2i+1}, b_{2i}, b_{2i-1}\}$ with $b_{-1} = 0$. Each group is recoded into a digit $d_i \in \{-2, -1, 0, +1, +2\}$:

$$d_i = -2b_{2i+1} + b_{2i} + b_{2i-1}, \qquad A \cdot B = \sum_{i=0}^{W/2-1} d_i \, A \, 4^i. \tag{2.4}$$

A W-bit multiplication therefore needs only $W/2$ partial products instead of W, which halves the depth of the adder tree [3]. Each partial product is 0, $\pm A$ or $\pm 2A$, so it can be formed with a shift and an optional negation. No real multiplication is required.

Table 2.2: Radix-4 Booth recoding table.

b_{2i+1}	b_{2i}	b_{2i-1}	Digit d_i	Partial product
0	0	0	0	0
0	0	1	+1	$+A$
0	1	0	+1	$+A$
0	1	1	+2	$+2A$
1	0	0	−2	$-2A$
1	0	1	−1	$-A$
1	1	0	−1	$-A$
1	1	1	0	0

Worked example. Take $A = 7$ and $B = -3$ with $W = 6$, so $B = \texttt{111101}_2$ and the extended multiplier is `111101 0`. The groups, from LSB to MSB, are `010` $\to +1$, `110` $\to -1$ and `111` $\to 0$. Hence $A \cdot B = (+1)A \cdot 4^0 + (-1)A \cdot 4^1 + 0 = 7 - 28 = -21$, as expected.

Signed and unsigned operation on one multiplier. Booth recoding assumes two's-complement operands. For an *unsigned* 32-bit multiplication, the operands are zero-extended to 34 bits, which makes them non-negative signed numbers. For a signed multiplication they are sign-extended. Using 34 bits instead of 33 also keeps W even, as radix-4 recoding requires. The true product always fits in 64 bits, so the low 64 bits of the 68-bit result are exact in both cases.

2.6 Pipelining

Pipelining splits a datapath into stages separated by registers. This raises the achievable clock frequency and lets a new operation start every cycle. The MAC unit uses two stages: multiplication, then accumulation. Both the accumulator read and write happen in the second stage, in program order, so a dependent sequence such as `mac`, `mac`, `rdacc` needs no forwarding paths and no pipeline stalls.

The 5-stage RISC pipeline. The classic RISC processor pipeline [13] has five stages: instruction fetch (*IF*), decode and register read (*ID*), execute or address computation (*EX*), data memory access (*MEM*) and register write-back (*WB*). Ideally, one instruction completes every cycle (CPI = 1), and three kinds of hazard can prevent this. Data hazards occur when an instruction needs a result that has not been written back yet; they are removed by *forwarding* results from later pipeline registers, and only when a value is produced too late, such as a load result at the end of MEM, is a one-cycle *stall* needed. Control hazards occur because the instructions fetched after a taken branch are on the wrong path and must be *flushed*. Structural hazards occur when two instructions need the same resource in the same cycle, and they are avoided by giving each stage its own resources. Chapter 5 applies these rules to a processor that contains the MAC unit.

3 Design Specification

3.1 Instruction encoding

All MAC instructions use the RISC-V R-type format with `opcode = 0001011` (`custom-0`) and `funct7 = 0000000` (Figure 3.1).

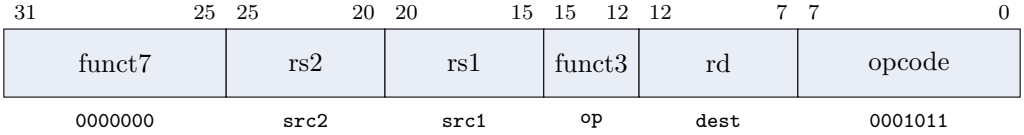


Figure 3.1: R-type encoding of the MAC instructions (custom-0 major opcode).

3.2 Instruction set

Table 3.1: MAC unit instruction set. ACC is the 64-bit accumulator and OVF the sticky overflow flag.

`funct3`	Assembly	Operation	Value written to `rd`
000	`mac rd,rs1,rs2`	ACC $\leftarrow \mathrm{sat}_{64}(\mathrm{ACC} + \mathrm{rs1}\cdot\mathrm{rs2})$, signed	ACC[31:0]
001	`msu rd,rs1,rs2`	ACC $\leftarrow \mathrm{sat}_{64}(\mathrm{ACC} - \mathrm{rs1}\cdot\mathrm{rs2})$, signed	ACC[31:0]
010	`mul.acc rd,rs1,rs2`	ACC $\leftarrow \mathrm{rs1}\cdot\mathrm{rs2}$; OVF $\leftarrow 0$	ACC[31:0]
011	`macu rd,rs1,rs2`	ACC $\leftarrow$ ACC + rs1·rs2, unsigned mod 2^{64}	ACC[31:0]
100	`rdacc.lo rd`	read only	ACC[31:0]
101	`rdacc.hi rd`	read only	ACC[63:32]
110	`rdacc.q15 rd`	read only	$\mathrm{sext}(\mathrm{sat}_{16}(\mathrm{ACC} \ggg 15))$
111	`clracc rd`	ACC $\leftarrow 0$; OVF $\leftarrow 0$	0

Three additional rules complete the specification. As in all RISC-V instructions, writes to `x0` are discarded (`rd_we_o = 0`), while the accumulator is still updated. An instruction whose opcode is not `custom-0`, or whose `funct7` is not zero, raises `illegal_o` and leaves the accumulator and flag unchanged; a core would turn this into an illegal-instruction exception. Finally, OVF is set by any saturating `mac`/`msu` and stays set until `clracc` or `mul.acc`, which lets software check a whole dot product for overflow once, at the end.

3.3 Module interface

Table 3.2: Port list of `mac_unit`.

Port	Dir.	Width	Description
`clk_i`	in	1	Clock (rising-edge triggered)
`rst_ni`	in	1	Synchronous reset, active low
`valid_i`	in	1	An instruction and its operands are valid this cycle
`instr_i`	in	32	32-bit RISC-V instruction word
`rs1_i`, `rs2_i`	in	32	Operand values read from the register file
`ready_o`	out	1	Always 1: the unit never stalls
`valid_o`	out	1	Result valid (2 cycles after `valid_i`)
`rd_addr_o`	out	5	Destination register index
`rd_we_o`	out	1	Register-file write enable (0 for `x0` or illegal)
`rd_data_o`	out	32	Value to write back
`illegal_o`	out	1	Instruction is not a valid MAC instruction
`ovf_o`	out	1	Sticky saturation flag
`acc_o`	out	64	Current accumulator value (for debug and observability)

4 Implementation

4.1 Source files

Table 4.1: Project source files.

File	Contents
`rtl/mac_unit.v`	Main module: decoder, 2-stage pipeline, saturating accumulator, Q15 conversion
`rtl/booth_multiplier.v`	Parameterised radix-4 Booth multiplier (instantiated with $W = 34$)
`tb/tb_mac_unit.v`	Self-checking system testbench (reference model, scoreboard, 13 test groups)
`tb/tb_booth_multiplier.v`	Multiplier unit testbench (exhaustive 8-bit, random 34-bit)
`run_sim.bat`, `run_sim.sh`	Compile-and-run scripts for Icarus Verilog
`synth/*.ys`, `run_synth.sh`	Yosys synthesis scripts and the synthesis / P&R / gate-level flow
`synth/fmax_wrapper.v`	Registered-I/O wrapper used only for timing analysis
`rtl/rv32_core.v`, `rtl/riscv_soc.v`	5-stage RV32IM processor with the integrated MAC unit, and its memories (Chapter 5)
`tb/tb_riscv_soc.v`	System testbench that runs RISC-V programs on the processor
`sw/*`	Assembler, build script, test and FIR programs, simulation runner

4.2 Micro-architecture

Figure 4.1 shows the datapath. Stage 1 decodes the instruction, extends the operands and multiplies them. Stage 2 accumulates or reads out the result and writes it back.

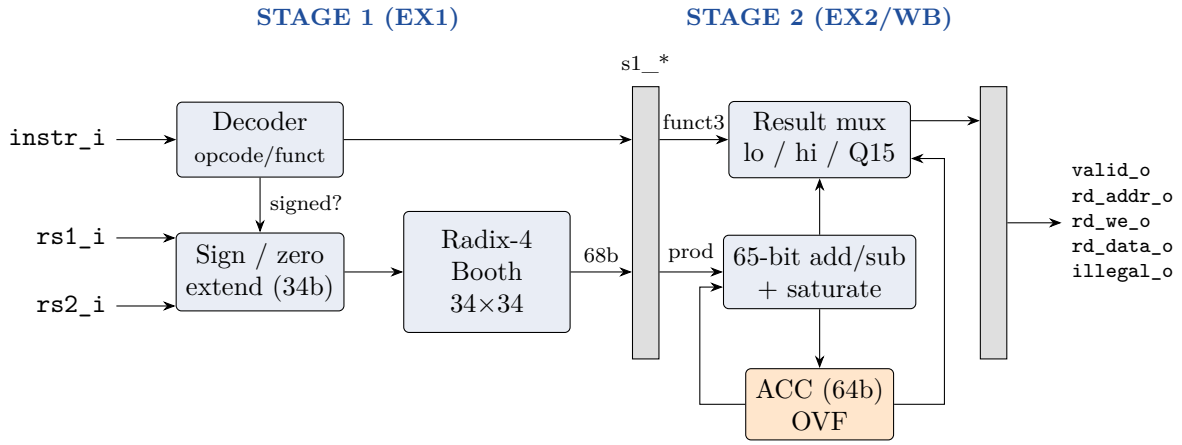


Figure 4.1: Two-stage MAC unit datapath. Grey bars are pipeline registers.

Timing behaviour. The unit sustains a throughput of one instruction per clock cycle, since `ready_o` is tied to 1, and its latency is two cycles from `valid_i` to `valid_o`. There are no hazards: every accumulator access happens in stage 2 in program order, so the results of back-to-back dependent instructions are always correct.

4.3 Booth multiplier

The multiplier is written as a behavioural but synthesizable loop that explicitly forms each Booth partial product (Table 2.2) and adds it with weight 4^i. A synthesis tool unrolls the loop into $W/2 = 17$ partial-product selectors and an adder tree.

Listing 4.1: Booth recoding loop in `booth_multiplier.v`.

```
always @* begin
    acc = {2*W{1'b0}};
    for (i = 0; i < W/2; i = i + 1) begin
        grp = b_ext[2*i +: 3];
        case (grp)                                  // Booth recoding
            3'b000, 3'b111: pp = {2*W{1'b0}};      //  0
            3'b001, 3'b010: pp =  a_ext;           // +A
            3'b011:         pp =  a_ext <<< 1;     // +2A
            3'b100:         pp = -(a_ext <<< 1);   // -2A
            3'b101, 3'b110: pp = -a_ext;           // -A
            default:        pp = {2*W{1'b0}};
        endcase
        acc = acc + (pp <<< (2*i));                // weight 4^i
    end
    p_o = acc;
end
```

4.4 Stage 1: decode and multiply

The decoder pulls the opcode, `funct3`, `funct7` and `rd` out of the instruction word. The flag `is_mac_instr` is true only when the opcode is `custom-0` and `funct7` is zero. For `macu` the operands are zero-extended; for every other operation they are sign-extended. At the end of the stage, the low 64 bits of the product, `funct3`, `rd`, the valid bit and the illegal flag are registered.

Listing 4.2: Operand extension and multiplier instantiation.

```
wire is_unsigned = (funct3 == F3_MACU);
wire [33:0] op_a = is_unsigned ? {2'b00, rs1_i} : {{2{rs1_i[31]}}, rs1_i};
wire [33:0] op_b = is_unsigned ? {2'b00, rs2_i} : {{2{rs2_i[31]}}, rs2_i};

booth_multiplier #(.W(34)) u_booth (.a_i(op_a), .b_i(op_b), .p_o(prod_full));
```

4.5 Stage 2: accumulate, saturate and read out

Stage 2 implements the overflow detection and saturation from Section 2.4, using 65-bit sign-extended addition and subtraction.

Listing 4.3: Saturating accumulation and Q15 conversion.

```
wire signed [64:0] sum_add  = acc_ext + prod_ext;
wire signed [64:0] sum_sub  = acc_ext - prod_ext;
wire ovf_add = sum_add[64] ^ sum_add[63];
wire ovf_sub = sum_sub[64] ^ sum_sub[63];
wire [63:0] sat_add = ovf_add ? (sum_add[64] ? ACC_MIN : ACC_MAX) : sum_add[63:0];
wire [63:0] sat_sub = ovf_sub ? (sum_sub[64] ? ACC_MIN : ACC_MAX) : sum_sub[63:0];

// Q30 -> Q15 (CMSIS-DSP convention: __SSAT(acc >> 15, 16))
wire signed [63:0] acc_shr15 = acc_q >>> 15;
wire [15:0] q15_val = (acc_shr15 >  64'sd32767) ? 16'h7FFF :
                      (acc_shr15 < -64'sd32768) ? 16'h8000 : acc_shr15[15:0];
```

A combinational `case` statement on `funct3` chooses the next accumulator value, the next OVF value and the result. On the clock edge these are registered together with `valid_o`, `rd_addr_o`, `rd_we_o = valid & ~illegal & (rd != 0)` and `illegal_o`. The complete code is listed in Appendix A.

4.6 Coding guidelines followed

The RTL uses only synthesizable Verilog-2005 constructs, with no delays, no initial blocks and no vendor primitives. Sequential logic uses non-blocking assignments, whereas combinational logic uses `always @*` with default assignments, so that no latches are inferred. Reset is synchronous and active low, and all encodings are named `localparam` constants. Every file starts with a header describing its purpose, ports and parameters, and the code is commented inline. Compiling with `iverilog -Wall` produces no warnings.

5 RISC-V Pipelined Processor Integration

A MAC unit is only useful inside a processor that fetches, decodes and issues its instructions. To show this, the MAC unit was built into a complete **5-stage pipelined RISC-V processor** (`rtl/rv32_core.v`, 638 lines) that implements RV32I, the multiply subset of RV32M, and the eight MAC instructions of Chapter 3. The processor runs real RISC-V machine code produced by a purpose-written assembler. Its results are checked bit-exactly in simulation (Chapter 8) and it was synthesised and placed-and-routed on an FPGA (Chapter 9).

5.1 Pipeline organisation

Figure 5.1 shows the datapath. It is the classic five-stage RISC pipeline (IF, ID, EX, MEM, WB) described by Patterson and Hennessy [13], extended with the MAC unit.

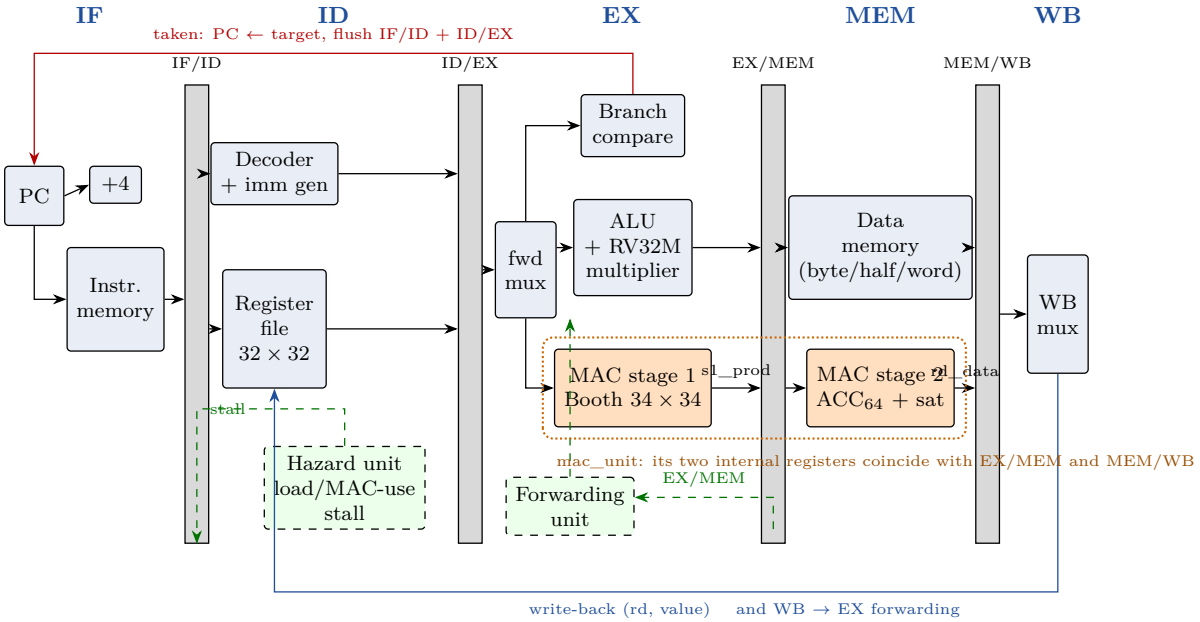


Figure 5.1: 5-stage RV32IM pipeline with the integrated MAC unit. Grey bars are pipeline registers; dashed green blocks are the hazard and forwarding units; the MAC unit (orange) spans EX and MEM.

Table 5.1: Work done in each pipeline stage.

Stage	Ordinary instructions	MAC instructions
IF	PC register addresses the instruction memory; next PC = PC+4 or branch target	same
ID	decode, immediate generation, register-file read (with write-first bypass), hazard check	same; only `mac/msu/mul.acc/macu` read `rs1`/`rs2`
EX	forwarding muxes, ALU, RV32M multiplier, branch/jump resolution	forwarded operands enter **MAC stage 1**: decode + Booth multiply
MEM	load/store access with byte/half/word alignment	**MAC stage 2**: accumulate, saturate, read-out
WB	write ALU result or load data to `rd`	write the MAC result to `rd`

Why the MAC unit fits without stalls. The MAC unit has two internal pipeline registers (Chapter 4). In the processor they line up exactly with the EX/MEM and MEM/WB registers: a MAC instruction is in MAC stage 1 while it is in EX, in MAC stage 2 while it is in MEM, and its `rd_data_o` is valid when it reaches WB. The MAC unit never stalls, and the core never stalls EX, MEM or WB, so the two pipelines always move together. The testbench checks this alignment on every MAC write-back.

5.2 Hazard handling

Data hazards: forwarding. An instruction in EX can read a value that has been computed but not yet written to the register file. The forwarding unit selects each EX operand from three sources, in priority order:

$$\mathrm{op}_A = \begin{cases} \text{EX/MEM.result} & \text{if EX/MEM writes } rd = rs_1 \neq 0 \text{ and is not a load or MAC,} \\ \text{WB value} & \text{else if MEM/WB writes } rd = rs_1 \neq 0, \\ \text{ID/EX.rs1_val} & \text{otherwise,} \end{cases} \tag{5.1}$$

and the same for op_B. The register file is also bypassed in ID, so an instruction three positions behind a writer always reads the new value.

Load-use and MAC-use stalls. A load result is ready only at the end of MEM, and so is a MAC result, because it leaves MAC stage 2 at the MEM/WB boundary. An instruction that uses such a result immediately afterwards must therefore wait one cycle. The hazard unit detects

$$\begin{aligned} \mathrm{stall} = {} & \text{IF/ID.valid} \wedge \text{ID/EX.valid} \wedge (\text{ID/EX.load} \vee \text{ID/EX.mac}) \\ & \wedge \left(rd_{EX} = rs_{1,ID} \vee rd_{EX} = rs_{2,ID}\right) \wedge rd_{EX} \neq 0, \end{aligned} \tag{5.2}$$

freezes PC and IF/ID, and inserts a bubble into ID/EX. One cycle later, the value is forwarded from WB. Section 8.2 shows this happening in the simulation.

Control hazards. Branches and jumps are resolved in EX. A taken branch loads the target into the PC and turns the two younger instructions (in IF and ID) into bubbles, so it costs two cycles. ECALL/EBREAK stops instruction fetch; when it reaches WB, the core halts (`halted_o`). An illegal instruction halts the core with `error_o`.

5.3 Instruction set and system

Table 5.2: Instructions implemented by `rv32_core`.

Group	Instructions
RV32I arithmetic/logic	`ADD SUB SLL SLT SLTU XOR SRL SRA OR AND` and immediate forms `ADDI SLTI SLTIU XORI ORI ANDI SLLI SRLI SRAI`, `LUI AUIPC`
RV32I control	`JAL JALR BEQ BNE BLT BGE BLTU BGEU`
RV32I memory	`LB LH LW LBU LHU SB SH SW`, `FENCE` (no-op)
RV32I system	`ECALL EBREAK` (halt)
RV32M (multiply subset)	`MUL MULH MULHSU MULHU`, using one shared 33×33 signed multiplier
Xmac (custom-0)	`mac msu mul.acc macu rdacc.lo rdacc.hi rdacc.q15 clracc`

Division (`DIV/REM`) and CSR instructions are not implemented, since none of the DSP programs need them. `riscv_soc.v` connects the core to a 4 KiB instruction memory at address `0x0000_0000` and a 16 KiB byte-enabled data memory at `0x0001_0000`. Both are initialised by the testbench with `$readmemh`.

5.4 Software toolchain

There is no RISC-V GCC toolchain on the development machine, and a standard toolchain would not know the custom instructions anyway. So a small two-pass assembler, `sw/rvasm.py` (386 lines of Python), was written for this project. It supports all instructions in Table 5.2, the common pseudo-instructions (`li la mv j call ret beqz bnez`, ...), labels, `.equ`/`.include` and data directives. Its output encodings were checked against reference encodings from the RISC-V specification (for example `lw x5,8(x2)` → `0x00812283` and `bne x1,x2,-8` → `0xfe209ce3`). `sw/build.py` designs the test filter, computes the golden results and assembles all the programs (Table 5.3).

Table 5.3: Programs executed on the processor.

Program	Static instr.	Purpose
`isa_test`	282	18 self-checking tests: every instruction class, sign/zero extension, all branch types, call/return, RV32M, every MAC instruction, forwarding, load-use and MAC-use hazards, `x0`
`fir_mac_loop`	27	16-tap Q15 FIR over 128 samples using `mac` and `rdacc.q15`
`fir_rv32im_loop`	51	the same filter in plain RV32IM (`mul`/`mulh` + 64-bit add with carry + software saturation)
`fir_mac_unrolled`	64	MAC version with the tap loop fully unrolled
`fir_rv32im_unrolled`	166	RV32IM version with the tap loop fully unrolled

Listing 5.1 compares the two inner loops. Per tap, the MAC version needs 7 instructions where RV32IM needs 12, because one `mac` replaces the six-instruction 64-bit multiply-accumulate sequence. After the loop, the MAC version converts and saturates the result with a single `rdacc.q15`, where RV32IM needs a 14-instruction shift-and-saturate routine. Both loops are scheduled so that no load-use stall occurs.

`fir_mac_loop` (7 instr./tap)

```
inner:
  lh   t3, 0(t0)    # x
  lh   t4, 0(t1)    # h
  addi t0, t0, 2
  addi t1, t1, 2
  addi t2, t2, -1
  mac  x0, t3, t4   # ACC += x*h
  bnez t2, inner
  rdacc.q15 t5      # Q30->Q15, sat
```

`fir_rv32im_loop` (12 instr./tap)

```
inner:
  lh   t3, 0(t0)
  lh   t4, 0(t1)
  addi t0, t0, 2
  addi t1, t1, 2
  mul  a2, t3, t4    # product low
  mulh a3, t3, t4    # product high
  addi t2, t2, -1
  add  s4, s4, a2    # 64-bit add
  sltu a4, s4, a2    #   carry
  add  s5, s5, a3
  add  s5, s5, a4
  bnez t2, inner
  ... 14 instr.: acc>>>15 + saturate
```

Listing 5.1: FIR inner loops with and without the MAC unit.

5.5 DSP test case: low-pass FIR filter

The benchmark is a realistic filtering task. A 16-tap low-pass filter (windowed sinc, Hamming window, cut-off 0.1 cycles/sample, unity DC gain) is quantised to Q15:

$h = \{-114, -159, -139, 291, 1450, 3284, 5246, 6524, 6524, 5246, 3284, 1450, 291, -139, -159, -114\}$.

The input contains 128 Q15 samples of a wanted low-frequency tone ($0.5\sin(2\pi\,0.015\,n)$) plus an unwanted high-frequency tone ($0.3\sin(2\pi\,0.3\,n)$). `build.py` computes the bit-exact expected output $y[n] = \mathrm{sat}_{16}((\sum_k h[k]x[n-k]) \ggg 15)$ in Python (independently of the hardware) and writes it to `build/fir_golden.hex` for the testbench.

6 Verification Methodology

6.1 Testbench architecture

The system testbench `tb_mac_unit` uses the *reference model + scoreboard* structure shown in Figure 6.1.

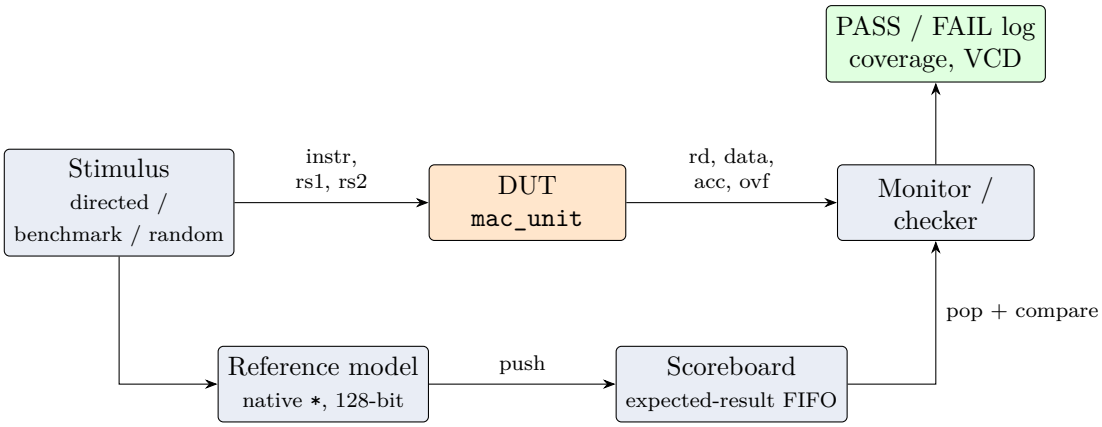


Figure 6.1: Self-checking testbench architecture.

Independent reference model. The reference model is intentionally implemented in a different way from the RTL, so that a shared misunderstanding cannot hide a bug. It multiplies with the simulator's native `*` operator instead of Booth recoding, and it detects saturation by computing a 128-bit sum and comparing it with INT64_MAX and INT64_MIN, instead of using the 65-bit sign-bit XOR.

Checks on every result. For each issued instruction the driver calls the model and pushes six expected values into the scoreboard: `rd` index, write enable, illegal flag, rd data, the 64-bit accumulator and OVF. On every `valid_o`, the monitor pops one entry and compares all six values, and also checks that exactly two clock cycles have passed since the instruction was driven.

Analytical golden values. The model is not the only reference. The benchmark tests (T6, T10, T11, T12) also compare the DUT output with values worked out by hand from the mathematics of the problem.

6.2 Test plan

Table 6.1: Test plan of the system testbench.

ID	Test	Purpose / stimulus
T1	Reset state	Accumulator reads 0 after reset
T2	Basic arithmetic	`mul.acc`, `mac`, `msu` issued back-to-back
T3	Sign combinations	$(\pm a)(\pm b)$; sign extension of the high word
T4	Extreme operands	INT32_MAX^2, INT32_MIN^2, MIN·MAX, (-1)·MIN$= +2^{31}$
T5	Saturation	Positive and negative 64-bit saturation; sticky OVF; clearing
T6	Unsigned `macu`	UINT32_MAX^2, modulo-2^{64} wrap; $\sum_{i=1}^{10} i^2 = 385$
T7	Destination `x0`	No write-back, but the accumulator updates
T8	Illegal instructions	RV32M `MUL`, wrong `funct7`, `NOP`
T9	Q15 read-out	$0.5 \cdot 0.5$, $-0.5 \cdot 0.5$, $-1 \cdot -1$ (saturates), ±saturation
T10	Dot product	$[1..8] \cdot [8..1] = 120$; throughput measurement
T11	Moving-average FIR	8 taps, step input; analytical ramp response
T12	Low-pass FIR	Triangular 8-tap filter; DC + Nyquist input
T13	Constrained random	3000 instructions, operands biased to corners, ∼5 % illegal, random bubbles

6.3 Multiplier unit test

`tb_booth_multiplier` instantiates the multiplier twice. The 8-bit instance is checked *exhaustively* against `*` for all $2^{16} = 65{,}536$ operand pairs. The algorithm does not depend on the width, so this is strong evidence that the recoding logic is correct. The 34-bit instance, which is the one used in the MAC, is checked on 9 corner cases (including UINT32_MAX^2 and the 34-bit extremes) and 20,000 random pairs.

6.4 Constrained-random stimulus

In T13, each operand is drawn from a biased distribution: 10 % each for 0, 1, −1, INT32_MAX and INT32_MIN, plus small signed values, 16-bit values and full-range random values. `funct3` and `rd` are uniformly random. About 5 % of the instructions are deliberately illegal, and about 25 % of the instructions are followed by 1–3 idle cycles, so the test exercises both back-to-back and gapped issue. A fixed seed makes the regression reproducible.

6.5 Mutation test (checking the checker)

A testbench that never fails proves nothing. To confirm that this one detects real bugs, a copy of `mac_unit.v` was made with saturation disabled (`sat_add` hard-wired to the wrapped sum) and simulated with the same testbench. The testbench reported FAIL on the T5 saturation tests and in the random regression, as Listing 6.1 shows.

Listing 6.1: Testbench output for the mutated design (excerpt).

```
[FAIL]          mac +2^62 -> 2^63 SAT to INT64_MAX
[FAIL]          mac  (stays saturated, OVF sticky)
[FAIL]        rdacc.hi               -> 0x7FFFFFFF
[FAIL]       mac  -> below -2^63, SAT to INT64_MIN
[FAIL]        rdacc.hi               -> 0x80000000
[FAIL]                                    random
```

6.6 System-level verification

The unit-level testbenches above drive the MAC unit directly. The processor of Chapter 5 is verified at system level by `tb/tb_riscv_soc.v`, which runs complete RISC-V programs: a self-checking ISA and hazard test (`isa_test`, 18 tests), and four versions of a 16-tap FIR filter whose 128 outputs are compared bit-exactly against golden values computed independently in Python. The testbench also checks the MAC write-back alignment on every MAC instruction and records a per-cycle pipeline trace. The results are in Chapter 8.

7 MAC Unit Simulation Results

All simulations were run with Icarus Verilog 12 (`iverilog -g2005 -Wall`, `vvp`). The full terminal log is reproduced in Appendix D.

7.1 Summary

Table 7.1: Verification summary.

Testbench	Checks	Errors	Result
`tb_booth_multiplier` (8-bit exhaustive + 34-bit)	85,545	0	**PASS**
`tb_mac_unit` (T1–T13, 3,328 instructions)	3,351	0	**PASS**
`tb_mac_unit` on the Yosys gate-level netlist (Ch. 9)	3,351	0	**PASS**

7.2 Directed test results

Table 7.2 shows a selection of directed results taken from the simulation log. Each one matched the reference model on all six compared fields and on latency.

Table 7.2: Selected directed test results (DUT output = expected).

Test	Instruction	rd_data_o	acc_o	OVF
T2	`mul.acc 7, 6`	42	`0x000000000000002A`	0
T2	`mac 3, -4`	30	`0x000000000000001E`	0
T2	`msu -5, 10`	80	`0x0000000000000050`	0
T2	`mac 100, 1000`	100080	`0x00000000000186F0`	0
T3	`mul.acc 12, -13`	-156	`0xFFFFFFFFFFFFFF64`	0
T4	`mul.acc MAX, MAX`	`0x00000001`	`0x3FFFFFFF00000001`	0
T4	`mul.acc MIN, MAX`	`0x80000000`	`0xC000000080000000`	0
T5	`mac MIN, MIN` (sat.)	`0xFFFFFFFF`	`0x7FFFFFFFFFFFFFFF`	1
T5	`mac MIN, MAX` (sat.)	`0x00000000`	`0x8000000000000000`	1
T5	`mul.acc 2, 3`	6	`0x0000000000000006`	0
T6	`macu 0xFFFFFFFF`^{2}	`0x00000001`	`0xFFFFFFFE00000001`	0
T6	`macu` (again, wraps)	`0x00000002`	`0xFFFFFFFC00000002`	0
T6	$\sum_{i=1}^{10} i^2$	385	`0x0000000000000181`	0
T8	RV32M `MUL` (illegal)	0, ILLEGAL	unchanged (`0x64`)	0
T9	`rdacc.q15` after $0.5 \cdot 0.5$	8192 (0.25)	`0x0000000010000000`	0
T9	`rdacc.q15` after $-1 \cdot -1$	32767 (sat.)	`0x0000000040000000`	0
T9	`rdacc.q15` after ≈ -2.0	-32768 (sat.)	`0xFFFFFFFF80010000`	0
T10	dot product $[1..8] \cdot [8..1]$	120	`0x0000000000000078`	0

7.3 Simulation waveforms

Figures 7.1 and 7.2 were generated from the VCD dump (`sim/mac_unit.vcd`) produced by the testbench.

In Figure 7.1, six instructions are issued in consecutive cycles starting at 80 ns. The product of each instruction shows up in `s1_prod` one cycle after issue (42, -12, -50, 100000), and the accumulated result shows up on `rd_data_o`/`acc_o` one cycle after that (42, 30, 80, 100080). This confirms the two-cycle latency and the throughput of one instruction per cycle without stalls.

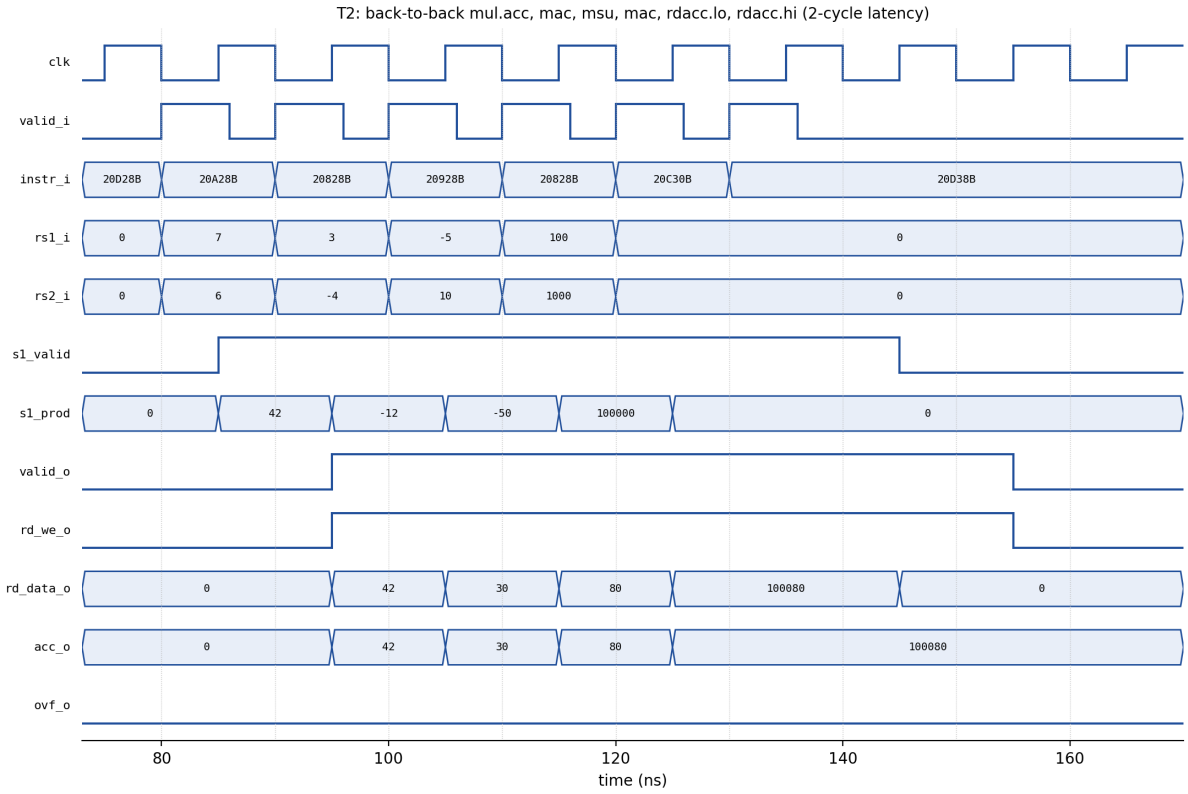


Figure 7.1: Simulation waveform for T2: back-to-back MAC instructions.

In Figure 7.2, `mul.acc MIN,MIN` loads 2^{62} (`0x4000000000000000`). The next `mac MIN,MIN` would give 2^{63}, which is not representable, so the accumulator saturates to `0x7FFFFFFFFFFFFFFF` and `ovf_o` goes high. It stays high during the following `mac` and `rdacc.hi`, and is cleared by `clracc` at 415 ns.

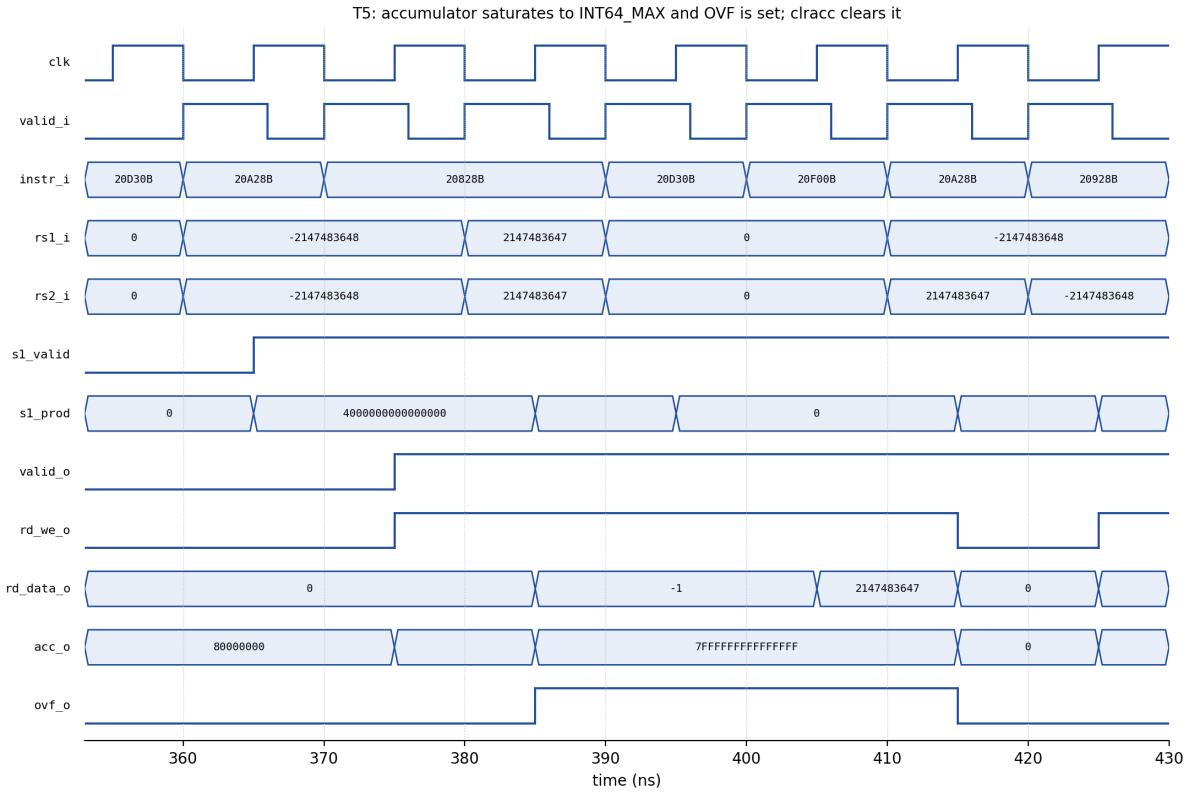


Figure 7.2: Simulation waveform for T5: positive saturation and the sticky overflow flag.

7.4 DSP benchmark 1: moving-average FIR

An 8-tap moving-average filter with $h[k] = 4096$ (0.125 in Q15) is driven by a step of amplitude 16384 (0.5). The analytical output is $y[n] = 2048(n+1)$ for $n < 8$ and 16384 afterwards. Each output sample is computed by the instruction sequence `mul.acc`, seven `mac`, and `rdacc.q15`. All 12 DUT outputs exactly match the analytical values (Table 7.3, Figure 7.3a).

Table 7.3: Moving-average FIR step response.

n	0	1	2	3	4	5	6	7	8–11
DUT $y[n]$	2048	4096	6144	8192	10240	12288	14336	16384	16384
Analytical	2048	4096	6144	8192	10240	12288	14336	16384	16384

7.5 DSP benchmark 2: triangular low-pass FIR

The symmetric filter $h = \{1024, 3072, 5120, 7168, 7168, 5120, 3072, 1024\}$ has a DC gain of $\sum h[k]/2^{15} = 1$. Its gain at the Nyquist frequency is zero:

$$H(e^{j\pi}) = \sum_k h[k](-1)^k = 1024 - 3072 + 5120 - 7168 + 7168 - 5120 + 3072 - 1024 = 0. \quad (7.1)$$

The input $x[n] = 8000 + 6000(-1)^n$, a DC level plus a Nyquist-frequency tone, must therefore give exactly $y[n] = 8000$ once the 8-sample window is full ($n \geq 7$). The DUT produces exactly 8000 for every steady-state sample: all 9 checks pass (Table 7.4, Figure 7.3b). The tone is removed completely.

Table 7.4: Low-pass FIR response to DC + Nyquist input.

n	0	1	2	3	4	5	6	7	8–15
$x[n]$	14000	2000	14000	2000	14000	2000	14000	2000	alt.
DUT $y[n]$	437	1375	2812	4750	6312	7375	7937	**8000**	**8000**

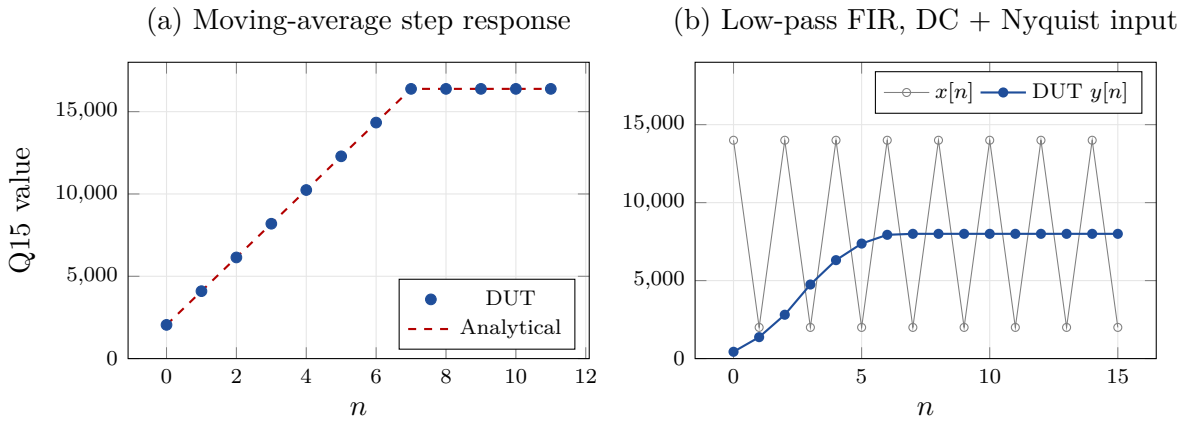


Figure 7.3: FIR benchmark results computed by the MAC unit.

7.6 Functional coverage

Table 7.5: Functional coverage collected over the full regression.

Coverage item	Count	Coverage item	Count
`mac`	570	`rdacc.hi`	386
`msu`	338	`rdacc.q15`	393
`mul.acc`	378	`clracc`	374
`macu`	381	Illegal instructions	162
`rdacc.lo`	346	Saturation events	7 (+ directed)
`rd = x0`	360	Idle-gap insertions	684

Every instruction, the illegal-instruction path, saturation in both directions, `x0` suppression, and both back-to-back and gapped issue were exercised many times.

7.7 Performance analysis

Measured. In T10, nine instructions (eight multiply-accumulates and one read) were issued in nine consecutive cycles, i.e. one instruction per cycle. An N-tap FIR therefore costs $N + 1$ MAC-unit instructions per output sample: 9 for the 8-tap filters in this report.

Comparison with RV32IM (instruction count estimate). On a base RV32IM core, adding a full 32×32-bit signed product into a 64-bit sum requires `mul` (low word), `mulh` (high word), `add` (low), `sltu` (carry), `add` (high) and `add` (carry): six instructions, with no saturation. The `mac` instruction does the same work, plus saturation, in one instruction. Table 7.6 compares the two for one output of an 8-tap FIR. These figures count only the arithmetic instructions and exclude loads and loop overhead, which are the same for both. Chapter 8 replaces this estimate with *measured* cycle counts from complete programs running on the 5-stage processor.

Table 7.6: Arithmetic instruction count per output of an 8-tap FIR (estimate).

Implementation	Per tap	Per output (8 taps)	Saturation
RV32IM, 64-bit accumulation	6	48 + conversion	no
RV32IM, 32-bit accumulation (Q15 only)	2	16 + conversion	no
This MAC unit	1	8 + 1 (`rdacc.q15`)	yes

8 System-Level Simulation Results

The processor was simulated in Icarus Verilog with the system testbench `tb/tb_riscv_soc.v`. For each program, the testbench loads the program and data images and runs the core until ECALL. It then checks that `a0` = `0x600D`, the success code of the self-checking programs, and compares every output sample with the golden file, bit by bit. In addition, on every MAC write-back it checks that the MAC unit's `valid_o`/`rd_addr_o` line up with the core's MEM/WB register. Finally, it reports the core's hardware performance counters, writes a per-cycle pipeline trace (CSV) and, optionally, a VCD waveform. The whole flow (build, compile, run, collect) is automated by `sw/run_cpu_sims.py`.

8.1 Results summary

Table 8.1: Programs executed on the 5-stage core (all results from the hardware performance counters).

Program	Result	Cycles	Instr.	CPI	Stalls	Flushes	MAC instr.
`isa_test`	**PASS**	348	300	1.160	8	18	22
`fir_mac_loop`	**PASS**	19,724	15,626	1.262	0	2,047	2,304
`fir_rv32im_loop`	**PASS**	31,363	26,895	1.166	0	2,232	0
`fir_mac_unrolled`	**PASS**	7,180	6,922	1.037	0	127	2,176
`fir_rv32im_unrolled`	**PASS**	19,203	18,575	1.034	0	312	0

The `isa_test` program passed all 18 self-checks (`a0 = 0x600D`). Its 8 stall cycles are the deliberate load-use and MAC-use hazards in tests T7, T8, T12, T13 and T14, and every one was resolved correctly. All four FIR programs produced **128/128 output samples bit-identical** to the golden model, and no MAC write-back was ever misaligned with the core pipeline.

Mutation testing of the processor. To show that the system tests can find real pipeline bugs, two faulty copies of the core were simulated with the same programs (Table 8.2). Both bugs are caught by `isa_test`. The FIR programs still pass, because they were scheduled to avoid hazards, which is exactly why a dedicated ISA/hazard test is needed in addition to application benchmarks.

Table 8.2: Processor mutation tests.

Injected fault	`isa_test`	`fir_mac_loop`
WB→EX forwarding of operand A disabled	**FAIL** at T1 (`a0`=1)	pass
Load-use / MAC-use stall disabled	**FAIL** at T7 (`a0`=7)	pass

8.2 Pipeline behaviour

The pipeline diagrams below were *reconstructed from the simulation trace*, not drawn by hand. Each row is one dynamic instruction; each column is one clock cycle. Colours mark the stage the instruction occupies; orange MAC1/MAC2 cells show MAC instructions in the MAC unit's two stages; hatched cells are stall cycles; *fwd* marks operands forwarded from MEM or WB; grey rows are wrong-path instructions squashed by a taken branch.

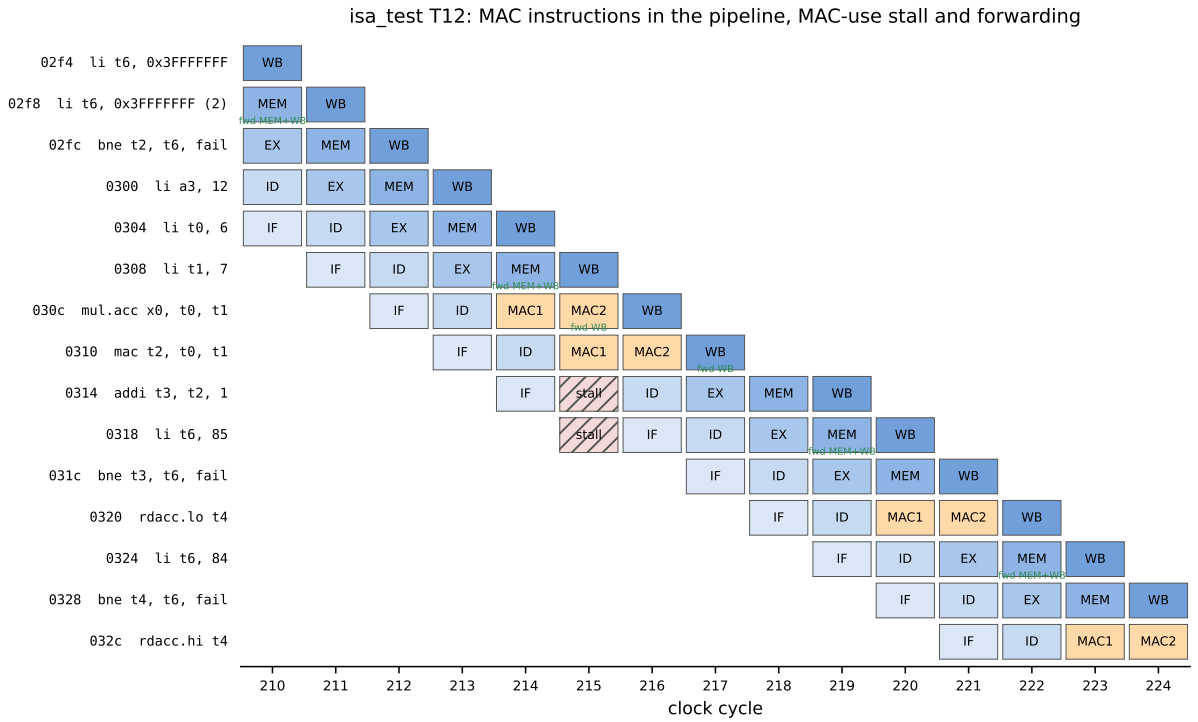


Figure 8.1: `isa_test` T12 executing: `mul.acc`, `mac` and the dependent `addi t3,t2,1`. The `addi` is held in ID for one cycle (MAC-use stall) and then receives `t2` from WB; the result is 85 as expected.

In Figure 8.1, `mul.acc` and `mac` occupy MAC1/MAC2 in consecutive cycles without any interference, because the accumulator is read and written only in MAC stage 2. The instruction that consumes the MAC result is stalled once by the hazard unit and then gets the value forwarded from WB. This is exactly the behaviour designed in Chapter 5.

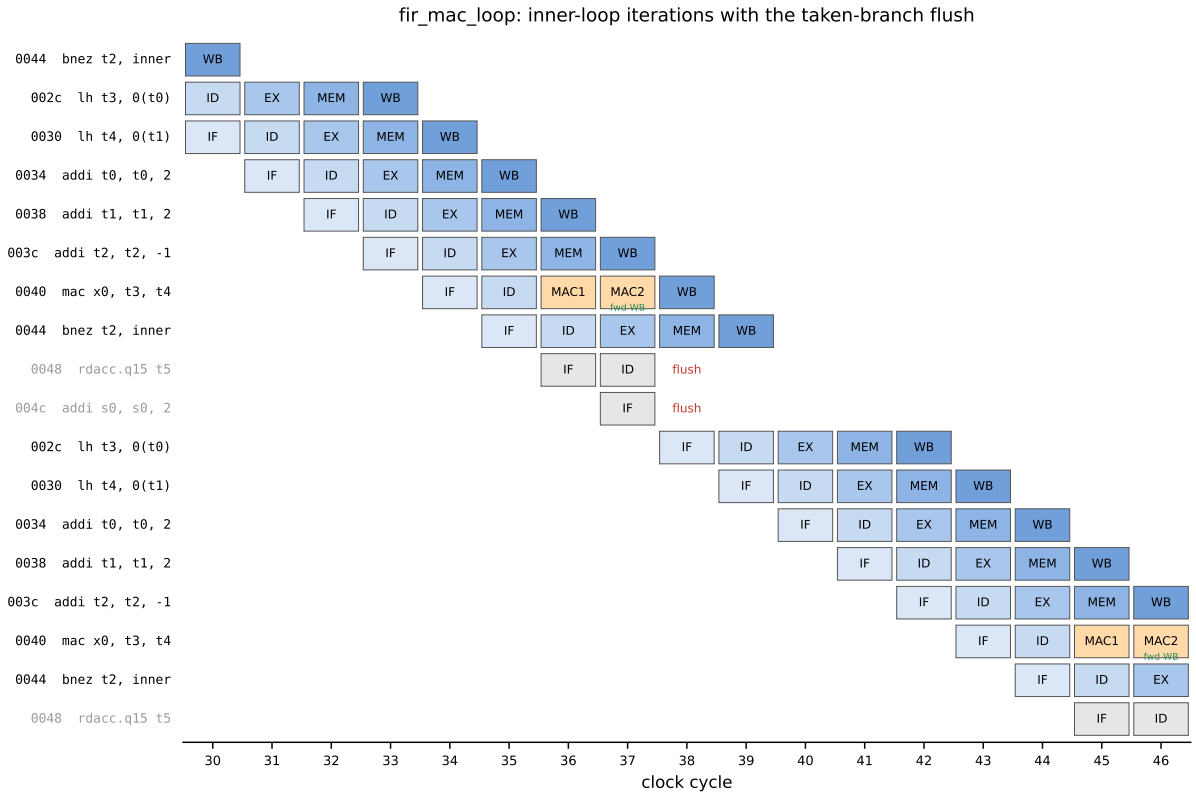


Figure 8.2: `fir_mac_loop`: two iterations of the inner loop. Every iteration ends with a taken `bnez`, which squashes the two instructions fetched behind it (2-cycle penalty).

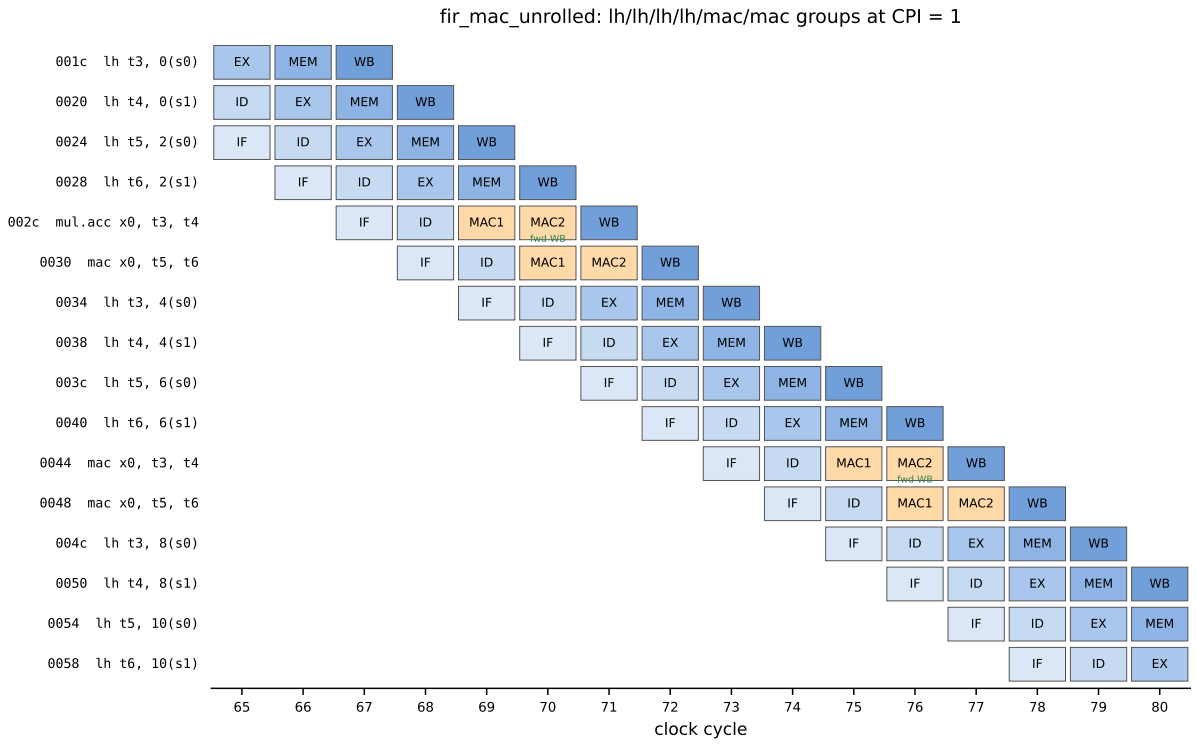


Figure 8.3: `fir_mac_unrolled`: the unrolled `lh`/`lh`/`lh`/`lh`/`mac`/`mac` pattern issues one instruction per cycle with no stalls or flushes.

Figure 8.2 explains why the looped FIR has a CPI of 1.26: the taken branch at the end of each 7-instruction iteration wastes 2 of every 9 cycles. Unrolling (Figure 8.3) removes the branches from the tap loop. The pipeline then reaches CPI 1.037, and the MAC unit receives a new operand pair every third cycle (`lh`, `lh`, `mac`).

8.3 Processor waveform

Figure 8.4 shows the VCD waveform of `fir_mac_loop` around the end of output sample $y[2]$. The first five rows show which instruction is in each pipeline stage. Note the bubbles after the taken `bnez` and the `mac` moving from EX to MEM to WB. The orange rows show the MAC unit itself. The `valid_i` signal is high while a `mac` is in EX, and its operands $x[n]$ and $h[k]$ (for example -2708 and -114) arrive through the forwarding network. One cycle later, the Booth product appears in `s1_prod` ($-2708 \times -114 = 308{,}712$), and one cycle after that the accumulator updates ($-1{,}731{,}669 + 308{,}712 = -1{,}422{,}957$). After the loop exits, `rdacc.q15` writes $\lfloor -1{,}422{,}957/2^{15} \rfloor = -44$ to register `t5` (x30), and the following `sh` stores it as $y[2]$. This value matches the golden model.

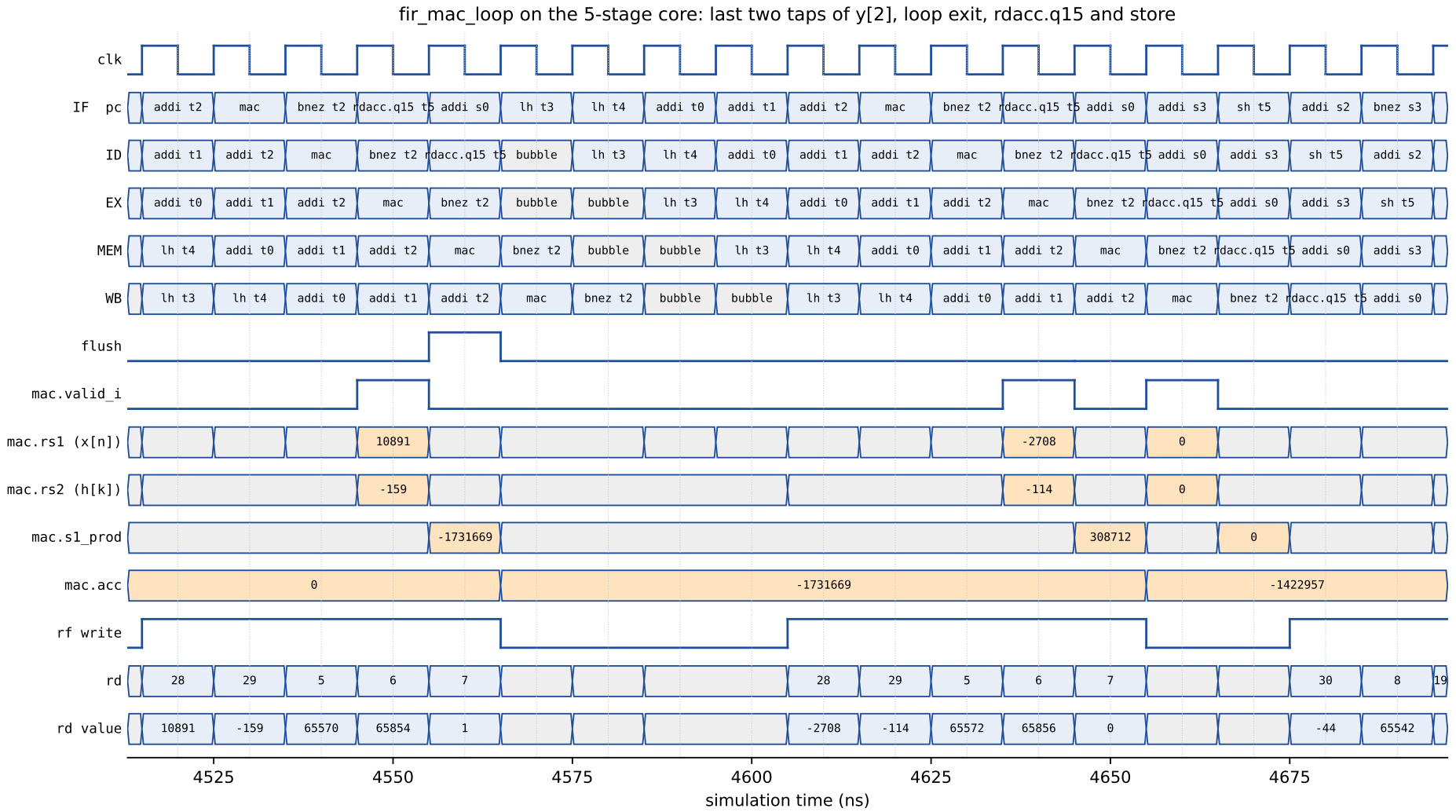


Figure 8.4: Simulation waveform of the 5-stage core executing `fir_mac_loop` (from `sim/cpu/fir_mac_loop.vcd`). Pipeline-stage rows show the instruction in each stage (grey = bubble); orange rows are MAC-unit signals; bus values are blanked while their valid signal is low.

8.4 DSP results: the FIR filter on the processor

Figure 8.5 shows the input signal and the output that the processor computed with the MAC instructions. The high-frequency tone is removed and the wanted tone passes through, delayed by the filter's group delay of 7.5 samples. The lower panel shows two error curves. The difference from the golden model is **exactly zero** for all 128 samples, while the difference from an ideal floating-point filter is at most 2.5 LSB (1.2 LSB rms), which is just the Q15 quantisation of the taps and the signal.

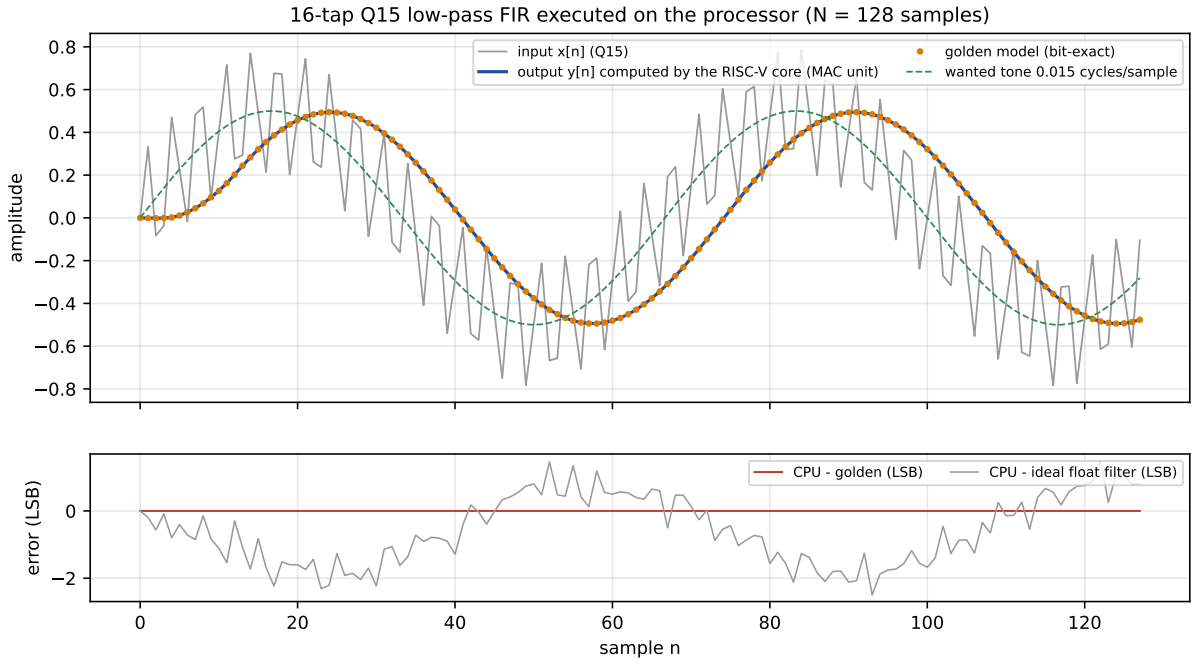


Figure 8.5: FIR filtering executed on the RISC-V core: input, CPU output and golden model (top), and the error against the golden model and against an ideal floating-point filter (bottom).

The spectra in Figure 8.6 quantify the result. The unwanted tone at 0.3 cycles/sample drops from −11.3 dBFS to −71.4 dBFS, an attenuation of **60.1 dB**, which agrees with the filter's theoretical response $|H(0.3)| = -60.2$ dB (Figure 8.7). The wanted tone at 0.015 cycles/sample, by contrast, changes by only −0.1 dB.

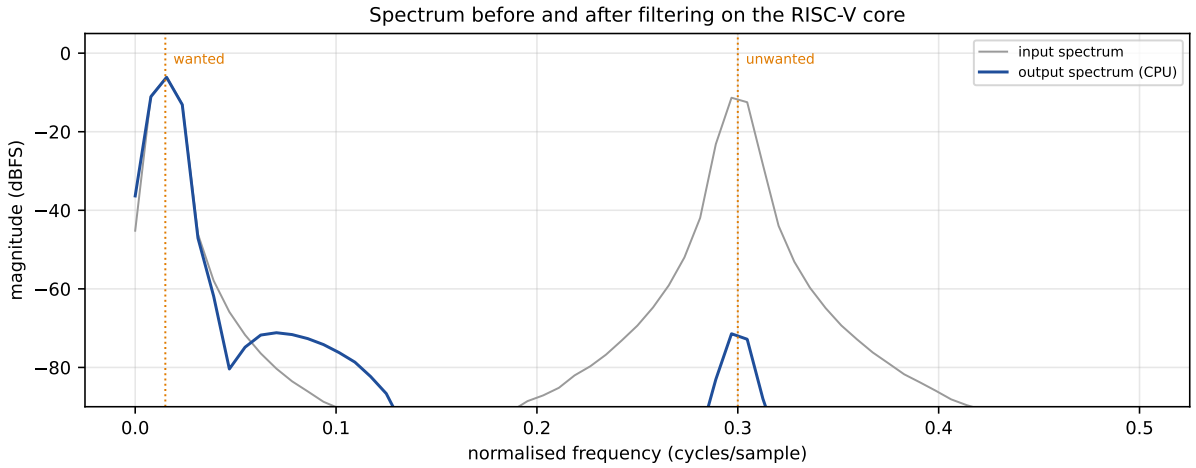


Figure 8.6: Spectrum of the input and of the output computed by the processor.

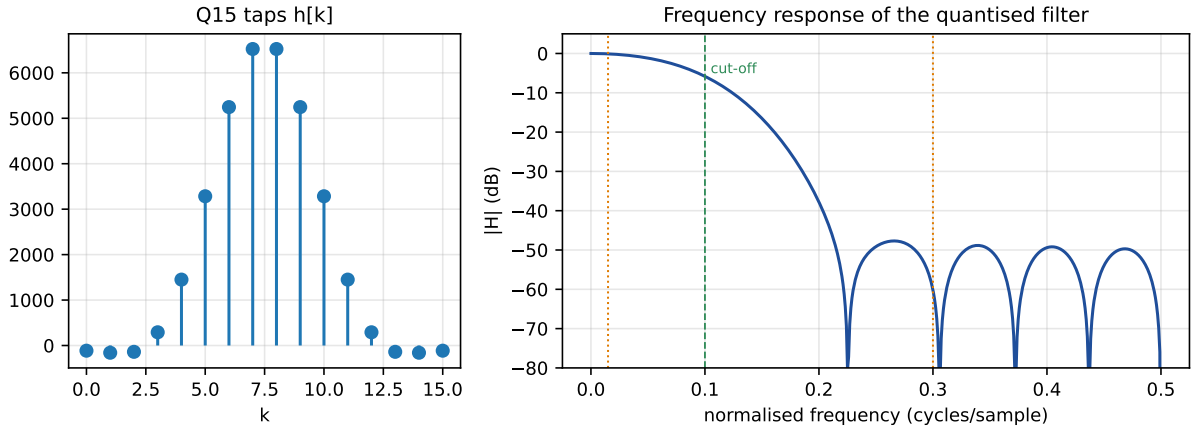


Figure 8.7: Quantised Q15 filter taps and frequency response.

8.5 Performance: MAC unit vs. plain RV32IM

Because both versions of the filter run on the same core and produce the same bit-exact output, their cycle counts can be compared directly (Figure 8.8, Table 8.3).

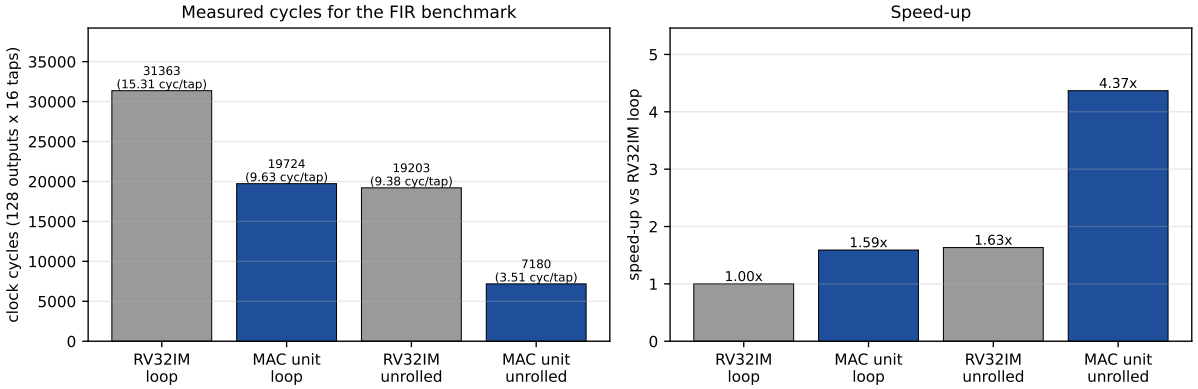


Figure 8.8: Measured FIR execution time (128 outputs × 16 taps) and speed-up.

Table 8.3: FIR benchmark: MAC instructions vs. RV32IM software (measured).

Version	Cycles	Cycles/tap	Instructions	Code size	Speed-up
RV32IM loop	31,363	15.31	26,895	51 instr.	1.00×
MAC loop	19,724	9.63	15,626	27 instr.	**1.59**×
RV32IM unrolled	19,203	9.38	18,575	166 instr.	1.63×
MAC unrolled	7,180	3.51	6,922	64 instr.	**4.37**×
MAC vs. RV32IM, same code style	loop: **1.59**× faster;		unrolled: **2.67**× faster, 2.6× smaller code		

Figure 8.9 shows where the cycles go, measured with the performance counters. With the unrolled MAC code, 96 % of all cycles retire a useful instruction, whereas in the looped versions 14–21 % of the cycles are lost to branch flushes. The scheduled FIR code, on the other hand, has no load-use or MAC-use stalls at all. At the processor's post-route clock frequency of 34.6 MHz on the ECP5 FPGA (Chapter 9), the unrolled MAC version filters one output sample in 56 cycles (1.6 μs, about 616,000 samples/s). The unrolled RV32IM version needs 150 cycles (4.3 μs).

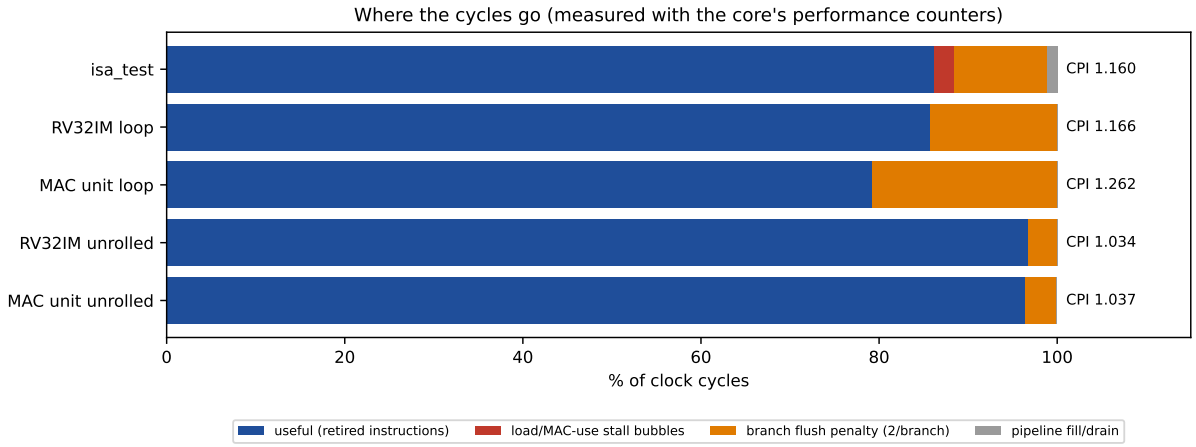


Figure 8.9: Cycle breakdown and CPI of every program.

9 Logic Synthesis and FPGA Implementation

The RTL was synthesised with the open-source **Yosys 0.70** synthesis suite [11] and placed and routed with **nextpnr-ecp5** [12]. Four flows were used (scripts in Appendix C). The first flow produces the RTL schematics: Yosys elaborates the design, and netlistsvg [14] draws the resulting netlist. The second flow is generic (ASIC-style) gate-level synthesis, in which technology mapping with ABC targets 2-input gates and D flip-flops, applied both to the MAC unit and to the complete processor. The third flow is FPGA implementation on a Lattice ECP5 LFE5U-85F (`synth_ecp5` + nextpnr place & route, speed grade 6), carried out for the MAC unit with the Booth multiplier, for the MAC unit with hard DSP multipliers as a comparison, and for the complete RV32IM core with the MAC unit. The fourth flow is post-synthesis gate-level simulation of the synthesised netlist.

9.1 RTL schematics generated by Yosys

Figure 9.1 shows the radix-4 Booth multiplier as Yosys elaborates it. A small instance ($W = 6$) is used so that the structure stays readable. Each of the three 3-bit groups of B is decoded by `$eq` comparators that drive a Booth-recoding multiplexer (`$pmux`) choosing $0, \pm A, \pm 2A$. Yosys then merged the weighted partial-product additions into multiply-accumulate (`$macc`) and adder (`$alu`) cells, exactly as described in Table 2.2. Figure 9.2 shows the same circuit after technology mapping: 216 two-input gates. Figure 9.3 shows the complete `mac_unit`: the Booth multiplier appears as one block, surrounded by the decoder, the stage-1 and stage-2 registers (`$dff`/`$sdff`), the 65-bit adder and subtractor, the saturation comparators and the result multiplexers.

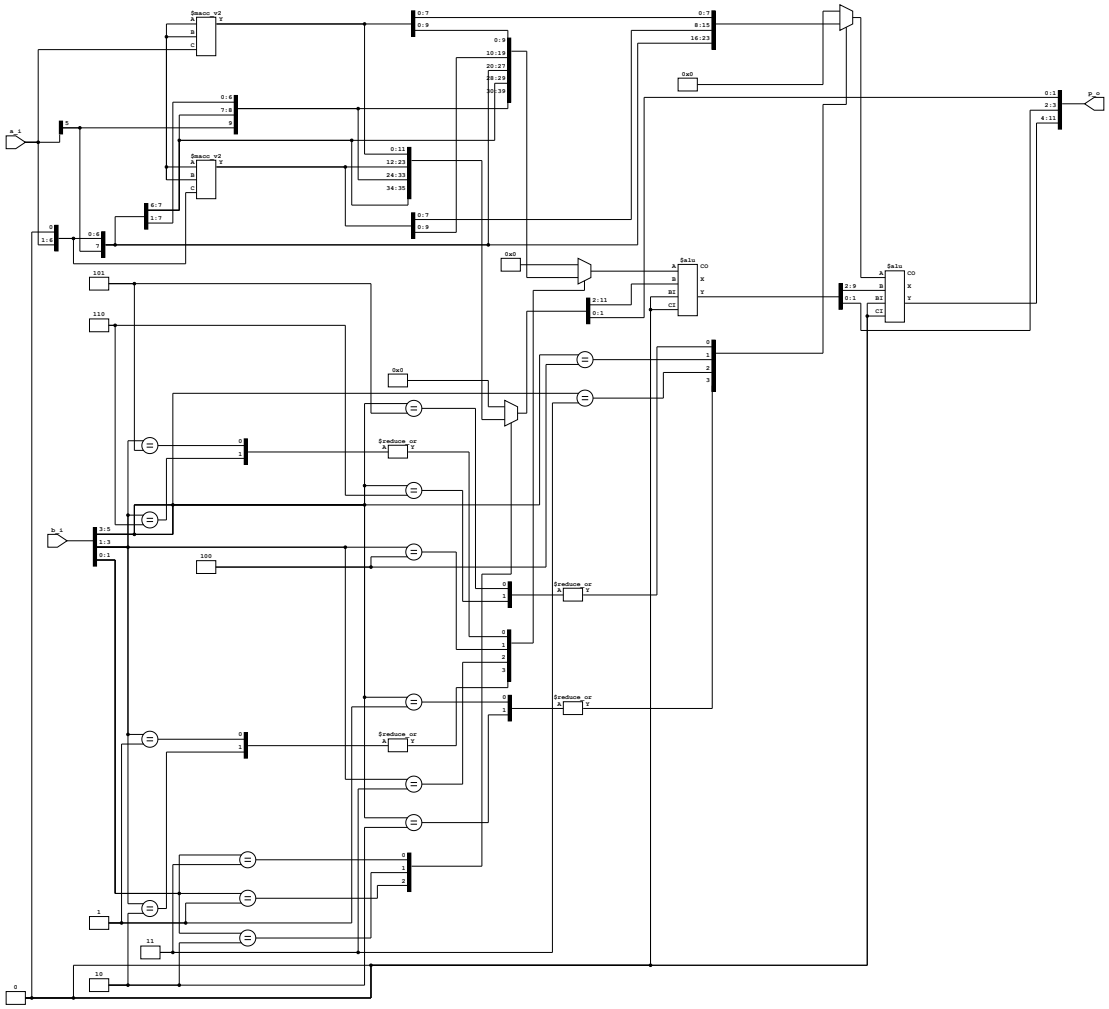


Figure 9.1: Yosys RTL schematic of `booth_multiplier` with $W = 6$ (word-level cells: `$eq` group decoders, Booth-recoding `$pmux`, partial-product `$macc`/`$alu`).

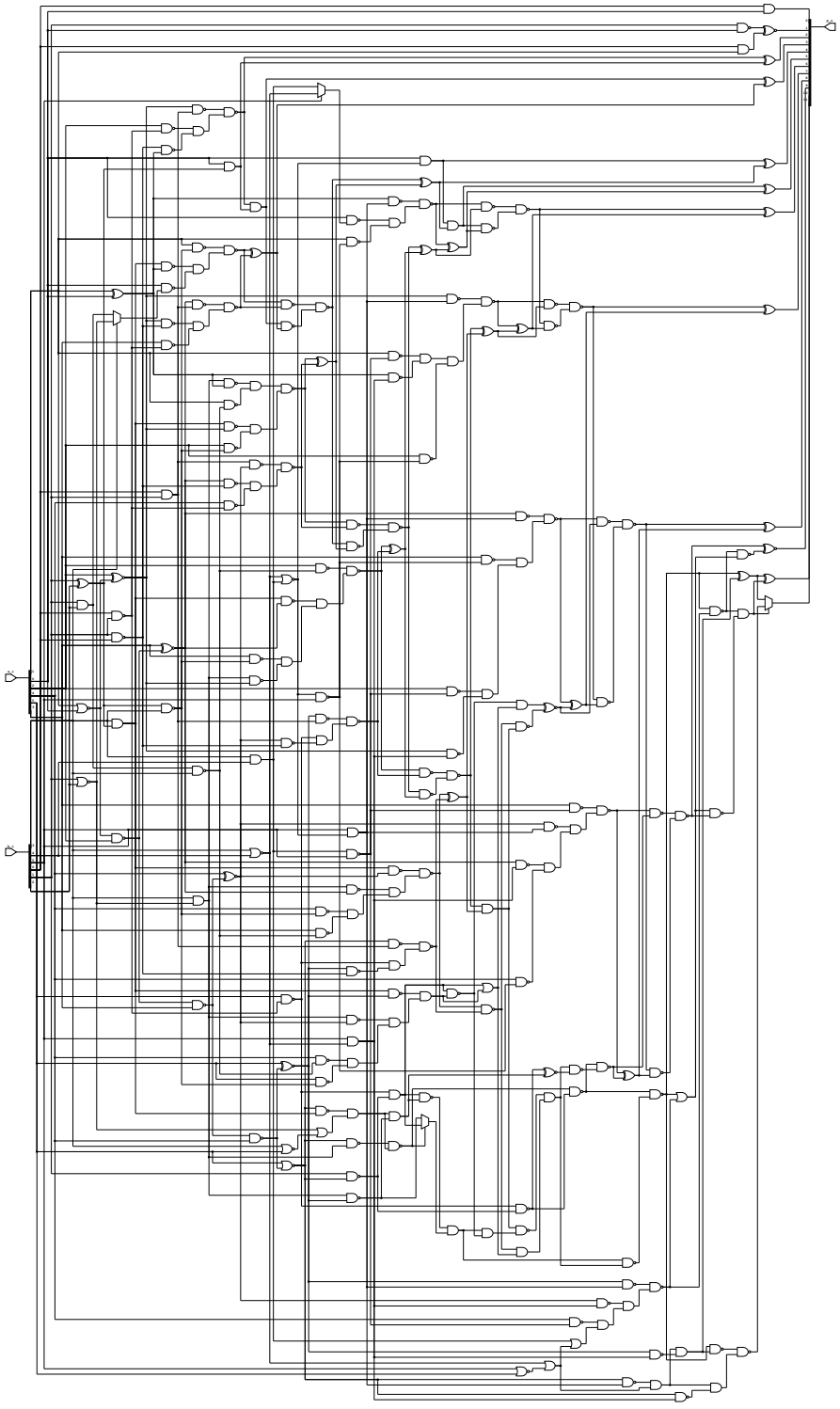


Figure 9.2: The same 6-bit Booth multiplier after gate-level synthesis (216 gates).

9.2 Generic gate-level synthesis of the MAC unit

The flow `synth -flatten` followed by `abc -g AND,NAND,OR,NOR,XOR,XNOR,ANDNOT,ORNOT,MUX` produces the netlist summarised in Table 9.1 and Figure 9.4. `check -assert` reports no combinational loops, no multiple drivers, no undriven nets and no inferred latches.

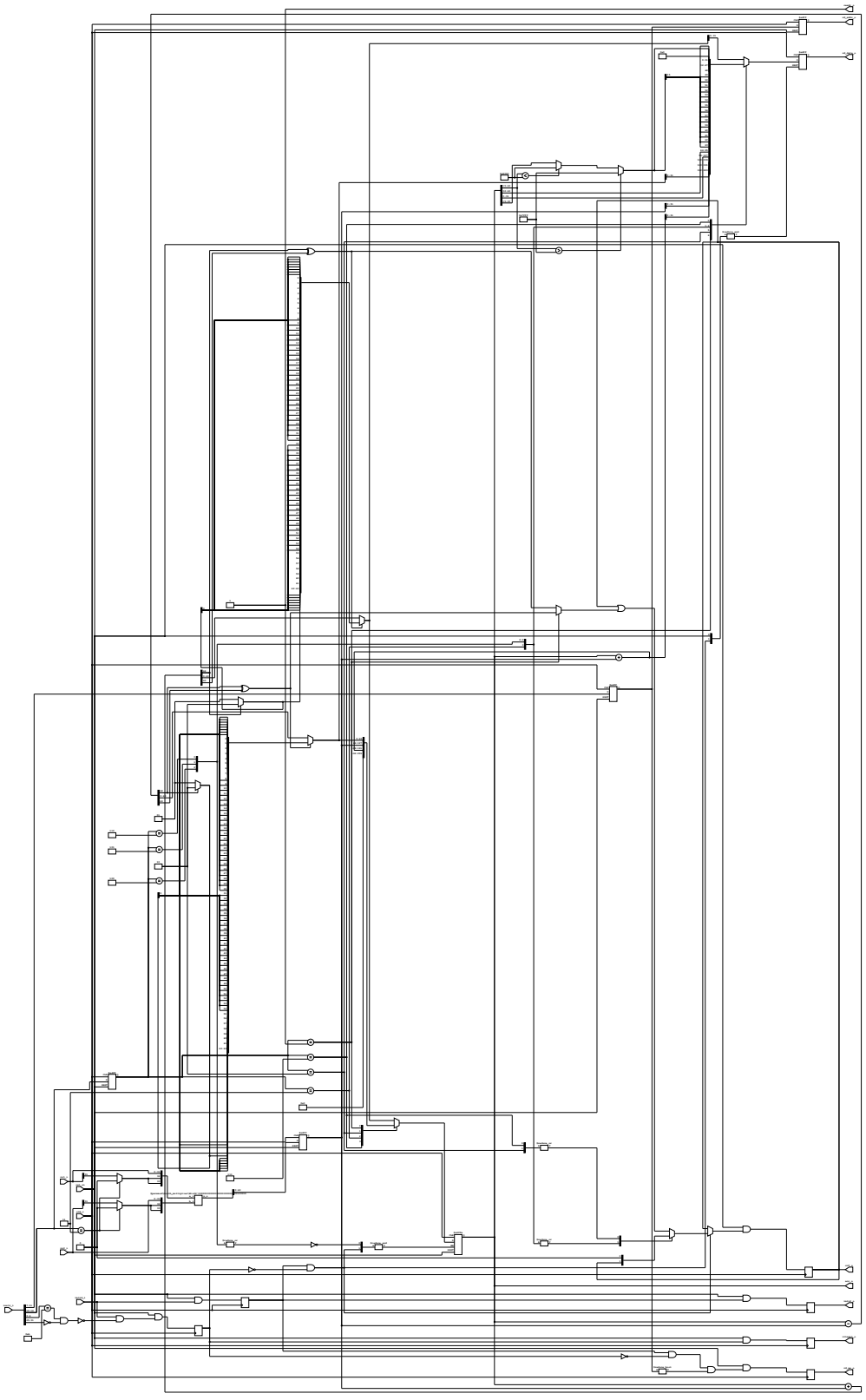


Figure 9.3: Yosys RTL schematic of the complete `mac_unit` (Booth multiplier shown as a single block).

Table 9.1: Generic synthesis result (flattened `mac_unit`).

Cell type	Count	Cell type	Count
NAND	5027	NOT	74
AND	1785	MUX	32
XOR	1459	DFF, sync. reset (`$_SDFF_`)	114
OR	376	DFF, sync. reset + enable (`$_SDFFE_`)	65
NOR	226	**Total cells**	**9532**
XNOR	199	**Flip-flops**	**179**
ANDNOT / ORNOT	96 / 79	Est. transistors (`stat -tech cmos`)	≈55,456

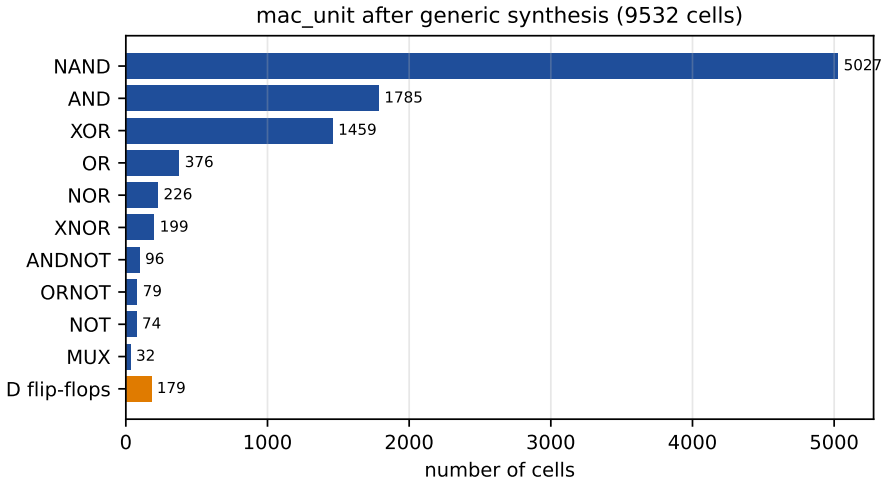


Figure 9.4: Cell-type distribution of the synthesised MAC unit.

Register count check. The 179 flip-flops match the RTL exactly. Stage 1 holds 1+1+3+5+64 = 74 bits (valid, illegal, `funct3`, `rd`, product), while stage 2 holds 64+1+1+5+1+32+1 = 105 bits (ACC, OVF, valid, `rd`, write enable, data, illegal). The 65 flip-flops with an enable input are the accumulator and the OVF flag. They keep their value on read-only instructions; Yosys inferred the enable from the default assignment `acc_nxt = acc_q`.

Logic depth. The longest combinational path (`ltp`) is **127 gate levels**, from the operands through the multiplier to `s1_prod[63]`. For comparison, the Booth module was replaced by `*` and synthesised with Yosys' own Booth-encoder + Wallace-tree generator (`synth -booth`). That version uses 21 % fewer gates (7,541) but its depth is the same (128 levels). In both cases, the depth comes from the ripple-carry carry-propagate adders, which account for about two gate levels per bit, not from the partial-product reduction.

9.3 Area breakdown: MAC unit and complete processor

Figure 9.5 compares the hierarchical gate counts. In the stand-alone MAC unit, the Booth multiplier accounts for 79 % of the logic (7,852 of 9,990 cells). The complete processor, synthesised by the script `synth/synth_core.ys`, contains 25,286 gates in total. Of these, the RV32 pipeline makes up 15,303, which includes the register file implemented as 1,024 flip-flops (the generic flow has no RAM cells) together with the RV32M 33×33 multiplier; the MAC unit's control and accumulator take 2,138 gates; and the Booth multiplier takes 7,845 gates. Adding the full DSP extension therefore costs about 40 % of the processor's gate count.

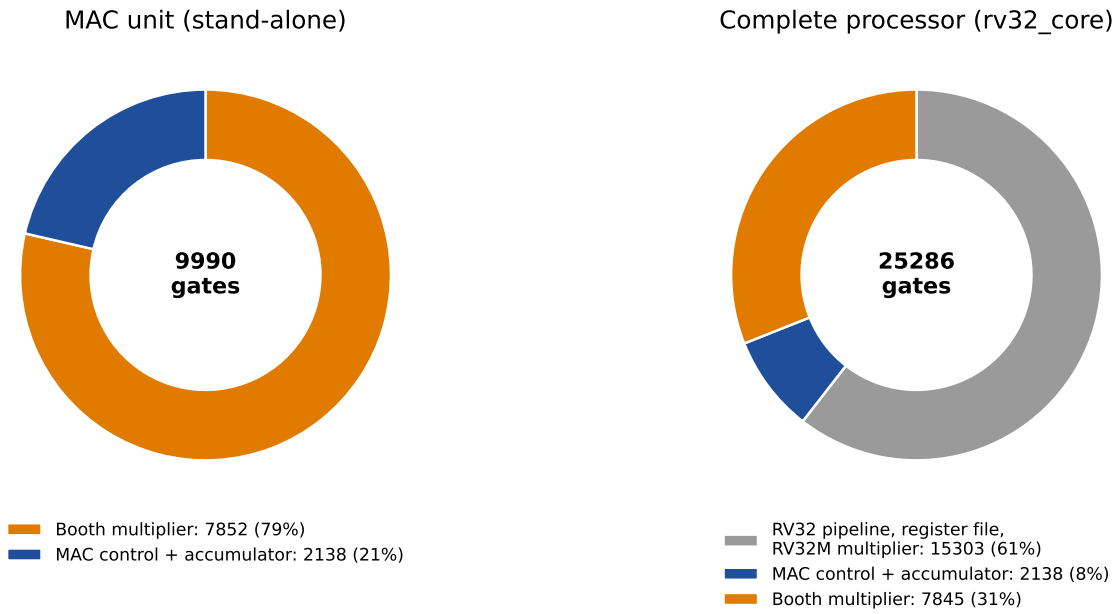


Figure 9.5: Generic-gate area breakdown of the MAC unit and of the complete processor.

9.4 FPGA implementation on the Lattice ECP5

Three designs were placed and routed. Design A is the MAC unit with the Booth multiplier, in which the radix-4 Booth multiplier is built from LUTs and carry chains. Design B is the MAC unit with DSP multipliers: the Booth module is replaced by `synth/native_multiplier.v` (`*`), so the multiplication maps onto four hard `MULT18X18D` blocks, and it serves as a comparison point only. Design C is the complete processor, `rv32_core` including the MAC unit with the Booth multiplier; here the register file maps to 32 LUT-RAM blocks and the RV32M multiplier to DSP blocks, while the instruction and data memories are outside the core. For A and B, the unit is wrapped in `synth/fmax_wrapper.v`, which registers every input and output, so that every timing path is register-to-register, as it would be inside a processor.

Table 9.2: ECP5 LFE5U-85F implementation results (post place & route).

Metric	A: MAC (Booth)	B: MAC (DSP)	C: RV32IM core + MAC
LUT4 (Yosys)	5,108	611	6,486
PFUMX / L6MUX21 (wide muxes)	1,855 / 584	210 / 52	1,992 / 631
CCU2C (carry cells)	155	162	399
Flip-flops (design)	179	179	730
LUT-RAM blocks (DPR16X4)	0	0	32
MULT18X18D (DSP multipliers)	0	4	4
Logic cells after P&R	4,854 (5.8 %)	963 (1.2 %)	7,566 (9.0 %)
I/O pins	204	204	330
Critical path (reg → reg)	17.73 ns	12.69 ns	28.94 ns
of which logic / routing	6.0 / 11.8 ns	7.9 / 4.8 ns	10.1 / 18.9 ns
Maximum frequency $F_{\max}$	**56.4 MHz**	**78.8 MHz**	**34.6 MHz**

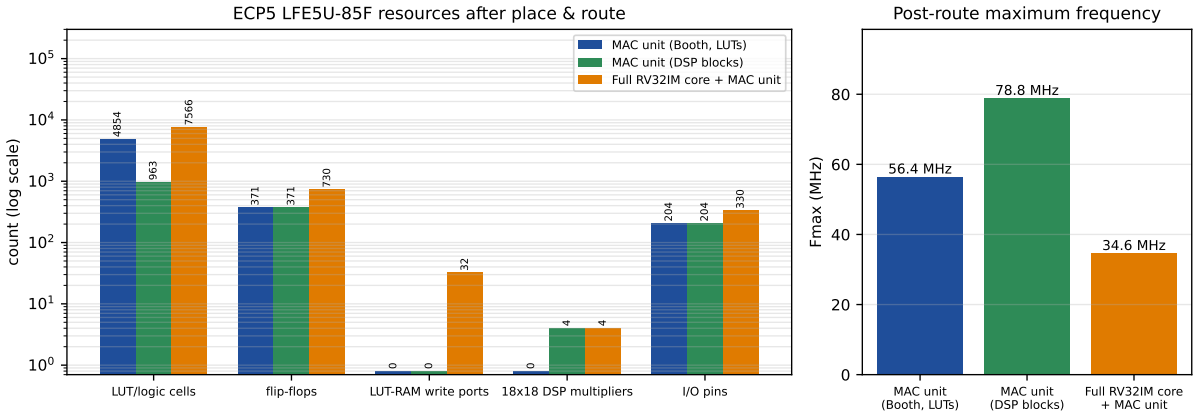


Figure 9.6: ECP5 resource usage (log scale) and post-route $F_{\max}$ of the three designs.

Critical paths. Figure 9.7 plots the cumulative delay along each design's critical path, as reported by nextpnr. For designs A and B, the path starts at the input operand register, passes through the stage-1 multiplier and ends at the `s1_prod` pipeline register; the stage-2 accumulate path is shorter, which confirms that the MAC pipeline is split in the right place. In the Booth version, routing (11.8 ns) dominates logic (6.0 ns), because the LUT-based multiplier is spread over a large area. For the complete processor, the critical path runs from the WB forwarding control, through the forwarding multiplexer and the *single-cycle RV32M multiplier* (DSP blocks plus carry-chain adders), to the EX/MEM result register, so in the full CPU it is the baseline `mul` instruction, not the pipelined MAC unit, that limits the clock.

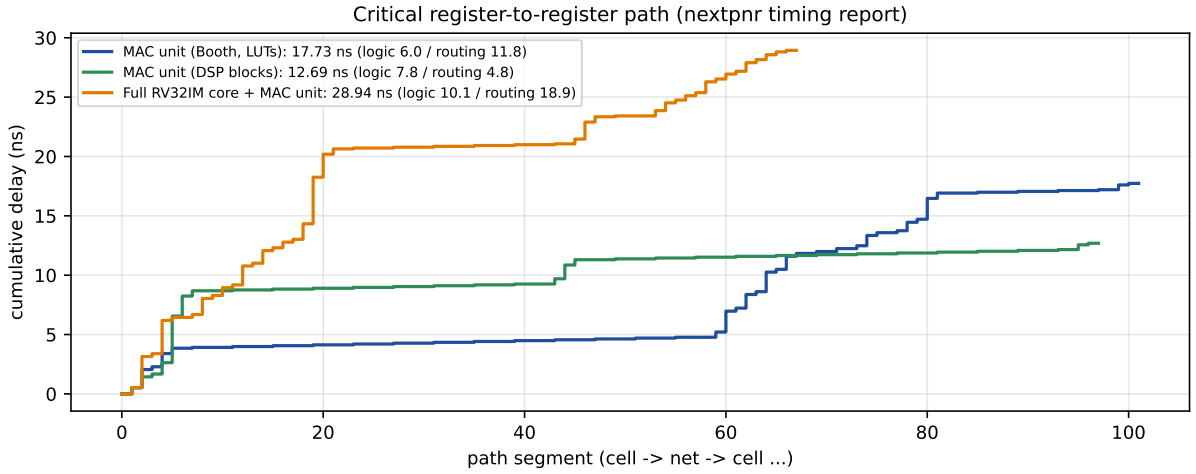


Figure 9.7: Cumulative delay along the critical register-to-register path of each design.

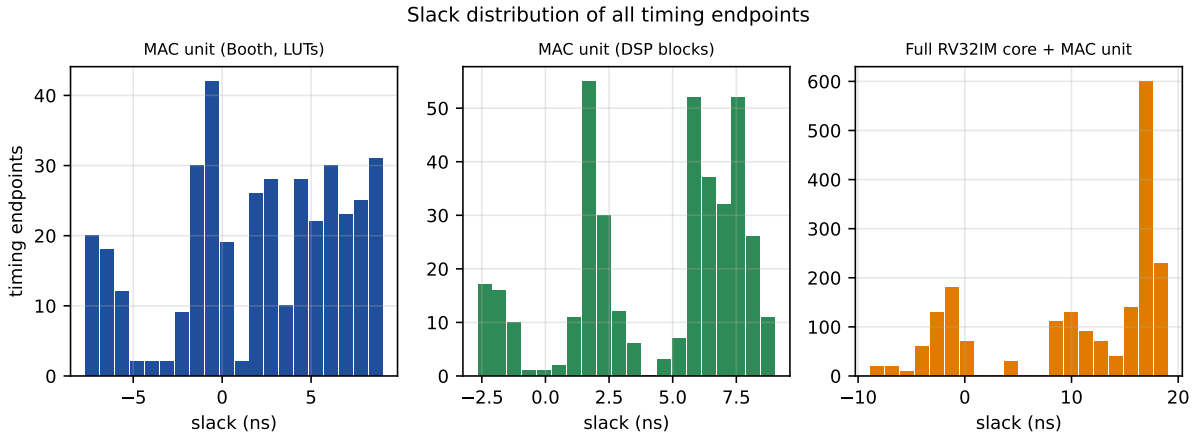


Figure 9.8: Slack histograms of all timing endpoints (100 MHz target for A/B, 50 MHz for C).

Floorplans. Figures 9.9 and 9.10 are die renders of the placed-and-routed designs, drawn from nextpnr's output (`report/scripts/export_floorplan.py`). Each square is one programmable tile of the ECP5 die (tile coordinates X 0–126, Y 0–95), coloured by the block whose cells nextpnr placed there; brighter means more cells in the tile. The copper glow underneath is the routing density (routed wire segments per tile, log scale), the ring of small squares is the used I/O pads, and the white line is the critical path, from its start (dot) to its end (star). An interactive version, with zoom, per-block highlighting and a per-tile breakdown, is on the project page. The Booth multiplier (orange) forms one dense block, with the MAC stage-2 logic (blue) at its edge, and it is the routing hot spot. In the complete processor, the core logic, the MAC unit and the four DSP blocks are packed into one region in the upper-left corner, close to the I/O pins on the top edge.

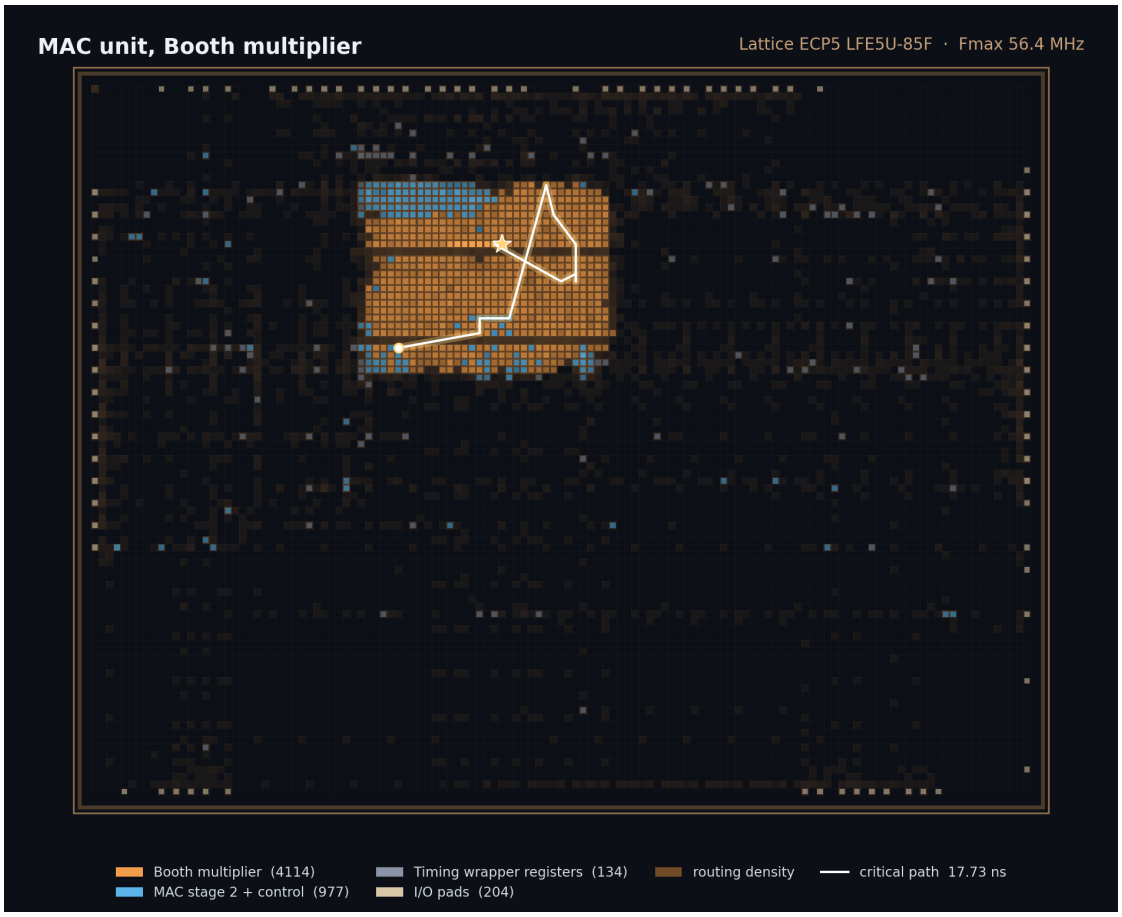


Figure 9.9: Die render of the MAC unit (design A) on the ECP5: placement by block, routing density and critical path.

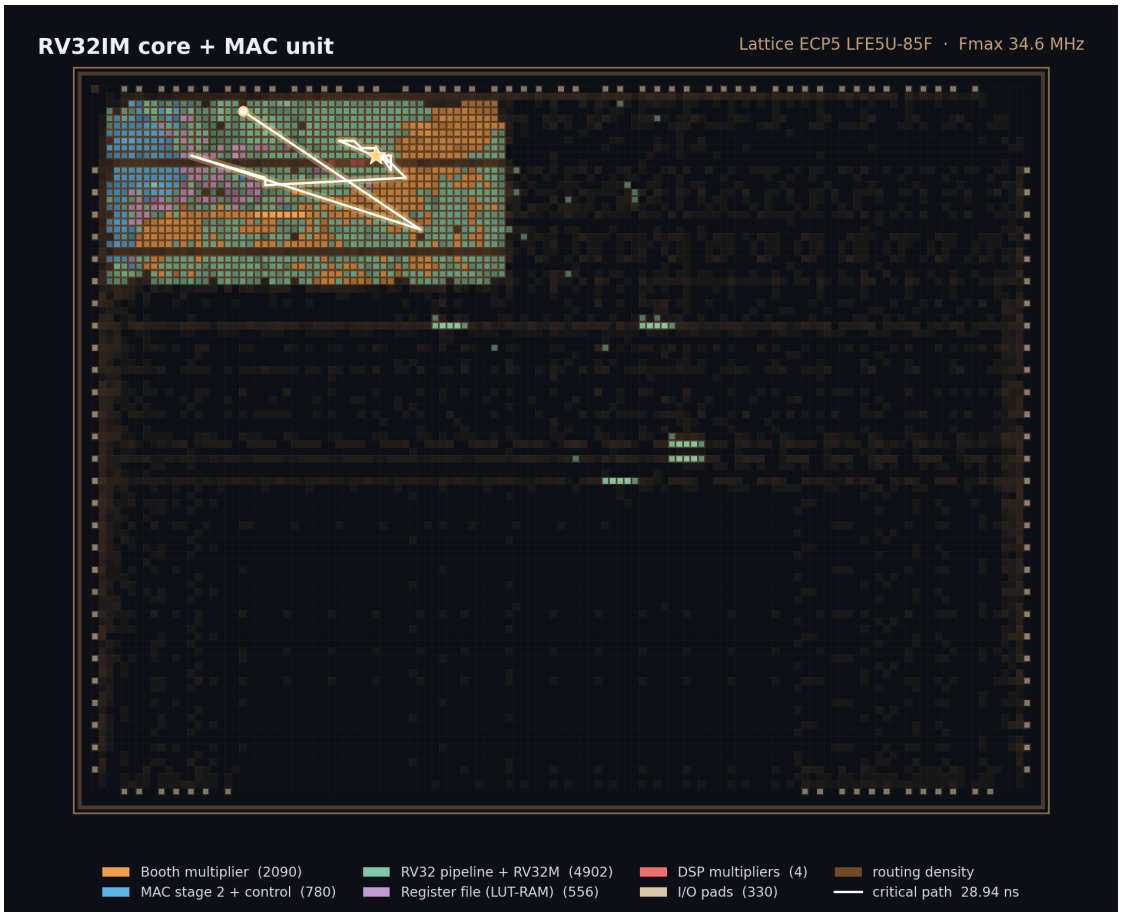


Figure 9.10: Die render of the complete RV32IM core with the MAC unit (design C). The register file (purple) sits next to the MAC unit; the critical path runs through the RV32M multiplier on the DSP blocks.

Interpretation. On an FPGA, the hand-written Booth multiplier uses about 8× more LUTs than the DSP version and is 28 % slower. This is expected, because FPGAs contain hard multipliers. The Booth design is still the right choice for this project: it shows the algorithm explicitly, it is fully portable, and it is what an ASIC implementation would use. For an FPGA product, variant B would be preferred.

9.5 Post-synthesis gate-level simulation

The flattened gate-level netlist written by Yosys (`synth/out/mac_unit_netlist.v`, about 19,000 lines) was simulated with the *unchanged* testbench `tb_mac_unit.v`. It passed all tests (Listing 9.1). This confirms that synthesis preserved the behaviour of the RTL and that there are no simulation/synthesis mismatches. The gate-level simulation has zero delay, so it checks function only; timing is covered by the static timing analysis above.

Listing 9.1: Gate-level simulation result (`sim/gate/mac_gate_sim.log`).

```
FUNCTIONAL COVERAGE
  mac=570 msu=338 mul.acc=378 macu=381 rdacc.lo=346 rdacc.hi=386 rdacc.q15=393 clracc=374
  illegal=162  saturation events=7  rd=x0 writes=360  bubble gaps=684
RESULTS
  instructions issued : 3328
  checks performed    : 3351
  errors              : 0
  >>> ALL TESTS PASSED <<<
```

10 Discussion

10.1 Design trade-offs

Dedicated accumulator vs. register-file accumulation. A dedicated 64-bit accumulator gives full precision and needs only two register-file read ports. The cost is extra architectural state, which an operating system must save on a context switch, and explicit `rdacc` instructions to read the result. The Xpulp `p.mac` [5] approach needs a third read port and accumulates in only 32 bits.

Saturation vs. wrap-around. Signed operations saturate, which is the expected DSP behaviour, whereas `macu` wraps modulo 2^{64}, which is what checksums and hashing need.

Two-stage MAC pipeline aligned with the CPU pipeline. Putting the multiplication in EX and the accumulation in MEM means MAC instructions issue back-to-back with no forwarding inside the unit. Only a consumer placed *directly* after a MAC result pays one stall cycle, exactly like a load, and in the scheduled FIR code this never happens.

Instruction count vs. loop overhead. The MAC instructions cut the arithmetic work per tap from six instructions to one. In the looped FIR, however, loads and loop control remain, so the measured speed-up is 1.59×. Once the loop overhead is removed by unrolling, the MAC unit's advantage grows to 2.67×. Adding post-increment loads or hardware loops (as in Xpulp) would bring the looped code close to the unrolled performance.

10.2 Limitations

The synthesis results come from open-source tools and a generic gate library. No commercial ASIC standard-cell library was used, so the report gives gate counts and logic depth rather than area in μm^2 or power figures. The MAC unit's critical path is the 34×34-bit combinational Booth multiplier (17.7 ns on ECP5, 127 gate levels), whose partial products are summed with ripple-carry adders. The processor implements the RV32M multiply instructions in a single cycle, which is simple but sets the processor's critical path (28.9 ns, 34.6 MHz on ECP5). It has no caches, interrupts, CSRs or division instructions, and its memories are modelled with combinational reads; on an FPGA they would be replaced by synchronous block RAM, which requires an extra fetch stage or registered addresses. Finally, the programs are written in assembly with a project-specific assembler, and C code would need compiler support for the custom instructions, for example through inline assembly.

10.3 Future work

Several extensions would build on this work. The critical paths could be shortened by using a carry-save (Wallace/Dadda) tree with a parallel-prefix final adder (for example Kogge-Stone) for the Booth multiplier, and by pipelining the RV32M multiplier over EX and MEM, as is already done for the MAC unit. Post-increment loads and a zero-overhead hardware loop could be added so that looped DSP code reaches the throughput of the unrolled code, and packed-SIMD operations (for example two 16×16 MACs per cycle), similar to the RISC-V P-extension proposals, would widen the arithmetic. The instruction and data memories could be mapped to FPGA block RAM so that the processor runs on a real ECP5 board, and the design could be taken through an ASIC flow with a real standard-cell library (for example OpenLane with the SkyWater 130 nm PDK). Finally, the Booth multiplier could be formally verified against `*`, for example with SymbiYosys.

11 Conclusion

This project designed a RISC-V custom-extension multiply-accumulate unit for digital signal processing, integrated it into a complete 5-stage pipelined RISC-V processor, and verified both thoroughly. The MAC unit adds eight `custom-0` instructions and is built around a radix-4 Booth multiplier and a 64-bit saturating accumulator with Q15 support. Its two-stage pipeline accepts one instruction per cycle and lines up exactly with the EX and MEM stages of the processor.

At the unit level, an independent reference model and scoreboard, exhaustive multiplier tests, directed corner cases, three analytical DSP benchmarks and a 3,000-instruction constrained-random regression performed 88,896 checks, all with zero errors.

At the system level, real RISC-V programs, built with a project-specific assembler, ran on the processor: an 18-part ISA and hazard self-test, and a 16-tap low-pass FIR over 128 samples in four versions. Every program passed. The FIR outputs were bit-identical to the golden model, the unwanted tone was attenuated by 60.1 dB as the filter design predicts, and mutation tests confirmed that the system tests detect forwarding and stall bugs. Compared with the same filter in plain RV32IM software, the MAC instructions made the FIR 1.59× faster in looped form and 2.67× faster in unrolled form, with 2.6× smaller code.

Yosys synthesis produced a latch-free MAC netlist of 9,532 gates and 179 flip-flops, which passes the full testbench at gate level. On a Lattice ECP5 FPGA, the MAC unit reaches 56.4 MHz, and the complete processor with the MAC unit reaches 34.6 MHz using 9 % of the device's logic. The schematics, floorplans and timing analysis show that the multipliers, both the Booth multiplier and the RV32M multiplier, are the largest blocks and set the critical paths. That makes them the main target for future optimisation.

References

[1] A. D. Booth, “A signed binary multiplication technique,” *Quarterly Journal of Mechanics and Applied Mathematics*, vol. 4, no. 2, pp. 236–240, 1951.

[2] O. L. MacSorley, “High-speed arithmetic in binary computers,” *Proceedings of the IRE*, vol. 49, no. 1, pp. 67–91, Jan. 1961.

[3] B. Parhami, *Computer Arithmetic: Algorithms and Hardware Designs*, 2nd ed. New York: Oxford University Press, 2010.

[4] M. Gautschi, P. D. Schiavone, A. Traber, I. Loi, A. Pullini, D. Rossi, E. Flamand, F. K. Gürkaynak, and L. Benini, “Near-threshold RISC-V core with DSP extensions for scalable IoT endpoint devices,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 25, no. 10, pp. 2700–2713, Oct. 2017.

[5] OpenHW Group, *CV32E40P User Manual*, section “CORE-V Instruction Set Extensions” (multiply-accumulate). [Online]. Available: `https://docs.openhwgroup.org/projects/cv32e40p-user-manual/`

[6] Arm Ltd., *CMSIS-DSP Software Library*, functions `arm_fir_q15` and `arm_dot_prod_q15`. [Online]. Available: `https://arm-software.github.io/CMSIS-DSP/`

[7] R. G. Lyons, *Understanding Digital Signal Processing*, 3rd ed. Upper Saddle River, NJ: Prentice Hall, 2010.

[8] A. Waterman and K. Asanović, Eds., *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, Document Version 20191213. RISC-V International, Dec. 2019.

[9] *IEEE Standard for Verilog Hardware Description Language*, IEEE Std 1364-2005, 2006.

[10] S. Williams, *Icarus Verilog*. [Online]. Available: `https://steveicarus.github.io/iverilog/`

[11] C. Wolf, *Yosys Open SYnthesis Suite*. [Online]. Available: `https://yosyshq.net/yosys/`

[12] D. Shah, E. Hung, C. Wolf, S. Bazanski, D. Gisselquist, and M. Milanović, “Yosys+nextpnr: An open source framework from Verilog to bitstream for commercial FPGAs,” in *Proc. IEEE 27th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2019, pp. 1–4.

[13] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface*, 2nd ed. Cambridge, MA: Morgan Kaufmann, 2020.

[14] N. Turley, *netlistsvg: draws an SVG schematic from a Yosys JSON netlist*. [Online]. Available: `https://github.com/nturley/netlistsvg`

A Source Code

A.1 `rtl/mac_unit.v` (main module)

```
//============================================================================
// File        : mac_unit.v
// Module      : mac_unit
// Project     : Design and Verification of a RISC-V Multiply-Accumulate (MAC)
//               Unit for Digital Signal Processing
// Language    : Verilog-2005 (IEEE Std 1364-2005), synthesizable
// Authors     : Mohsin Mushtaq (2025MECECS001)
//               Mubashir Karim Ganaie (2025MECECS005)
// Supervisor  : Dr. Afshan Amin Khan
// Institution : Department of Electronics and Communications,
//               National Institute of Technology Srinagar
//
// Description :
//   A 2-stage pipelined Multiply-Accumulate execution unit that plugs into a
//   RV32 core as a custom-instruction functional unit. It decodes R-type
//   instructions in the RISC-V "custom-0" major opcode (0001011), which the
//   RISC-V Unprivileged ISA specification reserves for non-standard
//   extensions, and operates on a dedicated 64-bit accumulator register.
//
//   A 64-bit accumulator holding the sum of 32x32 products is the classic
//   DSP "guard bit" arrangement (cf. CMSIS-DSP q63_t accumulators): it lets
//   long dot products / FIR filters run without intermediate overflow, and
//   the signed operations additionally saturate instead of wrapping.
//
//   Instruction encoding (R-type):
//
//    31        25 24    20 19    15 14  12 11     7 6         0
//   +------------+--------+--------+------+--------+-----------+
//   |  0000000   |  rs2   |  rs1   |funct3|   rd   |  0001011  |
//   +------------+--------+--------+------+--------+-----------+
//
//   funct3 | mnemonic       | operation                          | rd value
//   -------+----------------+------------------------------------+-------------
//    000   | mac   rd,a,b   | ACC = sat64(ACC + a*b)   (signed) | ACC[31:0]
//    001   | msu   rd,a,b   | ACC = sat64(ACC - a*b)   (signed) | ACC[31:0]
//    010   | mul.acc rd,a,b | ACC = a*b, OVF = 0       (signed) | ACC[31:0]
//    011   | macu  rd,a,b   | ACC = ACC + a*b (unsigned, mod 2^64)| ACC[31:0]
//    100   | rdacc.lo rd    | (read only)                        | ACC[31:0]
//    101   | rdacc.hi rd    | (read only)                        | ACC[63:32]
//    110   | rdacc.q15 rd   | (read only) Q30 -> Q15 conversion | sext(sat16(ACC>>>15))
//    111   | clracc rd      | ACC = 0, OVF = 0                   | 0
//
//   OVF is a sticky flag set whenever a signed operation saturates.
//   Writes to rd = x0 are suppressed (rd_we_o = 0), as in RISC-V.
//   Instructions with a different opcode or funct7 raise illegal_o and do
//   not modify the accumulator.
//
// Micro-architecture :
//
//            Stage 1 (EX1)                    Stage 2 (EX2 / WB)
//   instr -> decode ----------------> s1_* -> accumulate / saturate -> ACC
//   rs1   -> extend -> Booth 34x34 -> s1_prod                       -> rd_data_o
//   rs2   -> extend ->   multiplier
//
//   * Fully pipelined: one instruction may be issued every clock cycle.
//   * Fixed latency of 2 cycles from valid_i to valid_o.
//   * All accumulator reads and writes happen in stage 2, in program order,
//     so back-to-back dependent MAC / read instructions need no stalls or
//     forwarding.
//
// Ports :
//   clk_i, rst_ni     clock, active-low synchronous reset
//   valid_i           instruction + operands valid this cycle
//   instr_i[31:0]     32-bit RISC-V instruction word
//   rs1_i, rs2_i      register-file operand values for rs1 / rs2
```

```
//   ready_o           always 1 (unit never stalls)
//   valid_o           result valid (2 cycles after valid_i)
//   rd_addr_o         destination register index
//   rd_we_o           register-file write enable (0 for x0 / illegal)
//   rd_data_o         value to write to rd
//   illegal_o         instruction was not a valid MAC instruction
//   ovf_o             sticky saturation flag
//   acc_o             current accumulator value (debug / observability)
//============================================================================
`timescale 1ns/1ps

module mac_unit (
    input  wire        clk_i,
    input  wire        rst_ni,

    // Issue interface (from the core's decode/register-read stage)
    input  wire        valid_i,
    input  wire [31:0] instr_i,
    input  wire [31:0] rs1_i,
    input  wire [31:0] rs2_i,
    output wire        ready_o,

    // Write-back interface (to the core's register file)
    output reg         valid_o,
    output reg  [4:0]  rd_addr_o,
    output reg         rd_we_o,
    output reg  [31:0] rd_data_o,
    output reg         illegal_o,

    // Status / debug
    output reg         ovf_o,
    output reg  [63:0] acc_o
);

    //--------------------------------------------------------------------------
    // Encoding constants
    //--------------------------------------------------------------------------
    localparam [6:0] OPC_CUSTOM0 = 7'b0001011;   // RISC-V custom-0 opcode
    localparam [6:0] F7_MAC      = 7'b0000000;

    localparam [2:0] F3_MAC   = 3'b000;
    localparam [2:0] F3_MSU   = 3'b001;
    localparam [2:0] F3_MUL   = 3'b010;
    localparam [2:0] F3_MACU  = 3'b011;
    localparam [2:0] F3_RDLO  = 3'b100;
    localparam [2:0] F3_RDHI  = 3'b101;
    localparam [2:0] F3_RDQ15 = 3'b110;
    localparam [2:0] F3_CLR   = 3'b111;

    localparam signed [63:0] ACC_MAX = 64'sh7FFF_FFFF_FFFF_FFFF;
    localparam signed [63:0] ACC_MIN = 64'sh8000_0000_0000_0000;

    assign ready_o = 1'b1;   // fully pipelined, never back-pressures

    //============================================================================
    // STAGE 1 : decode + operand extension + Booth multiplication
    //============================================================================
    wire [6:0] opcode = instr_i[6:0];
    wire [4:0] rd     = instr_i[11:7];
    wire [2:0] funct3 = instr_i[14:12];
    wire [6:0] funct7 = instr_i[31:25];

    wire is_mac_instr = (opcode == OPC_CUSTOM0) && (funct7 == F7_MAC);

    // Unsigned multiply only for MACU; everything else is signed.
    wire is_unsigned = (funct3 == F3_MACU);

    // Extend the 32-bit operands to 34 bits (even width required by the
    // radix-4 Booth multiplier). Sign-extension gives a signed multiply,
    // zero-extension gives an unsigned multiply on the same hardware.
    wire [33:0] op_a = is_unsigned ? {2'b00, rs1_i} : {{2{rs1_i[31]}}, rs1_i};
    wire [33:0] op_b = is_unsigned ? {2'b00, rs2_i} : {{2{rs2_i[31]}}, rs2_i};

```

```
    // 34 x 34 -> 68-bit product. The true product always fits in 64 bits
    // (signed: |a*b| <= 2^62, unsigned: a*b < 2^64), so bits [63:0] are exact.
    wire signed [67:0] prod_full;

    booth_multiplier #(.W(34)) u_booth (
        .a_i (op_a),
        .b_i (op_b),
        .p_o (prod_full)
    );

    // Stage-1 pipeline registers
    reg        s1_valid;
    reg        s1_illegal;
    reg [2:0]  s1_funct3;
    reg [4:0]  s1_rd;
    reg [63:0] s1_prod;

    always @(posedge clk_i) begin
        if (!rst_ni) begin
            s1_valid   <= 1'b0;
            s1_illegal <= 1'b0;
            s1_funct3  <= 3'b000;
            s1_rd      <= 5'd0;
            s1_prod    <= 64'd0;
        end else begin
            s1_valid   <= valid_i;
            s1_illegal <= valid_i & ~is_mac_instr;
            s1_funct3  <= funct3;
            s1_rd      <= rd;
            s1_prod    <= prod_full[63:0];
        end
    end

    //=========================================================================
    // STAGE 2 : accumulate / saturate / read-out
    //=========================================================================
    reg signed [63:0] acc_q;     // architectural accumulator
    reg               ovf_q;     // sticky overflow flag

    // 65-bit add / subtract so the carry into the sign position is visible.
    wire signed [64:0] acc_ext  = {acc_q[63], acc_q};
    wire signed [64:0] prod_ext = {s1_prod[63], s1_prod};
    wire signed [64:0] sum_add  = acc_ext + prod_ext;
    wire signed [64:0] sum_sub  = acc_ext - prod_ext;

    // Signed overflow <=> the two top bits of the 65-bit result disagree.
    wire ovf_add = sum_add[64] ^ sum_add[63];
    wire ovf_sub = sum_sub[64] ^ sum_sub[63];

    // Saturated results: clamp toward the sign of the true (65-bit) result.
    wire [63:0] sat_add = ovf_add ? (sum_add[64] ? ACC_MIN : ACC_MAX) : sum_add[63:0];
    wire [63:0] sat_sub = ovf_sub ? (sum_sub[64] ? ACC_MIN : ACC_MAX) : sum_sub[63:0];

    // Q30 -> Q15 conversion for rdacc.q15 (same as CMSIS-DSP:
    // __SSAT(acc >> 15, 16)): arithmetic shift, then saturate to int16.
    wire signed [63:0] acc_shr15 = acc_q >>> 15;
    wire [15:0] q15_val = (acc_shr15 >  64'sd32767) ? 16'h7FFF :
                          (acc_shr15 < -64'sd32768) ? 16'h8000 :
                                                      acc_shr15[15:0];

    // Next-state / result logic for stage 2
    reg [63:0] acc_nxt;
    reg        ovf_nxt;
    reg [31:0] res_nxt;

    always @* begin
        acc_nxt = acc_q;
        ovf_nxt = ovf_q;
        res_nxt = 32'd0;

        if (s1_valid && !s1_illegal) begin
            case (s1_funct3)
                F3_MAC: begin
```

```
                    acc_nxt = sat_add;
                    ovf_nxt = ovf_q | ovf_add;
                    res_nxt = sat_add[31:0];
                end
                F3_MSU: begin
                    acc_nxt = sat_sub;
                    ovf_nxt = ovf_q | ovf_sub;
                    res_nxt = sat_sub[31:0];
                end
                F3_MUL: begin
                    acc_nxt = s1_prod;          // start a new accumulation
                    ovf_nxt = 1'b0;
                    res_nxt = s1_prod[31:0];
                end
                F3_MACU: begin
                    acc_nxt = acc_q + s1_prod;  // modular, no saturation
                    res_nxt = acc_nxt[31:0];
                end
                F3_RDLO:  res_nxt = acc_q[31:0];
                F3_RDHI:  res_nxt = acc_q[63:32];
                F3_RDQ15: res_nxt = {{16{q15_val[15]}}, q15_val};
                F3_CLR: begin
                    acc_nxt = 64'd0;
                    ovf_nxt = 1'b0;
                    res_nxt = 32'd0;
                end
                default: ;
            endcase
        end
    end

    // Stage-2 registers: accumulator state and write-back outputs
    always @(posedge clk_i) begin
        if (!rst_ni) begin
            acc_q     <= 64'd0;
            ovf_q     <= 1'b0;
            valid_o   <= 1'b0;
            rd_addr_o <= 5'd0;
            rd_we_o   <= 1'b0;
            rd_data_o <= 32'd0;
            illegal_o <= 1'b0;
        end else begin
            acc_q     <= acc_nxt;
            ovf_q     <= ovf_nxt;
            valid_o   <= s1_valid;
            rd_addr_o <= s1_rd;
            rd_we_o   <= s1_valid & ~s1_illegal & (s1_rd != 5'd0);
            rd_data_o <= res_nxt;
            illegal_o <= s1_illegal;
        end
    end

    // Status outputs mirror the architectural state
    always @* begin
        acc_o = acc_q;
        ovf_o = ovf_q;
    end

endmodule
```

A.2 rtl/booth_multiplier.v

```
//============================================================================
// File        : booth_multiplier.v
// Module      : booth_multiplier
// Project     : Design and Verification of a RISC-V Multiply-Accumulate (MAC)
//               Unit for Digital Signal Processing
// Language    : Verilog-2005 (IEEE Std 1364-2005), synthesizable
// Authors     : Mohsin Mushtaq (2025MECECS001)
//               Mubashir Karim Ganaie (2025MECECS005)
```

```
// Supervisor  : Dr. Afshan Amin Khan
// Institution : Department of Electronics and Communications,
//               National Institute of Technology Srinagar
//
// Description :
//   Combinational signed multiplier using Radix-4 Modified Booth Encoding
//   (A. D. Booth, 1951; O. L. MacSorley, 1961).
//
//   The multiplier operand B is scanned in overlapping 3-bit groups
//   {b[2i+1], b[2i], b[2i-1]} (with b[-1] = 0). Each group selects one
//   partial product from the set {0, +A, +2A, -A, -2A}:
//
//        b[2i+1] b[2i] b[2i-1] | partial product
//        ----------------------+----------------
//           0      0      0    |   0
//           0      0      1    |  +A
//           0      1      0    |  +A
//           0      1      1    |  +2A
//           1      0      0    |  -2A
//           1      0      1    |  -A
//           1      1      0    |  -A
//           1      1      1    |   0
//
//   Partial product i has weight 4^i, so a W-bit multiply needs only W/2
//   partial products instead of W (halves the adder tree vs. array mult).
//
//   P = sum_{i=0}^{W/2-1} PP_i * 4^i      (two's complement, exact)
//
// Parameters :
//   W  - operand width in bits. MUST be even. Both operands are treated as
//        two's-complement signed numbers. Unsigned operation is obtained by
//        the caller zero-extending its operands by at least one bit.
//
// Ports :
//   a_i  [W-1:0]   multiplicand (signed)
//   b_i  [W-1:0]   multiplier   (signed, Booth-recoded)
//   p_o  [2W-1:0]  full-precision signed product a_i * b_i
//============================================================================
`timescale 1ns/1ps

module booth_multiplier #(
    parameter integer W = 34                // operand width (even)
) (
    input  wire signed [W-1:0]   a_i,
    input  wire signed [W-1:0]   b_i,
    output reg  signed [2*W-1:0] p_o
);

    // Multiplicand sign-extended to the full product width so every
    // partial product (including the shifted/negated ones) is exact.
    wire signed [2*W-1:0] a_ext = {{W{a_i[W-1]}}, a_i};

    // Multiplier with the implicit b[-1] = 0 appended on the right.
    wire [W:0] b_ext = {b_i, 1'b0};

    integer           i;
    reg [2:0]         grp;     // current Booth group {b[2i+1], b[2i], b[2i-1]}
    reg signed [2*W-1:0] pp;   // current (un-shifted) partial product
    reg signed [2*W-1:0] acc;  // running sum of weighted partial products

    always @* begin
        acc = {2*W{1'b0}};
        for (i = 0; i < W/2; i = i + 1) begin
            grp = b_ext[2*i +: 3];

            // Booth recoding: select partial product from {0, ±A, ±2A}
            case (grp)
                3'b000, 3'b111: pp = {2*W{1'b0}};     //  0
                3'b001, 3'b010: pp =  a_ext;          // +A
                3'b011:         pp =  a_ext <<< 1;    // +2A
                3'b100:         pp = -(a_ext <<< 1);  // -2A
                3'b101, 3'b110: pp = -a_ext;          // -A
                default:        pp = {2*W{1'b0}};
```

```
            endcase

            // Accumulate with weight 4^i (shift by 2i bit positions)
            acc = acc + (pp <<< (2*i));
        end
        p_o = acc;
    end

endmodule
```

A.3 tb/tb_mac_unit.v (testbench)

```
//============================================================================
// File        : tb_mac_unit.v
// Module      : tb_mac_unit
// Project     : Design and Verification of a RISC-V Multiply-Accumulate (MAC)
//               Unit for Digital Signal Processing
// Language    : Verilog-2005 (simulation only)
// Authors     : Mohsin Mushtaq (2025MECECS001)
//               Mubashir Karim Ganaie (2025MECECS005)
// Supervisor  : Dr. Afshan Amin Khan
// Institution : Department of Electronics and Communications,
//               National Institute of Technology Srinagar
//
// Description :
//   Self-checking testbench for mac_unit.
//
//   Verification architecture
//   -------------------------
//
//     +-----------+  instr/rs1/rs2  +-----------+  rd/data/acc/ovf  +---------+
//     | stimulus  |---------------->|  DUT      |------------------>| monitor |
//     | (tasks)   |                 | mac_unit  |                   | checker |
//     +-----------+                 +-----------+                   +---------+
//           |                                                           ^
//           |   +-----------------------+    expected results (FIFO)     |
//           +-->| reference model       |--------------------------------+
//               | (behavioural, 128-bit)|
//               +-----------------------+
//
//   * The reference model is written independently of the RTL: it uses the
//     simulator's native '*' operator (not Booth) and 128-bit arithmetic
//     with explicit range comparisons for saturation (not carry/sign-bit
//     overflow detection like the RTL).
//   * For every issued instruction the expected rd value, rd address,
//     write-enable, illegal flag, accumulator and overflow flag are pushed
//     into a scoreboard FIFO; the monitor pops and compares ALL of them on
//     every valid_o, and also checks the 2-cycle latency.
//
//   Test plan
//   ---------
//     T1  Reset state
//     T2  Basic MUL / MAC / MSU arithmetic (small signed values)
//     T3  Sign combinations (+*+, +*-, -*+, -*-)
//     T4  Extreme operands (INT32_MAX, INT32_MIN) and 64-bit results
//     T5  Positive / negative saturation of the 64-bit accumulator + OVF
//     T6  Unsigned MACU (modular), sum of squares benchmark
//     T7  rd = x0 (write suppressed, accumulator still updated)
//     T8  Illegal instruction detection
//     T9  Q15 fixed-point read-out incl. the -1.0 * -1.0 corner case
//     T10 DSP benchmark : dot product [1..8].[8..1] = 120
//     T11 DSP benchmark : 8-tap moving-average FIR, step response
//     T12 DSP benchmark : 8-tap triangular low-pass FIR, DC + Nyquist tone
//     T13 Constrained-random regression (back-to-back + bubbles + illegal)
//
//   Outputs: PASS/FAIL lines on the terminal, a summary with functional
//   coverage counters, and a VCD waveform file (mac_unit.vcd).
//============================================================================
`timescale 1ns/1ps

```

```
59  module tb_mac_unit;
60
61      //--------------------------------------------------------------------------
62      // Clock (100 MHz) and reset
63      //--------------------------------------------------------------------------
64      localparam integer CLK_PERIOD = 10;
65
66      reg clk = 1'b0;
67      always #(CLK_PERIOD/2) clk = ~clk;
68
69      reg rst_n;
70
71      //--------------------------------------------------------------------------
72      // DUT interface
73      //--------------------------------------------------------------------------
74      reg         valid_i;
75      reg  [31:0] instr_i;
76      reg  [31:0] rs1_i;
77      reg  [31:0] rs2_i;
78      wire        ready_o;
79      wire        valid_o;
80      wire [4:0]  rd_addr_o;
81      wire        rd_we_o;
82      wire [31:0] rd_data_o;
83      wire        illegal_o;
84      wire        ovf_o;
85      wire [63:0] acc_o;
86
87      mac_unit dut (
88          .clk_i     (clk),
89          .rst_ni    (rst_n),
90          .valid_i   (valid_i),
91          .instr_i   (instr_i),
92          .rs1_i     (rs1_i),
93          .rs2_i     (rs2_i),
94          .ready_o   (ready_o),
95          .valid_o   (valid_o),
96          .rd_addr_o (rd_addr_o),
97          .rd_we_o   (rd_we_o),
98          .rd_data_o (rd_data_o),
99          .illegal_o (illegal_o),
100         .ovf_o     (ovf_o),
101         .acc_o     (acc_o)
102     );
103
104     //--------------------------------------------------------------------------
105     // Instruction encoding helpers
106     //--------------------------------------------------------------------------
107     localparam [6:0] OPC_CUSTOM0 = 7'b0001011;
108
109     localparam [2:0] F3_MAC   = 3'b000;
110     localparam [2:0] F3_MSU   = 3'b001;
111     localparam [2:0] F3_MUL   = 3'b010;
112     localparam [2:0] F3_MACU  = 3'b011;
113     localparam [2:0] F3_RDLO  = 3'b100;
114     localparam [2:0] F3_RDHI  = 3'b101;
115     localparam [2:0] F3_RDQ15 = 3'b110;
116     localparam [2:0] F3_CLR   = 3'b111;
117
118     localparam signed [63:0] ACC_MAX = 64'sh7FFF_FFFF_FFFF_FFFF;
119     localparam signed [63:0] ACC_MIN = 64'sh8000_0000_0000_0000;
120
121     // Build a 32-bit R-type instruction word.
122     // rs1/rs2 register indices are fixed (x1, x2): the operand VALUES are
123     // supplied separately on rs1_i / rs2_i, as a core's register file would.
124     function [31:0] enc;
125         input [6:0] f7;
126         input [2:0] f3;
127         input [4:0] rd;
128         input [6:0] opc;
129         begin
130             enc = {f7, 5'd2, 5'd1, f3, rd, opc};
131         end
```

```
132     endfunction
133
134     //------------------------------------------------------------------------
135     // Reference model state
136     //------------------------------------------------------------------------
137     reg signed [63:0] m_acc;
138     reg               m_ovf;
139
140     // Golden behavioural model of one instruction. Updates m_acc / m_ovf and
141     // returns the expected illegal flag, write-enable and rd value.
142     task model_exec;
143         input  [31:0] instr;
144         input  [31:0] a;
145         input  [31:0] b;
146         output        e_ill;
147         output        e_we;
148         output [31:0] e_res;
149         reg signed [63:0]  sa, sb, sp;   // signed operands / product
150         reg        [63:0]  up;           // unsigned product
151         reg signed [127:0] wide;         // overflow-free wide sum
152         reg signed [63:0]  sh;
153         begin
154             e_ill = !((instr[6:0] == OPC_CUSTOM0) && (instr[31:25] == 7'd0));
155             e_we  = !e_ill && (instr[11:7] != 5'd0);
156             e_res = 32'd0;
157
158             sa = $signed(a);                // sign-extend to 64 bits
159             sb = $signed(b);
160             sp = sa * sb;                   // native multiply (not Booth)
161             up = {32'd0, a} * {32'd0, b};   // unsigned 64-bit product
162
163             if (!e_ill) begin
164                 case (instr[14:12])
165                     F3_MAC, F3_MSU: begin
166                         wide = m_acc;
167                         if (instr[14:12] == F3_MAC) wide = wide + sp;
168                         else                        wide = wide - sp;
169                         if (wide > ACC_MAX) begin
170                             m_acc = ACC_MAX; m_ovf = 1'b1;
171                         end else if (wide < ACC_MIN) begin
172                             m_acc = ACC_MIN; m_ovf = 1'b1;
173                         end else begin
174                             m_acc = wide[63:0];
175                         end
176                         e_res = m_acc[31:0];
177                     end
178                     F3_MUL: begin
179                         m_acc = sp; m_ovf = 1'b0; e_res = m_acc[31:0];
180                     end
181                     F3_MACU: begin
182                         m_acc = m_acc + up;   // wraps modulo 2^64
183                         e_res = m_acc[31:0];
184                     end
185                     F3_RDLO: e_res = m_acc[31:0];
186                     F3_RDHI: e_res = m_acc[63:32];
187                     F3_RDQ15: begin
188                         sh = m_acc >>> 15;
189                         if      (sh >  32767) e_res = 32'h0000_7FFF;
190                         else if (sh < -32768) e_res = 32'hFFFF_8000;
191                         else                  e_res = {{16{sh[15]}}, sh[15:0]};
192                     end
193                     F3_CLR: begin
194                         m_acc = 64'd0; m_ovf = 1'b0; e_res = 32'd0;
195                     end
196                 endcase
197             end
198         end
199     endtask
200
201     //------------------------------------------------------------------------
202     // Scoreboard (expected-result FIFO)
203     //------------------------------------------------------------------------
204     localparam integer SB_DEPTH = 1024;
```

```
205
206     reg [31:0]     sb_data [0:SB_DEPTH-1];
207     reg [4:0]      sb_rd   [0:SB_DEPTH-1];
208     reg            sb_we   [0:SB_DEPTH-1];
209     reg            sb_ill  [0:SB_DEPTH-1];
210     reg [63:0]     sb_acc  [0:SB_DEPTH-1];
211     reg            sb_ovf  [0:SB_DEPTH-1];
212     reg            sb_show [0:SB_DEPTH-1];   // print a PASS line
213     reg            sb_cap  [0:SB_DEPTH-1];   // capture rd value for FIR tests
214     reg [8*44-1:0] sb_desc [0:SB_DEPTH-1];
215     time           sb_t    [0:SB_DEPTH-1];   // time operands were driven
216
217     integer wr_ptr, rd_ptr;
218     integer n_checks, n_errors, n_cap;
219     reg [31:0] cap_buf [0:255];
220     reg        verbose;
221
222     // Functional coverage counters
223     integer cov_op [0:7];
224     integer cov_ill, cov_sat, cov_x0, cov_bubble;
225
226     //------------------------------------------------------------------------
227     // Driver: issue one instruction (model + scoreboard push + drive DUT)
228     //------------------------------------------------------------------------
229     task issue;
230         input [31:0]     instr;
231         input [31:0]     a;
232         input [31:0]     b;
233         input            cap;
234         input [8*44-1:0] desc;
235         reg e_ill, e_we, ovf_before;
236         reg [31:0] e_res;
237         integer idx;
238         begin
239             ovf_before = m_ovf;
240             model_exec(instr, a, b, e_ill, e_we, e_res);
241
242             idx = wr_ptr % SB_DEPTH;
243             sb_data[idx] = e_res;
244             sb_rd[idx]   = instr[11:7];
245             sb_we[idx]   = e_we;
246             sb_ill[idx]  = e_ill;
247             sb_acc[idx]  = m_acc;
248             sb_ovf[idx]  = m_ovf;
249             sb_show[idx] = verbose;
250             sb_cap[idx]  = cap;
251             sb_desc[idx] = desc;
252             wr_ptr = wr_ptr + 1;
253
254             // coverage
255             if (e_ill) cov_ill = cov_ill + 1;
256             else       cov_op[instr[14:12]] = cov_op[instr[14:12]] + 1;
257             if (!ovf_before && m_ovf) cov_sat = cov_sat + 1;
258             if (!e_ill && instr[11:7] == 5'd0) cov_x0 = cov_x0 + 1;
259
260             // drive on the falling edge, DUT samples on the next rising edge
261             @(negedge clk);
262             sb_t[idx] = $time;
263             valid_i = 1'b1;
264             instr_i = instr;
265             rs1_i   = a;
266             rs2_i   = b;
267             @(posedge clk);
268             #1 valid_i = 1'b0;
269         end
270     endtask
271
272     // Shorthand for a legal MAC-unit instruction
273     task op;
274         input [2:0]      f3;
275         input [4:0]      rd;
276         input [31:0]     a;
277         input [31:0]     b;
```

```
278         input [8*44-1:0] desc;
279         begin
280             issue(enc(7'd0, f3, rd, OPC_CUSTOM0), a, b, 1'b0, desc);
281         end
282     endtask
283
284     // Insert idle (bubble) cycles
285     task idle;
286         input integer n;
287         begin
288             repeat (n) @(posedge clk);
289         end
290     endtask
291
292     // Wait until every issued instruction has been checked
293     task drain;
294         begin
295             while (rd_ptr < wr_ptr) @(negedge clk);
296             @(negedge clk);
297         end
298     endtask
299
300     //------------------------------------------------------------------------
301     // Monitor / checker
302     //------------------------------------------------------------------------
303     integer mi;
304     reg     ok;
305
306     always @(negedge clk) begin
307         if (rst_n && valid_o) begin
308             if (rd_ptr >= wr_ptr) begin
309                 n_errors = n_errors + 1;
310                 $display("  [FAIL] unexpected valid_o at t=%0t", $time);
311             end else begin
312                 mi = rd_ptr % SB_DEPTH;
313                 ok = (rd_data_o === sb_data[mi]) && (rd_addr_o === sb_rd[mi]) &&
314                      (rd_we_o   === sb_we[mi])   && (illegal_o === sb_ill[mi]) &&
315                      (acc_o     === sb_acc[mi])  && (ovf_o     === sb_ovf[mi]) &&
316                      ($time - sb_t[mi] == 2*CLK_PERIOD);   // 2-cycle latency
317                 n_checks = n_checks + 1;
318
319                 if (!ok) begin
320                     n_errors = n_errors + 1;
321                     $display("  [FAIL] %s", sb_desc[mi]);
322                     $display("         expected: rd=x%0d we=%b ill=%b data=0x%08h acc=0x%016h ovf=%b",
323                              sb_rd[mi], sb_we[mi], sb_ill[mi], sb_data[mi], sb_acc[mi], sb_ovf[mi]);
324                     $display("         got     : rd=x%0d we=%b ill=%b data=0x%08h acc=0x%016h ovf=%b
    latency=%0t",
325                              rd_addr_o, rd_we_o, illegal_o, rd_data_o, acc_o, ovf_o, $time - sb_t[mi]);
326                 end else if (sb_show[mi]) begin
327                     $display("  [PASS] %s | rd=0x%08h (%0d)  acc=0x%016h ovf=%b%s",
328                              sb_desc[mi], rd_data_o, $signed(rd_data_o), acc_o, ovf_o,
329                              illegal_o ? "  ILLEGAL" : (rd_we_o ? "" : "  (no write)"));
330                 end
331
332                 if (sb_cap[mi]) begin
333                     cap_buf[n_cap] = rd_data_o;
334                     n_cap = n_cap + 1;
335                 end
336                 rd_ptr = rd_ptr + 1;
337             end
338         end
339     end
340
341     //------------------------------------------------------------------------
342     // Helper: compare a single value against an analytically known result
343     //------------------------------------------------------------------------
344     task check_value;
345         input [8*44-1:0] name;
346         input [31:0]     got;
347         input [31:0]     exp;
348         begin
349             n_checks = n_checks + 1;
```

```
            if (got === exp)
                $display("  [PASS] %s | got %0d, expected %0d", name, $signed(got), $signed(exp));
            else begin
                n_errors = n_errors + 1;
                $display("  [FAIL] %s | got %0d, expected %0d", name, $signed(got), $signed(exp));
            end
        end
    endtask

    //--------------------------------------------------------------------------
    // FIR filter benchmark helper
    //   y[n] = sat16( (sum_k h[k] * x[n-k]) >>> 15 )   (Q15 in, Q15 out)
    //   executed on the DUT as:  mul.acc  x0, x[n],   h[0]
    //                            mac      x0, x[n-k], h[k]   k = 1..7
    //                            rdacc.q15 x10
    //--------------------------------------------------------------------------
    localparam integer TAPS = 8;
    reg signed [15:0] fir_h [0:TAPS-1];
    reg signed [15:0] fir_x [0:63];
    integer           fir_len;

    task run_fir;
        integer n, k;
        reg [31:0] xv;
        begin
            n_cap = 0;
            for (n = 0; n < fir_len; n = n + 1) begin
                for (k = 0; k < TAPS; k = k + 1) begin
                    xv = (n - k >= 0) ? {{16{fir_x[n-k][15]}}, fir_x[n-k]} : 32'd0;
                    op((k == 0) ? F3_MUL : F3_MAC, 5'd0, xv,
                       {{16{fir_h[k][15]}}, fir_h[k]}, "fir tap");
                end
                issue(enc(7'd0, F3_RDQ15, 5'd10, OPC_CUSTOM0), 32'd0, 32'd0, 1'b1, "fir out");
            end
            drain;
        end
    endtask

    //--------------------------------------------------------------------------
    // Random stimulus helpers (fixed seed => reproducible regression)
    //--------------------------------------------------------------------------
    integer seed;
    reg [31:0] rnd_val;

    // Operand generator biased toward arithmetic corner cases
    task rnd_operand;
        integer sel;
        begin
            sel = {$random(seed)} % 10;
            case (sel)
                0: rnd_val = 32'h0000_0000;
                1: rnd_val = 32'h0000_0001;
                2: rnd_val = 32'hFFFF_FFFF;               // -1
                3: rnd_val = 32'h7FFF_FFFF;               // INT32_MAX
                4: rnd_val = 32'h8000_0000;               // INT32_MIN
                5: rnd_val = $random(seed) % 256;         // small signed
                6: rnd_val = {{16{1'b0}}, $random(seed)} & 32'h0000_FFFF;
                default: rnd_val = $random(seed);         // full range
            endcase
        end
    endtask

    //--------------------------------------------------------------------------
    // Main test sequence
    //--------------------------------------------------------------------------
    integer i, n, k, t0, t1;
    reg [31:0] a_r, b_r, ins;
    reg [2:0]  f3_r;
    reg [4:0]  rd_r;
    integer    sum_sq;

    initial begin
        $dumpfile("mac_unit.vcd");
```

```
423         $dumpvars(0, tb_mac_unit);
424
425         // init
426         rst_n   = 1'b0;
427         valid_i = 1'b0;
428         instr_i = 32'd0;
429         rs1_i   = 32'd0;
430         rs2_i   = 32'd0;
431         m_acc   = 64'd0;
432         m_ovf   = 1'b0;
433         wr_ptr  = 0;  rd_ptr = 0;
434         n_checks = 0; n_errors = 0; n_cap = 0;
435         verbose = 1'b1;
436         seed    = 32'h1234_5678;
437         for (i = 0; i < 8; i = i + 1) cov_op[i] = 0;
438         cov_ill = 0; cov_sat = 0; cov_x0 = 0; cov_bubble = 0;
439
440         $display("======================================================================");
441         $display(" RISC-V MAC Unit (custom-0) -- Self-Checking Testbench");
442         $display("======================================================================");
443
444         repeat (3) @(posedge clk);
445         #1 rst_n = 1'b1;
446
447         //----------------------------------------------------------------
448         $display("\n--- T1: Reset state ---");
449         op(F3_RDLO, 5'd5, 0, 0, "rdacc.lo x5          (after reset)");
450         op(F3_RDHI, 5'd5, 0, 0, "rdacc.hi x5          (after reset)");
451         drain;
452
453         //----------------------------------------------------------------
454         $display("\n--- T2: Basic MUL / MAC / MSU (back-to-back issue) ---");
455         op(F3_MUL,  5'd5, 7,  6,            "mul.acc x5, 7, 6     -> 42");
456         op(F3_MAC,  5'd5, 3,  -32'sd4,      "mac x5, 3, -4        -> 30");
457         op(F3_MSU,  5'd5, -32'sd5, 10,      "msu x5, -5, 10       -> 80");
458         op(F3_MAC,  5'd5, 100, 1000,        "mac x5, 100, 1000    -> 100080");
459         op(F3_RDLO, 5'd6, 0, 0,             "rdacc.lo x6          -> 100080");
460         op(F3_RDHI, 5'd7, 0, 0,             "rdacc.hi x7          -> 0");
461         drain;
462
463         //----------------------------------------------------------------
464         $display("\n--- T3: Sign combinations ---");
465         op(F3_MUL,  5'd5,  12,       13,       "mul.acc  (+12)*(+13) -> 156");
466         op(F3_MUL,  5'd5,  12,      -32'sd13,  "mul.acc  (+12)*(-13) -> -156");
467         op(F3_RDHI, 5'd6,  0, 0,               "rdacc.hi (sign ext.) -> -1");
468         op(F3_MUL,  5'd5, -32'sd12,  13,       "mul.acc  (-12)*(+13) -> -156");
469         op(F3_MUL,  5'd5, -32'sd12, -32'sd13,  "mul.acc  (-12)*(-13) -> 156");
470         drain;
471
472         //----------------------------------------------------------------
473         $display("\n--- T4: Extreme operands, full 64-bit product ---");
474         op(F3_MUL,  5'd5, 32'h7FFFFFFF, 32'h7FFFFFFF, "mul.acc MAX*MAX = 0x3FFFFFFF00000001");
475         op(F3_RDHI, 5'd6, 0, 0,                       "rdacc.hi        -> 0x3FFFFFFF");
476         op(F3_MUL,  5'd5, 32'h80000000, 32'h80000000, "mul.acc MIN*MIN = 2^62");
477         op(F3_RDHI, 5'd6, 0, 0,                       "rdacc.hi        -> 0x40000000");
478         op(F3_MUL,  5'd5, 32'h80000000, 32'h7FFFFFFF, "mul.acc MIN*MAX = 0xC000000080000000");
479         op(F3_RDHI, 5'd6, 0, 0,                       "rdacc.hi        -> 0xC0000000");
480         op(F3_MUL,  5'd5, 32'hFFFFFFFF, 32'h80000000, "mul.acc (-1)*MIN = +2^31");
481         op(F3_RDHI, 5'd6, 0, 0,                       "rdacc.hi        -> 0");
482         drain;
483
484         //----------------------------------------------------------------
485         $display("\n--- T5: 64-bit accumulator saturation and sticky OVF ---");
486         op(F3_MUL,  5'd5, 32'h80000000, 32'h80000000, "mul.acc MIN*MIN        acc=2^62");
487         op(F3_MAC,  5'd5, 32'h80000000, 32'h80000000, "mac +2^62 -> 2^63 SAT to INT64_MAX");
488         op(F3_MAC,  5'd5, 32'h7FFFFFFF, 32'h7FFFFFFF, "mac  (stays saturated, OVF sticky)");
489         op(F3_RDHI, 5'd6, 0, 0,                       "rdacc.hi               -> 0x7FFFFFFF");
490         op(F3_CLR,  5'd0, 0, 0,                       "clracc (clears ACC and OVF)");
491         op(F3_MUL,  5'd5, 32'h80000000, 32'h7FFFFFFF, "mul.acc MIN*MAX        acc=-2^62");
492         op(F3_MSU,  5'd5, 32'h80000000, 32'h80000000, "msu  -2^62             (no ovf)");
493         op(F3_MAC,  5'd5, 32'h80000000, 32'h7FFFFFFF, "mac  -> below -2^63, SAT to INT64_MIN");
494         op(F3_RDHI, 5'd6, 0, 0,                       "rdacc.hi               -> 0x80000000");
495         op(F3_MUL,  5'd5, 2, 3,                       "mul.acc (restarts, clears OVF) -> 6");
```

```
        drain;

        //-------------------------------------------------------------------
        $display("\n--- T6: Unsigned MACU (modulo 2^64) ---");
        op(F3_CLR,  5'd0, 0, 0,                        "clracc");
        op(F3_MACU, 5'd5, 32'hFFFFFFFF, 32'hFFFFFFFF, "macu 0xFFFFFFFF^2 = 0xFFFFFFFE00000001");
        op(F3_RDHI, 5'd6, 0, 0,                        "rdacc.hi        -> 0xFFFFFFFE");
        op(F3_MACU, 5'd5, 32'hFFFFFFFF, 32'hFFFFFFFF, "macu again (wraps, no OVF)");
        op(F3_RDHI, 5'd6, 0, 0,                        "rdacc.hi        -> 0xFFFFFFFC");
        verbose = 1'b0;
        op(F3_CLR,  5'd0, 0, 0, "clracc");
        for (i = 1; i <= 10; i = i + 1)
            op(F3_MACU, 5'd0, i, i, "macu i*i");
        verbose = 1'b1;
        n_cap = 0;
        issue(enc(7'd0, F3_RDLO, 5'd10, OPC_CUSTOM0), 0, 0, 1'b1, "rdacc.lo  sum_{i=1..10} i^2");
        drain;
        check_value("sum of squares 1..10 (analytic 385)", cap_buf[0], 32'd385);

        //-------------------------------------------------------------------
        $display("\n--- T7: Destination x0 (no register write) ---");
        op(F3_MUL,  5'd0, 9, 9,  "mul.acc x0, 9, 9  (acc=81, no write)");
        op(F3_MAC,  5'd0, 1, 19, "mac     x0, 1, 19 (acc=100, no write)");
        op(F3_RDLO, 5'd31, 0, 0, "rdacc.lo x31      -> 100");
        drain;

        //-------------------------------------------------------------------
        $display("\n--- T8: Illegal instruction detection ---");
        issue(enc(7'b0000001, 3'b000, 5'd5, 7'b0110011), 5, 5, 1'b0,
              "RV32M MUL (opcode OP) -> illegal here");
        issue(enc(7'b0100000, F3_MAC, 5'd5, OPC_CUSTOM0), 5, 5, 1'b0,
              "custom-0 with funct7=0100000 -> illegal");
        issue(32'h0000_0013, 0, 0, 1'b0, "NOP (addi x0,x0,0) -> illegal here");
        op(F3_RDLO, 5'd5, 0, 0, "rdacc.lo  (ACC unchanged) -> 100");
        drain;

        //-------------------------------------------------------------------
        $display("\n--- T9: Q15 fixed-point read-out (rdacc.q15) ---");
        op(F3_MUL,   5'd0, 16384, 16384,              "mul.acc 0.5 * 0.5   (Q15)");
        op(F3_RDQ15, 5'd5, 0, 0,                      "rdacc.q15 -> 0.25  = 8192");
        op(F3_MUL,   5'd0, -32'sd16384, 16384,        "mul.acc -0.5 * 0.5  (Q15)");
        op(F3_RDQ15, 5'd5, 0, 0,                      "rdacc.q15 -> -0.25 = -8192");
        op(F3_MUL,   5'd0, 32'hFFFF8000, 32'hFFFF8000, "mul.acc -1.0 * -1.0 (Q15)");
        op(F3_RDQ15, 5'd5, 0, 0,                      "rdacc.q15 -> +1.0 SAT to 32767");
        op(F3_MAC,   5'd0, 32767, 32767,              "mac ~ +1.0");
        op(F3_MAC,   5'd0, 32767, 32767,              "mac ~ +1.0");
        op(F3_RDQ15, 5'd5, 0, 0,                      "rdacc.q15 -> ~3.0 SAT to 32767");
        op(F3_MUL,   5'd0, 32'hFFFF8000, 32767,       "mul.acc -1.0 * 0.99997");
        op(F3_MAC,   5'd0, 32'hFFFF8000, 32767,       "mac  -> ~ -2.0");
        op(F3_RDQ15, 5'd5, 0, 0,                      "rdacc.q15 -> SAT to -32768");
        drain;

        //-------------------------------------------------------------------
        $display("\n--- T10: DSP benchmark - dot product [1..8].[8..1] ---");
        verbose = 1'b0;
        t0 = $time;
        for (i = 1; i <= 8; i = i + 1)
            op((i == 1) ? F3_MUL : F3_MAC, 5'd0, i, 9 - i, "dot-product term");
        verbose = 1'b1;
        n_cap = 0;
        issue(enc(7'd0, F3_RDLO, 5'd10, OPC_CUSTOM0), 0, 0, 1'b1, "rdacc.lo  dot product result");
        t1 = $time;
        drain;
        check_value("dot product (analytic 120)", cap_buf[0], 32'd120);
        $display("  Throughput: 9 instructions issued in %0d cycles (1 instr/cycle)",
                 (t1 - t0) / CLK_PERIOD);

        //-------------------------------------------------------------------
        $display("\n--- T11: DSP benchmark - 8-tap moving-average FIR, step input ---");
        $display("  h[k] = 4096 (0.125 in Q15), x[n] = 16384 (0.5) for n >= 0");
        $display("  Analytic: y[n] = 2048*(n+1) for n < 8, then 16384");
        verbose = 1'b0;
        for (k = 0; k < TAPS; k = k + 1) fir_h[k] = 16'sd4096;
```

```
        fir_len = 12;
        for (n = 0; n < fir_len; n = n + 1) fir_x[n] = 16'sd16384;
        t0 = $time;
        run_fir;
        t1 = $time;
        verbose = 1'b1;
        $display("    n | x[n]   | y[n] DUT | y[n] analytic");
        for (n = 0; n < fir_len; n = n + 1) begin
            $display("   %2d | %6d | %8d | %8d %s", n, fir_x[n], $signed(cap_buf[n]),
                     (n < 8) ? 2048*(n+1) : 16384,
                     ($signed(cap_buf[n]) == ((n < 8) ? 2048*(n+1) : 16384)) ? "PASS" : "FAIL");
            check_value("FIR moving average y[n]", cap_buf[n], (n < 8) ? 2048*(n+1) : 16384);
        end
        $display("  FIR cost: %0d instructions/sample, %0d cycles for %0d samples (incl. drain)",
                 TAPS + 1, (t1 - t0) / CLK_PERIOD, fir_len);

        //------------------------------------------------------------------
        $display("\n--- T12: DSP benchmark - 8-tap triangular low-pass FIR ---");
        $display("  h = {1024,3072,5120,7168,7168,5120,3072,1024}  (sum = 1.0 in Q15)");
        $display("  x[n] = 8000 + 6000*(-1)^n   (DC + Nyquist-frequency tone)");
        $display("  Analytic: H(Nyquist) = sum h[k](-1)^k = 0, H(DC) = 1");
        $display("            => y[n] = 8000 exactly once the 8-tap window is full");
        fir_h[0] = 1024; fir_h[1] = 3072; fir_h[2] = 5120; fir_h[3] = 7168;
        fir_h[4] = 7168; fir_h[5] = 5120; fir_h[6] = 3072; fir_h[7] = 1024;
        fir_len = 16;
        for (n = 0; n < fir_len; n = n + 1)
            fir_x[n] = (n % 2 == 0) ? 16'sd14000 : 16'sd2000;
        verbose = 1'b0;
        run_fir;
        verbose = 1'b1;
        $display("    n | x[n]   | y[n] DUT");
        for (n = 0; n < fir_len; n = n + 1)
            $display("   %2d | %6d | %8d %s", n, fir_x[n], $signed(cap_buf[n]),
                     (n >= TAPS-1) ? "(steady state)" : "(warm-up)");
        for (n = TAPS-1; n < fir_len; n = n + 1)
            check_value("LPF steady-state output = 8000", cap_buf[n], 32'd8000);

        //------------------------------------------------------------------
        $display("\n--- T13: Constrained-random regression (3000 instructions) ---");
        verbose = 1'b0;
        for (i = 0; i < 3000; i = i + 1) begin
            rnd_operand; a_r = rnd_val;
            rnd_operand; b_r = rnd_val;
            f3_r = $random(seed);
            rd_r = $random(seed);
            if ({$random(seed)} % 20 == 0) begin
                // ~5% illegal: wrong opcode or non-zero funct7
                if ({$random(seed)} % 2)
                    ins = enc(7'd0, f3_r, rd_r, 7'b0110011);
                else
                    ins = enc(7'b0000001 + ({$random(seed)} % 127), f3_r, rd_r, OPC_CUSTOM0);
            end else
                ins = enc(7'd0, f3_r, rd_r, OPC_CUSTOM0);

            issue(ins, a_r, b_r, 1'b0, "random");

            // ~25% chance of 1..3 bubble cycles between instructions
            if ({$random(seed)} % 4 == 0) begin
                idle(1 + {$random(seed)} % 3);
                cov_bubble = cov_bubble + 1;
            end
        end
        drain;
        $display("  random regression complete");

        //------------------------------------------------------------------
        // Summary
        //------------------------------------------------------------------
        $display("\n========================================================================");
        $display(" FUNCTIONAL COVERAGE");
        $display("   mac=%0d msu=%0d mul.acc=%0d macu=%0d rdacc.lo=%0d rdacc.hi=%0d rdacc.q15=%0d clracc=%0
    d",
                 cov_op[0], cov_op[1], cov_op[2], cov_op[3],
```

```
                cov_op[4], cov_op[5], cov_op[6], cov_op[7]);
        $display("   illegal=%0d  saturation events=%0d  rd=x0 writes=%0d  bubble gaps=%0d",
                cov_ill, cov_sat, cov_x0, cov_bubble);
        $display(" RESULTS");
        $display("   instructions issued : %0d", wr_ptr);
        $display("   checks performed    : %0d", n_checks);
        $display("   errors              : %0d", n_errors);
        if (n_errors == 0 && rd_ptr == wr_ptr)
            $display("   >>> ALL TESTS PASSED <<<");
        else
            $display("   >>> TEST FAILED <<<");
        $display("================================================================");
        $finish;
    end

    // Safety timeout
    initial begin
        #2_000_000;
        $display("TIMEOUT");
        $finish;
    end

endmodule
```

A.4 tb/tb_booth_multiplier.v

```
//================================================================================
// File        : tb_booth_multiplier.v
// Module      : tb_booth_multiplier
// Project     : Design and Verification of a RISC-V Multiply-Accumulate (MAC)
//               Unit for Digital Signal Processing
// Language    : Verilog-2005 (simulation only)
// Authors     : Mohsin Mushtaq (2025MECECS001)
//               Mubashir Karim Ganaie (2025MECECS005)
// Supervisor  : Dr. Afshan Amin Khan
// Institution : Department of Electronics and Communications,
//               National Institute of Technology Srinagar
//
// Description :
//   Unit-level testbench for the radix-4 Booth multiplier.
//
//   Part A : EXHAUSTIVE check of an 8-bit instance - all 2^16 = 65,536
//            signed operand pairs are compared against the simulator's
//            native '*' operator.
//   Part B : The 34-bit instance actually used in mac_unit, checked on
//            directed corner cases (0, +-1, INT32_MIN/MAX as used by the MAC,
//            34-bit extremes) and 20,000 random operand pairs.
//
//   Because the Booth algorithm is width-independent, the exhaustive 8-bit
//   proof plus the random/corner 34-bit regression gives high confidence in
//   the multiplier used by the MAC unit.
//================================================================================
`timescale 1ns/1ps

module tb_booth_multiplier;

    //--------------------------------------------------------------------------
    // Part A DUT : 8-bit exhaustive
    //--------------------------------------------------------------------------
    reg  signed [7:0]  a8, b8;
    wire signed [15:0] p8;

    booth_multiplier #(.W(8)) dut8 (.a_i(a8), .b_i(b8), .p_o(p8));

    //--------------------------------------------------------------------------
    // Part B DUT : 34-bit (same configuration as inside mac_unit)
    //--------------------------------------------------------------------------
    reg  signed [33:0] a34, b34;
    wire signed [67:0] p34;

```

```
    booth_multiplier #(.W(34)) dut34 (.a_i(a34), .b_i(b34), .p_o(p34));

    integer i, j, errs, checks, seed;
    reg signed [67:0] ref34;

    // Check one 34-bit pair against the reference product
    task check34;
        input signed [33:0] a;
        input signed [33:0] b;
        input               show;
        begin
            a34 = a; b34 = b;
            #1;
            ref34  = a * b;          // 68-bit reference (context-sized)
            checks = checks + 1;
            if (p34 !== ref34) begin
                errs = errs + 1;
                $display("  [FAIL] %0d * %0d : got %0d, expected %0d", a, b, p34, ref34);
            end else if (show)
                $display("  [PASS] %0d * %0d = %0d", a, b, p34);
        end
    endtask

    initial begin
        errs = 0; checks = 0; seed = 32'hC0FFEE;

        $display("========================================================================");
        $display(" Radix-4 Booth Multiplier -- Unit Testbench");
        $display("========================================================================");

        //---------------- Part A: exhaustive 8x8 ----------------
        $display("\n--- Part A: exhaustive 8-bit signed (65,536 cases) ---");
        for (i = -128; i < 128; i = i + 1) begin
            for (j = -128; j < 128; j = j + 1) begin
                a8 = i; b8 = j;
                #1;
                checks = checks + 1;
                if (p8 !== i * j) begin
                    errs = errs + 1;
                    if (errs < 10)
                        $display("  [FAIL] %0d * %0d : got %0d, expected %0d", i, j, p8, i * j);
                end
            end
        end
        $display("  exhaustive 8-bit check done, errors so far = %0d", errs);

        //---------------- Part B: 34-bit directed ----------------
        $display("\n--- Part B: 34-bit directed corner cases ---");
        check34(0, 0, 1);
        check34(1, -1, 1);
        check34(-1, -1, 1);
        check34(123456, -654321, 1);
        check34(34'sh0_7FFF_FFFF, 34'sh0_7FFF_FFFF, 1);  // INT32_MAX^2
        check34(34'sh3_8000_0000, 34'sh3_8000_0000, 1);  // INT32_MIN^2 (sign-ext)
        check34(34'sh0_FFFF_FFFF, 34'sh0_FFFF_FFFF, 1);  // UINT32_MAX^2 (zero-ext)
        check34(34'sh1_FFFF_FFFF, 34'sh2_0000_0000, 1);  // 34-bit MAX * MIN
        check34(34'sh2_0000_0000, 34'sh2_0000_0000, 1);  // 34-bit MIN * MIN

        //---------------- Part B: 34-bit random ----------------
        $display("\n--- Part B: 34-bit random (20,000 cases) ---");
        for (i = 0; i < 20000; i = i + 1)
            check34({$random(seed), $random(seed)}, {$random(seed), $random(seed)}, 0);
        $display("  random 34-bit check done");

        $display("\n========================================================================");
        $display("   checks performed : %0d", checks);
        $display("   errors           : %0d", errs);
        if (errs == 0) $display("   >>> ALL TESTS PASSED <<<");
        else           $display("   >>> TEST FAILED <<<");
        $display("========================================================================");
        $finish;
    end

```

```
endmodule
```

A.5 rtl/rv32_core.v (5-stage RISC-V processor)

```
//============================================================================
// File        : rv32_core.v
// Module      : rv32_core
// Project     : Design and Verification of a RISC-V Multiply-Accumulate (MAC)
//               Unit for Digital Signal Processing
// Language    : Verilog-2005 (IEEE Std 1364-2005), synthesizable
// Authors     : Mohsin Mushtaq (2025MECECS001)
//               Mubashir Karim Ganaie (2025MECECS005)
// Supervisor  : Dr. Afshan Amin Khan
// Institution : Department of Electronics and Communications,
//               National Institute of Technology Srinagar
//
// Description :
//   Classic 5-stage in-order pipelined RISC-V processor (RV32I + the
//   multiply subset of RV32M) with the MAC unit integrated as a custom-0
//   functional unit.
//
//       IF  ->  ID  ->  EX  ->  MEM  ->  WB
//                       |       |        ^
//                       +--MAC---+--MAC---+   (mac_unit stage 1 / stage 2)
//
//   * IF  : PC register, instruction memory read
//   * ID  : decode, immediate generation, register-file read, hazard unit
//   * EX  : forwarding muxes, ALU (incl. MUL/MULH/MULHSU/MULHU),
//           branch / jump resolution, MAC stage 1 (Booth multiply)
//   * MEM : data memory access (byte/half/word), MAC stage 2 (accumulate)
//   * WB  : register-file write-back (ALU / load / MAC result)
//
//   The two internal pipeline registers of mac_unit line up exactly with
//   the EX/MEM and MEM/WB registers of the core, so a MAC instruction flows
//   down the pipeline like any other instruction and writes back in WB.
//
//   Hazards
//   -------
//   * Data hazards : full forwarding from EX/MEM and MEM/WB into EX, plus
//                    write-first bypass of the register file in ID.
//   * Load-use and MAC-use : a load result (end of MEM) or a MAC result
//                    (end of MEM, out of mac_unit stage 2) is not ready for
//                    an instruction directly behind it, so the hazard unit
//                    inserts ONE bubble; the value is then forwarded from WB.
//   * Control      : branches and jumps resolve in EX; a taken branch
//                    flushes the two younger instructions (2-cycle penalty).
//   * ECALL / EBREAK stop instruction fetch; when the instruction reaches
//     WB the core halts (halted_o). An illegal instruction halts with
//     error_o = 1.
//
//   Supported instructions
//   ----------------------
//   RV32I : LUI AUIPC JAL JALR BEQ BNE BLT BGE BLTU BGEU LB LH LW LBU LHU
//           SB SH SW ADDI SLTI SLTIU XORI ORI ANDI SLLI SRLI SRAI ADD SUB
//           SLL SLT SLTU XOR SRL SRA OR AND FENCE(nop) ECALL EBREAK
//   RV32M : MUL MULH MULHSU MULHU   (divide instructions not implemented)
//   Xmac  : mac msu mul.acc macu rdacc.lo rdacc.hi rdacc.q15 clracc
//
// Ports :
//   clk_i, rst_ni        clock, active-low synchronous reset
//   imem_addr_o / imem_rdata_i                  instruction memory port
//   dmem_addr_o / dmem_wdata_o / dmem_be_o /
//   dmem_we_o / dmem_re_o / dmem_rdata_i        data memory port
//   halted_o, error_o    program finished (ECALL) / illegal instruction
//   *_cnt_o              performance counters for the testbench
//============================================================================
`timescale 1ns/1ps

module rv32_core (
    input  wire        clk_i,
```

```
    input  wire        rst_ni,

    // Instruction memory (combinational read)
    output wire [31:0] imem_addr_o,
    input  wire [31:0] imem_rdata_i,

    // Data memory (combinational read, synchronous write)
    output wire [31:0] dmem_addr_o,
    output reg  [31:0] dmem_wdata_o,
    output reg  [3:0]  dmem_be_o,
    output wire        dmem_we_o,
    output wire        dmem_re_o,
    input  wire [31:0] dmem_rdata_i,

    // Status
    output reg         halted_o,
    output reg         error_o,

    // Performance counters
    output reg  [31:0] cycle_cnt_o,
    output reg  [31:0] instret_cnt_o,
    output reg  [31:0] stall_cnt_o,
    output reg  [31:0] flush_cnt_o,
    output reg  [31:0] mac_cnt_o
);

    //---------------------------------------------------------------------
    // Encoding constants
    //---------------------------------------------------------------------
    localparam [6:0] OP_LUI    = 7'b0110111;
    localparam [6:0] OP_AUIPC  = 7'b0010111;
    localparam [6:0] OP_JAL    = 7'b1101111;
    localparam [6:0] OP_JALR   = 7'b1100111;
    localparam [6:0] OP_BRANCH = 7'b1100011;
    localparam [6:0] OP_LOAD   = 7'b0000011;
    localparam [6:0] OP_STORE  = 7'b0100011;
    localparam [6:0] OP_IMM    = 7'b0010011;
    localparam [6:0] OP_REG    = 7'b0110011;
    localparam [6:0] OP_FENCE  = 7'b0001111;
    localparam [6:0] OP_SYSTEM = 7'b1110011;
    localparam [6:0] OP_CUSTO  = 7'b0001011;   // MAC unit

    // ALU operations
    localparam [3:0] ALU_ADD    = 4'd0;
    localparam [3:0] ALU_SUB    = 4'd1;
    localparam [3:0] ALU_SLL    = 4'd2;
    localparam [3:0] ALU_SLT    = 4'd3;
    localparam [3:0] ALU_SLTU   = 4'd4;
    localparam [3:0] ALU_XOR    = 4'd5;
    localparam [3:0] ALU_SRL    = 4'd6;
    localparam [3:0] ALU_SRA    = 4'd7;
    localparam [3:0] ALU_OR     = 4'd8;
    localparam [3:0] ALU_AND    = 4'd9;
    localparam [3:0] ALU_PASSB  = 4'd10;
    localparam [3:0] ALU_MUL    = 4'd11;
    localparam [3:0] ALU_MULH   = 4'd12;
    localparam [3:0] ALU_MULHSU = 4'd13;
    localparam [3:0] ALU_MULHU  = 4'd14;

    // ALU operand-A select
    localparam [1:0] A_RS1  = 2'd0;
    localparam [1:0] A_PC   = 2'd1;

    //=====================================================================
    // Pipeline registers
    //=====================================================================
    // IF
    reg [31:0] pc_q;
    reg        fetch_stop_q;          // set once ECALL has left ID

    // IF/ID
    reg        if_id_valid;
    reg [31:0] if_id_pc;
```

```
    reg [31:0] if_id_instr;

    // ID/EX
    reg        id_ex_valid;
    reg [31:0] id_ex_pc;
    reg [31:0] id_ex_instr;
    reg [4:0]  id_ex_rs1, id_ex_rs2, id_ex_rd;
    reg [31:0] id_ex_rs1_val, id_ex_rs2_val, id_ex_imm;
    reg [3:0]  id_ex_alu_op;
    reg [1:0]  id_ex_a_sel;
    reg        id_ex_b_imm;
    reg        id_ex_reg_write, id_ex_mem_read, id_ex_mem_write;
    reg [2:0]  id_ex_funct3;
    reg        id_ex_branch, id_ex_jal, id_ex_jalr;
    reg        id_ex_is_mac, id_ex_is_ecall, id_ex_illegal;

    // EX/MEM
    reg        ex_mem_valid;
    reg [31:0] ex_mem_pc;
    reg [4:0]  ex_mem_rd;
    reg [31:0] ex_mem_result;          // ALU result or PC+4 (jumps)
    reg [31:0] ex_mem_store_data;
    reg        ex_mem_reg_write, ex_mem_mem_read, ex_mem_mem_write;
    reg [2:0]  ex_mem_funct3;
    reg        ex_mem_is_mac, ex_mem_is_ecall, ex_mem_illegal;

    // MEM/WB
    reg        mem_wb_valid;
    reg [31:0] mem_wb_pc;
    reg [4:0]  mem_wb_rd;
    reg [31:0] mem_wb_data;            // ALU result or load data
    reg        mem_wb_reg_write;
    reg        mem_wb_is_mac, mem_wb_is_ecall, mem_wb_illegal;

    // Register file
    reg [31:0] rf [0:31];

    //=========================================================================
    // WB stage (declared first: its outputs feed ID bypass and forwarding)
    //=========================================================================
    wire        mac_valid, mac_we, mac_illegal, mac_ovf;
    wire [4:0]  mac_rd;
    wire [31:0] mac_data;
    wire [63:0] mac_acc;

    wire        wb_we    = mem_wb_valid && !halted_o &&
                           (mem_wb_is_mac ? mac_we : mem_wb_reg_write) &&
                           (mem_wb_rd != 5'd0);
    wire [31:0] wb_value = mem_wb_is_mac ? mac_data : mem_wb_data;

    //=========================================================================
    // IF stage
    //=========================================================================
    assign imem_addr_o = pc_q;

    //=========================================================================
    // ID stage: decode
    //=========================================================================
    wire [31:0] instr  = if_id_instr;
    wire [6:0]  opcode = instr[6:0];
    wire [4:0]  rd     = instr[11:7];
    wire [2:0]  funct3 = instr[14:12];
    wire [4:0]  rs1    = instr[19:15];
    wire [4:0]  rs2    = instr[24:20];
    wire [6:0]  funct7 = instr[31:25];

    wire [31:0] imm_i = {{20{instr[31]}}, instr[31:20]};
    wire [31:0] imm_s = {{20{instr[31]}}, instr[31:25], instr[11:7]};
    wire [31:0] imm_b = {{19{instr[31]}}, instr[31], instr[7], instr[30:25], instr[11:8], 1'b0};
    wire [31:0] imm_u = {instr[31:12], 12'b0};
    wire [31:0] imm_j = {{11{instr[31]}}, instr[31], instr[19:12], instr[20], instr[30:21], 1'b0};

    reg [31:0] d_imm;
```

```
213     reg [3:0]  d_alu_op;
214     reg [1:0]  d_a_sel;
215     reg        d_b_imm;
216     reg        d_reg_write, d_mem_read, d_mem_write;
217     reg        d_branch, d_jal, d_jalr, d_is_mac, d_is_ecall, d_illegal;
218     reg        d_uses_rs1, d_uses_rs2;
219
220     always @* begin
221         d_imm       = 32'd0;
222         d_alu_op    = ALU_ADD;
223         d_a_sel     = A_RS1;
224         d_b_imm     = 1'b0;
225         d_reg_write = 1'b0;
226         d_mem_read  = 1'b0;
227         d_mem_write = 1'b0;
228         d_branch    = 1'b0;
229         d_jal       = 1'b0;
230         d_jalr      = 1'b0;
231         d_is_mac    = 1'b0;
232         d_is_ecall  = 1'b0;
233         d_illegal   = 1'b0;
234         d_uses_rs1  = 1'b0;
235         d_uses_rs2  = 1'b0;
236
237         case (opcode)
238             OP_LUI: begin
239                 d_reg_write = 1'b1; d_alu_op = ALU_PASSB; d_b_imm = 1'b1; d_imm = imm_u;
240             end
241             OP_AUIPC: begin
242                 d_reg_write = 1'b1; d_a_sel = A_PC; d_b_imm = 1'b1; d_imm = imm_u;
243             end
244             OP_JAL: begin
245                 d_reg_write = 1'b1; d_jal = 1'b1; d_imm = imm_j;
246             end
247             OP_JALR: begin
248                 d_reg_write = 1'b1; d_jalr = 1'b1; d_imm = imm_i; d_uses_rs1 = 1'b1;
249                 d_illegal = (funct3 != 3'b000);
250             end
251             OP_BRANCH: begin
252                 d_branch = 1'b1; d_imm = imm_b; d_uses_rs1 = 1'b1; d_uses_rs2 = 1'b1;
253                 d_illegal = (funct3 == 3'b010) || (funct3 == 3'b011);
254             end
255             OP_LOAD: begin
256                 d_reg_write = 1'b1; d_mem_read = 1'b1; d_b_imm = 1'b1; d_imm = imm_i;
257                 d_uses_rs1 = 1'b1;
258                 d_illegal = (funct3 == 3'b011) || (funct3 == 3'b110) || (funct3 == 3'b111);
259             end
260             OP_STORE: begin
261                 d_mem_write = 1'b1; d_b_imm = 1'b1; d_imm = imm_s;
262                 d_uses_rs1 = 1'b1; d_uses_rs2 = 1'b1;
263                 d_illegal = (funct3[2] == 1'b1) || (funct3 == 3'b011);
264             end
265             OP_IMM: begin
266                 d_reg_write = 1'b1; d_b_imm = 1'b1; d_imm = imm_i; d_uses_rs1 = 1'b1;
267                 case (funct3)
268                     3'b000: d_alu_op = ALU_ADD;
269                     3'b010: d_alu_op = ALU_SLT;
270                     3'b011: d_alu_op = ALU_SLTU;
271                     3'b100: d_alu_op = ALU_XOR;
272                     3'b110: d_alu_op = ALU_OR;
273                     3'b111: d_alu_op = ALU_AND;
274                     3'b001: begin
275                         d_alu_op = ALU_SLL; d_illegal = (funct7 != 7'b0000000);
276                     end
277                     3'b101: begin
278                         d_alu_op  = funct7[5] ? ALU_SRA : ALU_SRL;
279                         d_illegal = (funct7 != 7'b0000000) && (funct7 != 7'b0100000);
280                     end
281                 endcase
282             end
283             OP_REG: begin
284                 d_reg_write = 1'b1; d_uses_rs1 = 1'b1; d_uses_rs2 = 1'b1;
285                 if (funct7 == 7'b0000001) begin            // RV32M (multiply only)
```

```
                    case (funct3)
                        3'b000:  d_alu_op = ALU_MUL;
                        3'b001:  d_alu_op = ALU_MULH;
                        3'b010:  d_alu_op = ALU_MULHSU;
                        3'b011:  d_alu_op = ALU_MULHU;
                        default: d_illegal = 1'b1;          // DIV/REM not implemented
                    endcase
                end else if (funct7 == 7'b0000000) begin
                    case (funct3)
                        3'b000: d_alu_op = ALU_ADD;
                        3'b001: d_alu_op = ALU_SLL;
                        3'b010: d_alu_op = ALU_SLT;
                        3'b011: d_alu_op = ALU_SLTU;
                        3'b100: d_alu_op = ALU_XOR;
                        3'b101: d_alu_op = ALU_SRL;
                        3'b110: d_alu_op = ALU_OR;
                        3'b111: d_alu_op = ALU_AND;
                    endcase
                end else if (funct7 == 7'b0100000 && funct3 == 3'b000) begin
                    d_alu_op = ALU_SUB;
                end else if (funct7 == 7'b0100000 && funct3 == 3'b101) begin
                    d_alu_op = ALU_SRA;
                end else begin
                    d_illegal = 1'b1;
                end
            end
            OP_CUST0: begin
                // MAC unit decodes funct3/funct7 itself; only multiply-type
                // operations (funct3[2] = 0) read rs1/rs2.
                d_is_mac    = 1'b1;
                d_reg_write = 1'b1;
                d_uses_rs1  = ~funct3[2];
                d_uses_rs2  = ~funct3[2];
            end
            OP_FENCE: ;                                     // no-op
            OP_SYSTEM: begin
                if (instr == 32'h0000_0073 || instr == 32'h0010_0073)
                    d_is_ecall = 1'b1;                      // ECALL / EBREAK
                else
                    d_illegal = 1'b1;                       // CSRs not implemented
            end
            default: d_illegal = 1'b1;
        endcase
    end

    // Register-file read with write-first bypass from WB
    wire [31:0] rs1_val = (rs1 == 5'd0) ? 32'd0 :
                          (wb_we && mem_wb_rd == rs1) ? wb_value : rf[rs1];
    wire [31:0] rs2_val = (rs2 == 5'd0) ? 32'd0 :
                          (wb_we && mem_wb_rd == rs2) ? wb_value : rf[rs2];

    //------------------------------------------------------------------------
    // Hazard detection unit: load-use / MAC-use => one bubble
    //------------------------------------------------------------------------
    wire ex_is_late = id_ex_valid && (id_ex_mem_read || id_ex_is_mac) && (id_ex_rd != 5'd0);
    wire stall = if_id_valid && ex_is_late &&
                 ((d_uses_rs1 && rs1 == id_ex_rd) || (d_uses_rs2 && rs2 == id_ex_rd));

    //========================================================================
    // EX stage
    //========================================================================
    // Forwarding unit. EX/MEM can forward only ALU/jump results; load and
    // MAC results are forwarded from MEM/WB (the stall guarantees this).
    wire fwd_a_mem = ex_mem_valid && ex_mem_reg_write && !ex_mem_mem_read &&
                     !ex_mem_is_mac && (ex_mem_rd != 5'd0) && (ex_mem_rd == id_ex_rs1);
    wire fwd_b_mem = ex_mem_valid && ex_mem_reg_write && !ex_mem_mem_read &&
                     !ex_mem_is_mac && (ex_mem_rd != 5'd0) && (ex_mem_rd == id_ex_rs2);
    wire fwd_a_wb  = wb_we && (mem_wb_rd == id_ex_rs1);
    wire fwd_b_wb  = wb_we && (mem_wb_rd == id_ex_rs2);

    wire [31:0] fwd_a = fwd_a_mem ? ex_mem_result : fwd_a_wb ? wb_value : id_ex_rs1_val;
    wire [31:0] fwd_b = fwd_b_mem ? ex_mem_result : fwd_b_wb ? wb_value : id_ex_rs2_val;

```

```
359     wire [31:0] alu_a = (id_ex_a_sel == A_PC) ? id_ex_pc : fwd_a;
360     wire [31:0] alu_b = id_ex_b_imm ? id_ex_imm : fwd_b;
361
362     // One shared 33x33 signed multiplier for all RV32M multiply variants:
363     // each operand gets a 33rd bit that is its sign (signed operand) or 0
364     // (unsigned operand). MUL uses the low word, MULH* the high word.
365     wire mul_a_signed = (id_ex_alu_op == ALU_MULH) || (id_ex_alu_op == ALU_MULHSU);
366     wire mul_b_signed = (id_ex_alu_op == ALU_MULH);
367     wire signed [32:0] mul_a = {mul_a_signed & alu_a[31], alu_a};
368     wire signed [32:0] mul_b = {mul_b_signed & alu_b[31], alu_b};
369     wire signed [65:0] mul_p = mul_a * mul_b;
370
371     reg [31:0] alu_out;
372     always @* begin
373         case (id_ex_alu_op)
374             ALU_ADD:    alu_out = alu_a + alu_b;
375             ALU_SUB:    alu_out = alu_a - alu_b;
376             ALU_SLL:    alu_out = alu_a << alu_b[4:0];
377             ALU_SLT:    alu_out = {31'd0, $signed(alu_a) < $signed(alu_b)};
378             ALU_SLTU:   alu_out = {31'd0, alu_a < alu_b};
379             ALU_XOR:    alu_out = alu_a ^ alu_b;
380             ALU_SRL:    alu_out = alu_a >> alu_b[4:0];
381             ALU_SRA:    alu_out = $signed(alu_a) >>> alu_b[4:0];
382             ALU_OR:     alu_out = alu_a | alu_b;
383             ALU_AND:    alu_out = alu_a & alu_b;
384             ALU_PASSB:  alu_out = alu_b;
385             ALU_MUL:    alu_out = mul_p[31:0];
386             ALU_MULH:   alu_out = mul_p[63:32];
387             ALU_MULHSU: alu_out = mul_p[63:32];
388             ALU_MULHU:  alu_out = mul_p[63:32];
389             default:    alu_out = 32'd0;
390         endcase
391     end
392
393     // Branch / jump resolution
394     reg br_cond;
395     always @* begin
396         case (id_ex_funct3)
397             3'b000:  br_cond = (fwd_a == fwd_b);                    // BEQ
398             3'b001:  br_cond = (fwd_a != fwd_b);                    // BNE
399             3'b100:  br_cond = ($signed(fwd_a) <  $signed(fwd_b));  // BLT
400             3'b101:  br_cond = ($signed(fwd_a) >= $signed(fwd_b));  // BGE
401             3'b110:  br_cond = (fwd_a <  fwd_b);                    // BLTU
402             3'b111:  br_cond = (fwd_a >= fwd_b);                    // BGEU
403             default: br_cond = 1'b0;
404         endcase
405     end
406
407     wire        ex_taken  = id_ex_valid && ((id_ex_branch && br_cond) || id_ex_jal || id_ex_jalr);
408     wire [31:0] ex_target = id_ex_jalr ? ((fwd_a + id_ex_imm) & ~32'd1) : (id_ex_pc + id_ex_imm);
409     wire [31:0] ex_result = (id_ex_jal || id_ex_jalr) ? (id_ex_pc + 32'd4) : alu_out;
410
411     //-----------------------------------------------------------------------
412     // MAC unit: stage 1 in EX, stage 2 in MEM, result in WB
413     //-----------------------------------------------------------------------
414     mac_unit u_mac (
415         .clk_i     (clk_i),
416         .rst_ni    (rst_ni),
417         .valid_i   (id_ex_valid && id_ex_is_mac),
418         .instr_i   (id_ex_instr),
419         .rs1_i     (fwd_a),
420         .rs2_i     (fwd_b),
421         .ready_o   (),
422         .valid_o   (mac_valid),
423         .rd_addr_o (mac_rd),
424         .rd_we_o   (mac_we),
425         .rd_data_o (mac_data),
426         .illegal_o (mac_illegal),
427         .ovf_o     (mac_ovf),
428         .acc_o     (mac_acc)
429     );
430
431     //=======================================================================
```

```
    // MEM stage
    //=========================================================================
    wire [1:0] byte_off = ex_mem_result[1:0];

    assign dmem_addr_o = ex_mem_result;
    assign dmem_we_o   = ex_mem_valid && ex_mem_mem_write && !halted_o;
    assign dmem_re_o   = ex_mem_valid && ex_mem_mem_read;

    // Store data alignment and byte enables
    always @* begin
        case (ex_mem_funct3[1:0])
            2'b00: begin                                    // SB
                dmem_wdata_o = {4{ex_mem_store_data[7:0]}};
                dmem_be_o    = 4'b0001 << byte_off;
            end
            2'b01: begin                                    // SH
                dmem_wdata_o = {2{ex_mem_store_data[15:0]}};
                dmem_be_o    = byte_off[1] ? 4'b1100 : 4'b0011;
            end
            default: begin                                  // SW
                dmem_wdata_o = ex_mem_store_data;
                dmem_be_o    = 4'b1111;
            end
        endcase
    end

    // Load data extraction and sign/zero extension
    wire [31:0] ld_shifted = dmem_rdata_i >> {byte_off, 3'b000};
    reg  [31:0] load_data;
    always @* begin
        case (ex_mem_funct3)
            3'b000:  load_data = {{24{ld_shifted[7]}},  ld_shifted[7:0]};   // LB
            3'b001:  load_data = {{16{ld_shifted[15]}}, ld_shifted[15:0]};  // LH
            3'b100:  load_data = {24'd0, ld_shifted[7:0]};                  // LBU
            3'b101:  load_data = {16'd0, ld_shifted[15:0]};                 // LHU
            default: load_data = dmem_rdata_i;                              // LW
        endcase
    end

    //=========================================================================
    // Sequential logic: pipeline registers
    //=========================================================================
    wire id_leaves_ecall = if_id_valid && d_is_ecall && !stall && !ex_taken;

    // IF: program counter
    always @(posedge clk_i) begin
        if (!rst_ni)                        pc_q <= 32'd0;
        else if (ex_taken)                  pc_q <= ex_target;
        else if (stall || fetch_stop_q ||
                 id_leaves_ecall)           pc_q <= pc_q;
        else                                pc_q <= pc_q + 32'd4;
    end

    always @(posedge clk_i) begin
        if (!rst_ni)              fetch_stop_q <= 1'b0;
        else if (id_leaves_ecall) fetch_stop_q <= 1'b1;
    end

    // IF/ID
    always @(posedge clk_i) begin
        if (!rst_ni || ex_taken) begin
            if_id_valid <= 1'b0;
            if_id_pc    <= 32'd0;
            if_id_instr <= 32'h0000_0013;                   // NOP
        end else if (stall) begin
            // hold
        end else if (fetch_stop_q || id_leaves_ecall) begin
            if_id_valid <= 1'b0;
        end else begin
            if_id_valid <= 1'b1;
            if_id_pc    <= pc_q;
            if_id_instr <= imem_rdata_i;
        end
```

```
505     end
506
507     // ID/EX (bubble on reset, flush, stall or empty ID)
508     always @(posedge clk_i) begin
509         if (!rst_ni || ex_taken || stall || !if_id_valid) begin
510             id_ex_valid     <= 1'b0;
511             id_ex_reg_write <= 1'b0;
512             id_ex_mem_read  <= 1'b0;
513             id_ex_mem_write <= 1'b0;
514             id_ex_branch    <= 1'b0;
515             id_ex_jal       <= 1'b0;
516             id_ex_jalr      <= 1'b0;
517             id_ex_is_mac    <= 1'b0;
518             id_ex_is_ecall  <= 1'b0;
519             id_ex_illegal   <= 1'b0;
520             id_ex_rd        <= 5'd0;
521             id_ex_rs1       <= 5'd0;
522             id_ex_rs2       <= 5'd0;
523             id_ex_pc        <= if_id_pc;
524             id_ex_instr     <= 32'h0000_0013;
525         end else begin
526             id_ex_valid     <= 1'b1;
527             id_ex_pc        <= if_id_pc;
528             id_ex_instr     <= if_id_instr;
529             id_ex_rs1       <= d_uses_rs1 ? rs1 : 5'd0;
530             id_ex_rs2       <= d_uses_rs2 ? rs2 : 5'd0;
531             id_ex_rd        <= rd;
532             id_ex_rs1_val   <= rs1_val;
533             id_ex_rs2_val   <= rs2_val;
534             id_ex_imm       <= d_imm;
535             id_ex_alu_op    <= d_alu_op;
536             id_ex_a_sel     <= d_a_sel;
537             id_ex_b_imm     <= d_b_imm;
538             id_ex_reg_write <= d_reg_write;
539             id_ex_mem_read  <= d_mem_read;
540             id_ex_mem_write <= d_mem_write;
541             id_ex_funct3    <= funct3;
542             id_ex_branch    <= d_branch;
543             id_ex_jal       <= d_jal;
544             id_ex_jalr      <= d_jalr;
545             id_ex_is_mac    <= d_is_mac;
546             id_ex_is_ecall  <= d_is_ecall;
547             id_ex_illegal   <= d_illegal;
548         end
549     end
550
551     // EX/MEM
552     always @(posedge clk_i) begin
553         if (!rst_ni) begin
554             ex_mem_valid      <= 1'b0;
555             ex_mem_reg_write  <= 1'b0;
556             ex_mem_mem_read   <= 1'b0;
557             ex_mem_mem_write  <= 1'b0;
558             ex_mem_is_mac     <= 1'b0;
559             ex_mem_is_ecall   <= 1'b0;
560             ex_mem_illegal    <= 1'b0;
561             ex_mem_rd         <= 5'd0;
562             ex_mem_pc         <= 32'd0;
563             ex_mem_result     <= 32'd0;
564             ex_mem_store_data <= 32'd0;
565             ex_mem_funct3     <= 3'd0;
566         end else begin
567             ex_mem_valid      <= id_ex_valid;
568             ex_mem_pc         <= id_ex_pc;
569             ex_mem_rd         <= id_ex_rd;
570             ex_mem_result     <= ex_result;
571             ex_mem_store_data <= fwd_b;
572             ex_mem_reg_write  <= id_ex_valid & id_ex_reg_write;
573             ex_mem_mem_read   <= id_ex_valid & id_ex_mem_read;
574             ex_mem_mem_write  <= id_ex_valid & id_ex_mem_write;
575             ex_mem_funct3     <= id_ex_funct3;
576             ex_mem_is_mac     <= id_ex_valid & id_ex_is_mac;
577             ex_mem_is_ecall   <= id_ex_valid & id_ex_is_ecall;
```

```
            ex_mem_illegal    <= id_ex_valid & id_ex_illegal;
        end
    end

    // MEM/WB
    always @(posedge clk_i) begin
        if (!rst_ni) begin
            mem_wb_valid     <= 1'b0;
            mem_wb_reg_write <= 1'b0;
            mem_wb_is_mac    <= 1'b0;
            mem_wb_is_ecall  <= 1'b0;
            mem_wb_illegal   <= 1'b0;
            mem_wb_rd        <= 5'd0;
            mem_wb_pc        <= 32'd0;
            mem_wb_data      <= 32'd0;
        end else begin
            mem_wb_valid     <= ex_mem_valid;
            mem_wb_pc        <= ex_mem_pc;
            mem_wb_rd        <= ex_mem_rd;
            mem_wb_data      <= ex_mem_mem_read ? load_data : ex_mem_result;
            mem_wb_reg_write <= ex_mem_reg_write;
            mem_wb_is_mac    <= ex_mem_is_mac;
            mem_wb_is_ecall  <= ex_mem_is_ecall;
            mem_wb_illegal   <= ex_mem_illegal;
        end
    end

    // WB: register file write (x0 is never written). The register file has
    // no reset, as in most RISC-V cores, so synthesis can map it to RAM;
    // programs initialise every register before reading it.
    always @(posedge clk_i) begin
        if (wb_we) rf[mem_wb_rd] <= wb_value;
    end

    // Halt on ECALL/EBREAK or illegal instruction reaching WB
    always @(posedge clk_i) begin
        if (!rst_ni) begin
            halted_o <= 1'b0;
            error_o  <= 1'b0;
        end else if (mem_wb_valid && !halted_o) begin
            if (mem_wb_is_ecall) halted_o <= 1'b1;
            if (mem_wb_illegal || (mem_wb_is_mac && mac_illegal)) begin
                halted_o <= 1'b1;
                error_o  <= 1'b1;
            end
        end
    end

    // Performance counters
    always @(posedge clk_i) begin
        if (!rst_ni) begin
            cycle_cnt_o   <= 32'd0;
            instret_cnt_o <= 32'd0;
            stall_cnt_o   <= 32'd0;
            flush_cnt_o   <= 32'd0;
            mac_cnt_o     <= 32'd0;
        end else if (!halted_o) begin
            cycle_cnt_o <= cycle_cnt_o + 32'd1;
            if (mem_wb_valid)                instret_cnt_o <= instret_cnt_o + 32'd1;
            if (stall)                       stall_cnt_o   <= stall_cnt_o + 32'd1;
            if (ex_taken)                    flush_cnt_o   <= flush_cnt_o + 32'd1;
            if (mem_wb_valid && mem_wb_is_mac) mac_cnt_o    <= mac_cnt_o + 32'd1;
        end
    end

endmodule
```

A.6 rtl/riscv_soc.v

```
//============================================================================
```

```
// File        : riscv_soc.v
// Module      : riscv_soc
// Project     : Design and Verification of a RISC-V Multiply-Accumulate (MAC)
//               Unit for Digital Signal Processing
// Language    : Verilog-2005
// Authors     : Mohsin Mushtaq (2025MECECS001)
//               Mubashir Karim Ganaie (2025MECECS005)
// Supervisor  : Dr. Afshan Amin Khan
// Institution : Department of Electronics and Communications,
//               National Institute of Technology Srinagar
//
// Description :
//   Minimal system-on-chip: rv32_core (with the integrated MAC unit) plus a
//   Harvard instruction memory and a byte-enabled data memory.
//
//   Memory map (as seen by programs):
//     0x0000_0000 ... 0x0000_0FFF   instruction memory (IMEM_WORDS words)
//     0x0001_0000 ... 0x0001_3FFF   data memory        (DMEM_WORDS words)
//
//   Both memories have combinational read; the data memory writes on the
//   rising clock edge using per-byte enables. The memories are initialised
//   by the testbench with $readmemh (program and data images produced by
//   sw/build.py).
//==============================================================================
`timescale 1ns/1ps

module riscv_soc #(
    parameter integer IMEM_WORDS = 1024,     // 4 KiB
    parameter integer DMEM_WORDS = 4096      // 16 KiB
) (
    input  wire clk_i,
    input  wire rst_ni,
    output wire halted_o,
    output wire error_o
);

    wire [31:0] imem_addr, imem_rdata;
    wire [31:0] dmem_addr, dmem_wdata, dmem_rdata;
    wire [3:0]  dmem_be;
    wire        dmem_we, dmem_re;
    wire [31:0] cycle_cnt, instret_cnt, stall_cnt, flush_cnt, mac_cnt;

    rv32_core u_core (
        .clk_i         (clk_i),
        .rst_ni        (rst_ni),
        .imem_addr_o   (imem_addr),
        .imem_rdata_i  (imem_rdata),
        .dmem_addr_o   (dmem_addr),
        .dmem_wdata_o  (dmem_wdata),
        .dmem_be_o     (dmem_be),
        .dmem_we_o     (dmem_we),
        .dmem_re_o     (dmem_re),
        .dmem_rdata_i  (dmem_rdata),
        .halted_o      (halted_o),
        .error_o       (error_o),
        .cycle_cnt_o   (cycle_cnt),
        .instret_cnt_o (instret_cnt),
        .stall_cnt_o   (stall_cnt),
        .flush_cnt_o   (flush_cnt),
        .mac_cnt_o     (mac_cnt)
    );

    //--------------------------------------------------------------------------
    // Instruction memory (ROM image loaded by the testbench)
    //--------------------------------------------------------------------------
    reg [31:0] imem [0:IMEM_WORDS-1];
    assign imem_rdata = imem[imem_addr[$clog2(IMEM_WORDS)+1:2]];

    //--------------------------------------------------------------------------
    // Data memory (word array, byte write enables)
    //--------------------------------------------------------------------------
    reg [31:0] dmem [0:DMEM_WORDS-1];
    wire [$clog2(DMEM_WORDS)-1:0] d_idx = dmem_addr[$clog2(DMEM_WORDS)+1:2];
```

```
75
76      assign dmem_rdata = dmem[d_idx];
77
78      always @(posedge clk_i) begin
79          if (dmem_we) begin
80              if (dmem_be[0]) dmem[d_idx][7:0]   <= dmem_wdata[7:0];
81              if (dmem_be[1]) dmem[d_idx][15:8]  <= dmem_wdata[15:8];
82              if (dmem_be[2]) dmem[d_idx][23:16] <= dmem_wdata[23:16];
83              if (dmem_be[3]) dmem[d_idx][31:24] <= dmem_wdata[31:24];
84          end
85      end
86
87  endmodule
```

A.7 tb/tb_riscv_soc.v (system testbench)

```
1   //============================================================================
2   // File        : tb_riscv_soc.v
3   // Module      : tb_riscv_soc
4   // Project     : Design and Verification of a RISC-V Multiply-Accumulate (MAC)
5   //               Unit for Digital Signal Processing
6   // Language    : Verilog-2005 (simulation only)
7   // Authors     : Mohsin Mushtaq (2025MECECS001)
8   //               Mubashir Karim Ganaie (2025MECECS005)
9   // Supervisor  : Dr. Afshan Amin Khan
10  // Institution : Department of Electronics and Communications,
11  //               National Institute of Technology Srinagar
12  //
13  // Description :
14  //   System-level testbench: runs a real RISC-V program on riscv_soc
15  //   (5-stage rv32_core + integrated MAC unit + memories) and checks it.
16  //
17  //   Plusargs
18  //     +IMEM=<file>     program image ($readmemh, 32-bit words)   [required]
19  //     +DMEM=<file>     data image                                 [optional]
20  //     +GOLDEN=<file>   expected Q15 outputs (16-bit per line)     [optional]
21  //     +YADDR=<hex>     byte address of the output array y[]
22  //     +NOUT=<n>        number of outputs to compare
23  //     +YOUT=<file>     write the outputs produced by the CPU
24  //     +TRACE=<file>    per-cycle pipeline trace (CSV)
25  //     +VCD=<file>      waveform dump
26  //     +NAME=<string>   program name used in the RESULT line
27  //
28  //   Checks performed
29  //     * the core halts on ECALL within the cycle budget, without an
30  //       illegal-instruction error
31  //     * a0 (x10) == 0x600D  (self-checking programs signal success)
32  //     * every output sample y[n] == golden[n]   (bit-exact)
33  //     * on every MAC write-back the MAC unit's valid / rd outputs line up
34  //       with the core's MEM/WB pipeline register (integration check)
35  //
36  //   The final 'RESULT ...' line is parsed by sw/run_cpu_sims.py.
37  //============================================================================
38  `timescale 1ns/1ps
39
40  module tb_riscv_soc;
41
42      localparam integer CLK_PERIOD = 10;
43      localparam integer IMEM_WORDS = 1024;
44      localparam integer DMEM_WORDS = 4096;
45
46      reg clk = 1'b0;
47      always #(CLK_PERIOD/2) clk = ~clk;
48      reg rst_n;
49
50      wire halted, error;
51
52      riscv_soc #(.IMEM_WORDS(IMEM_WORDS), .DMEM_WORDS(DMEM_WORDS)) dut (
53          .clk_i    (clk),
54          .rst_ni   (rst_n),
```

```
        .halted_o (halted),
        .error_o  (error)
    );

    `define CORE dut.u_core

    //--------------------------------------------------------------------------
    // Plusargs and bookkeeping
    //--------------------------------------------------------------------------
    reg [8*512-1:0] f_imem, f_dmem, f_golden, f_yout, f_trace, f_vcd;
    reg [8*64-1:0]  name;
    integer yaddr, nout, max_cycles;
    integer have_golden, have_yout;
    integer trace_fd, yout_fd;
    integer i, errs, align_errs, timeout;
    reg [15:0] golden [0:4095];
    reg [15:0] yv;
    reg [31:0] word, a0;

    // Read a 16-bit value from the data memory (byte address)
    function [15:0] dmem_half;
        input [31:0] addr;
        reg [31:0] w;
        begin
            w = dut.dmem[addr[13:2]];
            dmem_half = addr[1] ? w[31:16] : w[15:0];
        end
    endfunction

    //--------------------------------------------------------------------------
    // Integration check: MAC unit output aligned with MEM/WB register
    //--------------------------------------------------------------------------
    always @(negedge clk) begin
        if (rst_n && !halted && `CORE.mem_wb_valid && `CORE.mem_wb_is_mac) begin
            if (!(`CORE.mac_valid && `CORE.mac_rd == `CORE.mem_wb_rd)) begin
                align_errs = align_errs + 1;
                $display("  [FAIL] MAC write-back misaligned at pc=%08h", `CORE.mem_wb_pc);
            end
        end
    end

    //--------------------------------------------------------------------------
    // Pipeline trace: one CSV line per clock cycle (sampled mid-cycle)
    //   cycle, IF pc, ID v/pc, EX v/pc, MEM v/pc, WB v/pc,
    //   stall, flush, fwdA(EX/MEM,WB), fwdB(EX/MEM,WB), mac_in_EX
    //--------------------------------------------------------------------------
    always @(negedge clk) begin
        if (rst_n && trace_fd != 0 && !halted) begin
            $fwrite(trace_fd, "%0d,%0h,%0d,%0d,%0h,%0d,%0h,%0d,%0h,%0d,%0h,%0d,%0d,%0d,%0d,%0d,%0d\n",
                    `CORE.cycle_cnt_o,
                    `CORE.pc_q, `CORE.fetch_stop_q,
                    `CORE.if_id_valid,  `CORE.if_id_pc,
                    `CORE.id_ex_valid,  `CORE.id_ex_pc,
                    `CORE.ex_mem_valid, `CORE.ex_mem_pc,
                    `CORE.mem_wb_valid, `CORE.mem_wb_pc,
                    `CORE.stall, `CORE.ex_taken,
                    `CORE.fwd_a_mem, `CORE.fwd_a_wb, `CORE.fwd_b_mem, `CORE.fwd_b_wb,
                    `CORE.id_ex_valid & `CORE.id_ex_is_mac);
        end
    end

    //--------------------------------------------------------------------------
    // Main sequence
    //--------------------------------------------------------------------------
    initial begin
        errs = 0; align_errs = 0; timeout = 0;
        name = "program";
        void_args;

        // memories: NOP-filled program, zeroed data
        for (i = 0; i < IMEM_WORDS; i = i + 1) dut.imem[i] = 32'h0000_0013;
        for (i = 0; i < DMEM_WORDS; i = i + 1) dut.dmem[i] = 32'h0000_0000;
        $readmemh(f_imem, dut.imem);
```

```
        if ($value$plusargs("DMEM=%s", f_dmem)) $readmemh(f_dmem, dut.dmem);
        if (have_golden) $readmemh(f_golden, golden, 0, nout-1);

        rst_n = 1'b0;
        repeat (3) @(posedge clk);
        #1 rst_n = 1'b1;

        // run until ECALL / error / timeout
        while (!halted && !timeout) begin
            @(posedge clk);
            if (`CORE.cycle_cnt_o >= max_cycles) timeout = 1;
        end
        @(negedge clk);

        a0 = `CORE.rf[10];
        $display("======================================================================");
        $display(" RISC-V SoC simulation : %0s", name);
        $display("======================================================================");
        $display("  halted=%0d  error=%0d  timeout=%0d  a0=0x%08h %s",
                 halted, error, timeout, a0, (a0 == 32'h600D) ? "(PASS code)" : "(FAIL code)");
        $display("  cycles            : %0d", `CORE.cycle_cnt_o);
        $display("  instructions      : %0d", `CORE.instret_cnt_o);
        $display("  CPI               : %0.3f", `CORE.cycle_cnt_o * 1.0 / `CORE.instret_cnt_o);
        $display("  load/MAC-use stalls: %0d", `CORE.stall_cnt_o);
        $display("  taken branch flushes: %0d", `CORE.flush_cnt_o);
        $display("  MAC instructions  : %0d", `CORE.mac_cnt_o);

        if (have_golden) begin
            $display("  --- output check: %0d samples at 0x%08h ---", nout, yaddr);
            $display("     n |  y (CPU) | y (golden)");
            for (i = 0; i < nout; i = i + 1) begin
                yv = dmem_half(yaddr + 2*i);
                if (yv !== golden[i]) begin
                    errs = errs + 1;
                    if (errs <= 10)
                        $display("  [FAIL] y[%0d] = %0d, expected %0d", i, $signed(yv), $signed(golden[i]))
    ;
                end
                if (i < 8 || i == nout-1)
                    $display("   %3d | %8d | %8d", i, $signed(yv), $signed(golden[i]));
                if (have_yout) $fwrite(yout_fd, "%0d\n", $signed(yv));
            end
            $display("  output mismatches : %0d / %0d", errs, nout);
        end
        if (align_errs) $display("  MAC alignment errors: %0d", align_errs);

        if (halted && !error && !timeout && a0 == 32'h600D && errs == 0 && align_errs == 0)
            $display("  >>> PASS <<<");
        else
            $display("  >>> FAIL <<<");

        $display("RESULT name=%0s status=%0s cycles=%0d instret=%0d stalls=%0d flushes=%0d macs=%0d a0=%0h
    errors=%0d",
                 name,
                 (halted && !error && !timeout && a0 == 32'h600D && errs == 0 && align_errs == 0) ? "PASS"
    : "FAIL",
                 `CORE.cycle_cnt_o, `CORE.instret_cnt_o, `CORE.stall_cnt_o,
                 `CORE.flush_cnt_o, `CORE.mac_cnt_o, a0, errs + align_errs);

        if (trace_fd) $fclose(trace_fd);
        if (have_yout) $fclose(yout_fd);
        $finish;
    end

    // Parse plusargs (kept in a task so the initial block stays readable)
    task void_args;
        begin
            if (!$value$plusargs("IMEM=%s", f_imem)) begin
                $display("ERROR: +IMEM=<file> is required");
                $finish;
            end
            if (!$value$plusargs("NAME=%s", name)) name = "program";
            have_golden = $value$plusargs("GOLDEN=%s", f_golden);
```

```
198             if (!$value$plusargs("YADDR=%h", yaddr)) yaddr = 0;
199             if (!$value$plusargs("NOUT=%d", nout)) nout = 0;
200             if (!$value$plusargs("MAXCYCLES=%d", max_cycles)) max_cycles = 500000;
201             have_yout = $value$plusargs("YOUT=%s", f_yout);
202             if (have_yout) yout_fd = $fopen(f_yout, "w");
203             trace_fd = 0;
204             if ($value$plusargs("TRACE=%s", f_trace)) trace_fd = $fopen(f_trace, "w");
205             if ($value$plusargs("VCD=%s", f_vcd)) begin
206                 $dumpfile(f_vcd);
207                 $dumpvars(0, tb_riscv_soc);
208             end
209         end
210     endtask
211
212 endmodule
```

B RISC-V Programs

B.1 sw/isa_test.s

```
# ============================================================================
# isa_test.s - self-checking test program for rv32_core + MAC unit
#
# Every test loads its number into a3, computes a value and compares it
# with the expected constant. On the first mismatch the program jumps to
# 'fail' (a0 = failing test number). If everything passes, a0 = 0x600D.
# The testbench checks a0 when the core halts on ECALL.
#
# Besides instruction semantics, the tests are written so that the
# pipeline's hazard logic is exercised: back-to-back dependencies
# (EX/MEM and MEM/WB forwarding), load-use stalls, MAC-use stalls,
# store-after-load, taken/not-taken branches and jumps.
# ============================================================================
        .equ PASS, 0x600D

        .text
start:
# ---- T1: ADD / SUB ---------------------------------------------------------
        li   a3, 1
        li   t0, 5
        li   t1, 7
        add  t2, t0, t1
        li   t6, 12
        bne  t2, t6, fail
        sub  t2, t0, t1
        li   t6, -2
        bne  t2, t6, fail

# ---- T2: forwarding EX/MEM and MEM/WB (back-to-back dependencies) ---------
        li   a3, 2
        addi t0, x0, 3
        addi t1, t0, 4          # t0 from EX/MEM
        add  t2, t1, t0         # t1 from EX/MEM, t0 from MEM/WB
        add  t2, t2, t2         # chain
        li   t6, 20
        bne  t2, t6, fail

# ---- T3: logic operations --------------------------------------------------
        li   a3, 3
        li   t0, 0x0F0F
        li   t1, 0x00FF
        xor  t2, t0, t1
        li   t6, 0x0FF0
        bne  t2, t6, fail
        or   t2, t0, t1
        li   t6, 0x0FFF
        bne  t2, t6, fail
        and  t2, t0, t1
        li   t6, 0x000F
        bne  t2, t6, fail
        xori t2, t0, -1
        li   t6, 0xFFFFF0F0
        bne  t2, t6, fail
        andi t2, t0, 0x0F0
        li   t6, 0x000
        bne  t2, t6, fail
        ori  t2, t1, 0x700
        li   t6, 0x7FF
        bne  t2, t6, fail

# ---- T4: shifts ------------------------------------------------------------
        li   a3, 4
        li   t0, -16            # 0xFFFFFFF0
        srai t1, t0, 2
        li   t6, -4
```

```
        bne  t1, t6, fail
        srli t1, t0, 28
        li   t6, 0xF
        bne  t1, t6, fail
        slli t1, t0, 4
        li   t6, -256
        bne  t1, t6, fail
        li   t2, 4
        sll  t1, t0, t2
        li   t6, -256
        bne  t1, t6, fail
        srl  t1, t0, t2
        li   t6, 0x0FFFFFFF
        bne  t1, t6, fail
        sra  t1, t0, t2
        li   t6, -1
        bne  t1, t6, fail

# ---- T5: set-less-than ------------------------------------------------------
        li   a3, 5
        li   t0, -16
        li   t1, 1
        slt  t2, t0, t1
        li   t6, 1
        bne  t2, t6, fail
        sltu t2, t0, t1
        bnez t2, fail
        slti t2, t0, -15
        li   t6, 1
        bne  t2, t6, fail
        sltiu t2, t1, 2
        li   t6, 1
        bne  t2, t6, fail

# ---- T6: LUI / AUIPC --------------------------------------------------------
        li   a3, 6
        lui  t0, 0x12345
        li   t6, 0x12345000
        bne  t0, t6, fail
here:   auipc t1, 0
        la   t2, here
        bne  t1, t2, fail

# ---- T7: loads / stores, sign & zero extension ----------------------------
        li   a3, 7
        la   s0, scratch
        li   t0, 0x80FF7F01
        sw   t0, 0(s0)
        lw   t1, 0(s0)
        bne  t1, t0, fail
        lb   t1, 0(s0)
        li   t6, 0x01
        bne  t1, t6, fail
        lb   t1, 1(s0)
        li   t6, 0x7F
        bne  t1, t6, fail
        lb   t1, 2(s0)
        li   t6, -1
        bne  t1, t6, fail
        lbu  t1, 2(s0)
        li   t6, 0xFF
        bne  t1, t6, fail
        lh   t1, 2(s0)
        li   t6, 0xFFFF80FF
        bne  t1, t6, fail
        lhu  t1, 2(s0)
        li   t6, 0x80FF
        bne  t1, t6, fail
        lh   t1, 0(s0)
        li   t6, 0x7F01
        bne  t1, t6, fail
        li   t0, 0xAA
        sb   t0, 1(s0)
```

```
        lw   t1, 0(s0)
        li   t6, 0x80FFAA01
        bne  t1, t6, fail
        li   t0, 0x1234
        sh   t0, 2(s0)
        lw   t1, 0(s0)
        li   t6, 0x1234AA01
        bne  t1, t6, fail

# ---- T8: load-use hazard and store-after-load ------------------------------
        li   a3, 8
        lw   t1, 0(s0)
        addi t2, t1, 1          # load-use: needs one bubble
        li   t6, 0x1234AA02
        bne  t2, t6, fail
        lw   t1, 0(s0)
        sw   t1, 4(s0)          # store data comes straight from a load
        lw   t2, 4(s0)
        bne  t2, t1, fail

# ---- T9: branches, taken and not taken -------------------------------------
        li   a3, 9
        li   t0, -1
        li   t1, 1
        beq  t0, t1, fail
        bne  t0, t0, fail
        blt  t1, t0, fail
        bge  t0, t1, fail
        bltu t0, t1, fail
        bgeu t1, t0, fail
        blt  t0, t1, b1
        j    fail
b1:     bge  t1, t0, b2
        j    fail
b2:     bltu t1, t0, b3
        j    fail
b3:     bgeu t0, t1, b4
        j    fail
b4:     beq  t1, t1, b5
        j    fail
b5:     bne  t0, t1, b6
        j    fail
b6:

# ---- T10: JAL / JALR (call and return) -------------------------------------
        li   a3, 10
        li   t5, 0
        call func
        li   t6, 77
        bne  t5, t6, fail
        la   t0, after
        jalr x0, 0(t0)          # computed jump
        j    fail
after:

# ---- T11: RV32M multiply ---------------------------------------------------
        li   a3, 11
        li   t0, -7
        li   t1, 3
        mul  t2, t0, t1
        li   t6, -21
        bne  t2, t6, fail
        mulh t2, t0, t1
        li   t6, -1
        bne  t2, t6, fail
        mulhu t2, t0, t1        # 0xFFFFFFF9 * 3 = 0x2_FFFFFFEB
        li   t6, 2
        bne  t2, t6, fail
        mulhsu t2, t0, t1
        li   t6, -1
        bne  t2, t6, fail
        li   t0, 0x7FFFFFFF
        mulh t2, t0, t0
```

```
        li   t6, 0x3FFFFFFF
        bne  t2, t6, fail

# ---- T12: MAC basics + MAC-use hazard ------------------------------------
        li   a3, 12
        li   t0, 6
        li   t1, 7
        mul.acc x0, t0, t1      # ACC = 42
        mac  t2, t0, t1         # ACC = 84, t2 = 84
        addi t3, t2, 1          # MAC-use: one bubble, then forward from WB
        li   t6, 85
        bne  t3, t6, fail
        rdacc.lo t4
        li   t6, 84
        bne  t4, t6, fail
        rdacc.hi t4
        bnez t4, fail

# ---- T13: MAC operands from the forwarding network -----------------------
        li   a3, 13
        clracc
        addi t0, x0, 10
        addi t1, x0, 20
        mac  t2, t0, t1         # t1 from EX/MEM, t0 from MEM/WB
        li   t6, 200
        bne  t2, t6, fail
        lw   t0, 0(s0)          # 0x1234AA01
        mac  t2, t0, x0         # load-use into MAC operand: + 0
        li   t6, 200
        bne  t2, t6, fail

# ---- T14: dependent MAC chain --------------------------------------------
        li   a3, 14
        clracc
        li   t0, 2
        li   t1, 3
        mac  t2, t0, t1         # 6
        mac  t3, t0, t1         # 12
        add  t4, t2, t3         # 18
        li   t6, 18
        bne  t4, t6, fail
        msu  t5, t4, t1         # 12 - 18*3 = -42
        li   t6, -42
        bne  t5, t6, fail

# ---- T15: Q15 read-out and saturation through the core -------------------
        li   a3, 15
        li   t0, 16384          # 0.5 in Q15
        mul.acc x0, t0, t0
        rdacc.q15 t2
        li   t6, 8192           # 0.25
        bne  t2, t6, fail
        li   t0, 0x80000000
        mul.acc x0, t0, t0      # 2^62
        mac  x0, t0, t0         # overflow -> saturate to INT64_MAX
        rdacc.hi t2
        li   t6, 0x7FFFFFFF
        bne  t2, t6, fail
        rdacc.q15 t2
        li   t6, 32767
        bne  t2, t6, fail

# ---- T16: unsigned MACU --------------------------------------------------
        li   a3, 16
        clracc
        li   t0, -1             # 0xFFFFFFFF
        macu t2, t0, t0
        li   t6, 1
        bne  t2, t6, fail
        rdacc.hi t2
        li   t6, 0xFFFFFFFE
        bne  t2, t6, fail

```

```
# ---- T17: x0 is hard-wired to zero (ALU and MAC writes) --------------------
        li   a3, 17
        addi x0, x0, 5
        bnez x0, fail
        li   t0, 3
        mul.acc x0, t0, t0      # rd = x0: no write, ACC = 9
        bnez x0, fail
        rdacc.lo t2
        li   t6, 9
        bne  t2, t6, fail

# ---- T18: loop with backward branch: sum 1..10 -----------------------------
        li   a3, 18
        li   t0, 10
        li   t1, 0
loop:   add  t1, t1, t0
        addi t0, t0, -1
        bnez t0, loop
        li   t6, 55
        bne  t1, t6, fail

# ---- all passed ------------------------------------------------------------
        li   a0, PASS
        ecall

fail:   mv   a0, a3
        ecall

func:   addi t5, x0, 77
        ret

        .data
        .align 2
scratch: .space 16
```

B.2 sw/fir_mac_loop.s

```
# ============================================================================
# fir_mac_loop.s - Q15 FIR filter using the custom MAC instructions
#
#   y[n] = sat16( (sum_{k=0}^{TAPS-1} h[k] * x[n-k]) >>> 15 ),  n = 0..N-1
#
# Data (fir_data.inc, generated by build.py):
#   x_pad : TAPS-1 zeros followed by the N input samples (Q15, .half)
#   h_rev : filter taps in reverse order, so both pointers walk forward:
#           y[n] = sum_j h_rev[j] * x_pad[n + j]
#   y     : N output samples (Q15, .half)
#
# Per tap the inner loop needs 7 instructions; the 'mac' accumulates into
# the unit's 64-bit accumulator, and 'rdacc.q15' does the Q30 -> Q15
# conversion with saturation in a single instruction.
# ============================================================================
        .include "fir_data.inc"

        .text
start:
        la   s0, x_pad          # s0 -> x_pad[n]
        la   s1, h_rev
        la   s2, y
        li   s3, N
outer:
        mv   t0, s0
        mv   t1, s1
        li   t2, TAPS
        clracc
inner:
        lh   t3, 0(t0)          # x sample
        lh   t4, 0(t1)          # coefficient
        addi t0, t0, 2
        addi t1, t1, 2
```

```
        addi t2, t2, -1
        mac  x0, t3, t4         # ACC += x * h
        bnez t2, inner

        rdacc.q15 t5            # Q30 -> Q15 with saturation
        addi s0, s0, 2
        addi s3, s3, -1
        sh   t5, 0(s2)
        addi s2, s2, 2
        bnez s3, outer

        li   a0, 0x600D
        ecall
```

B.3 sw/fir_rv32im_loop.s

```
# ============================================================================
# fir_rv32im_loop.s - the same Q15 FIR filter in plain RV32IM software
#
# Baseline for the MAC unit: identical data, loop structure and results,
# but without the custom instructions. The 64-bit accumulator lives in the
# register pair {s5:s4} and is updated with mul / mulh / add / sltu (carry)
# / add / add. The Q30 -> Q15 conversion and the saturation to 16 bits
# are done in software after each output sample.
# ============================================================================
        .include "fir_data.inc"

        .text
start:
        la   s0, x_pad
        la   s1, h_rev
        la   s2, y
        li   s3, N
        li   a5, 32767
        li   a6, -32768
        li   a7, -1
outer:
        mv   t0, s0
        mv   t1, s1
        li   t2, TAPS
        li   s4, 0              # acc low
        li   s5, 0              # acc high
inner:
        lh   t3, 0(t0)
        lh   t4, 0(t1)
        addi t0, t0, 2
        addi t1, t1, 2
        mul  a2, t3, t4         # product low word
        mulh a3, t3, t4         # product high word
        addi t2, t2, -1
        add  s4, s4, a2         # 64-bit add with carry
        sltu a4, s4, a2
        add  s5, s5, a3
        add  s5, s5, a4
        bnez t2, inner

        # t6:t5 = acc >>> 15  (64-bit arithmetic shift)
        srli t5, s4, 15
        slli t6, s5, 17
        or   t5, t5, t6
        srai t6, s5, 15
        # saturate to [-32768, 32767]
        bltz t6, neg
        bnez t6, satpos
        bltu a5, t5, satpos
        j    store
neg:    bne  t6, a7, satneg
        bltu t5, a6, satneg
        j    store
satpos: mv   t5, a5
```

```
55          j    store
56  satneg: mv   t5, a6
57  store:
58          sh   t5, 0(s2)
59          addi s2, s2, 2
60          addi s0, s0, 2
61          addi s3, s3, -1
62          bnez s3, outer
63
64          li   a0, 0x600D
65          ecall
```

C Synthesis Scripts

C.1 synth/synth_generic.ys

```
# ============================================================================
# synth_generic.ys - technology-independent (ASIC-style) synthesis
#   Run from the project root:   yosys -s synth/synth_generic.ys
#
#   Gate library : 2-input CMOS-style cells mapped by ABC
#                  (AND/NAND/OR/NOR/XOR/XNOR/ANDNOT/ORNOT/MUX/NOT + D-FFs)
#
#   Run 1 : hierarchical  -> per-module gate counts
#   Run 2 : flattened     -> total area, transistor estimate, logic depth,
#                            gate-level netlist for post-synthesis simulation
#   Run 3 : comparison    -> booth_multiplier replaced by native '*' and
#                            Yosys' own Booth + Wallace-tree generator
#                            (synth -booth), to compare logic depth
# ============================================================================

# ---------------- Run 1: hierarchical gate counts ----------------
read_verilog rtl/booth_multiplier.v rtl/mac_unit.v
synth -top mac_unit
abc -g AND,NAND,OR,NOR,XOR,XNOR,ANDNOT,ORNOT,MUX
opt_clean -purge
tee -o synth/reports/generic_hier_stat.txt stat

# ---------------- Run 2: flattened design (the deliverable) -------
design -reset
read_verilog rtl/booth_multiplier.v rtl/mac_unit.v
synth -top mac_unit -flatten
abc -g AND,NAND,OR,NOR,XOR,XNOR,ANDNOT,ORNOT,MUX
opt_clean -purge
check -assert
tee -o synth/reports/generic_stat.txt stat
tee -o synth/reports/generic_stat_cmos.txt stat -tech cmos
# Longest topological path through combinational cells (= logic depth)
tee -o synth/reports/generic_ltp.txt ltp -noff
write_verilog -noattr synth/out/mac_unit_netlist.v

# ---------------- Run 3: Yosys Booth + Wallace tree (comparison) --
design -reset
read_verilog synth/native_multiplier.v rtl/mac_unit.v
synth -top mac_unit -flatten -booth
abc -g AND,NAND,OR,NOR,XOR,XNOR,ANDNOT,ORNOT,MUX
opt_clean -purge
tee -o synth/reports/generic_wallace_stat.txt stat
tee -o synth/reports/generic_wallace_ltp.txt ltp -noff
```

C.2 synth/synth_ecp5.ys

```
# ============================================================================
# synth_ecp5.ys - FPGA synthesis for a Lattice ECP5 (LFE5U-85F)
#   Run from the project root:   yosys -s synth/synth_ecp5.ys
#
#   Design A : mac_unit with the radix-4 Booth multiplier (as written)
#   Design B : same mac_unit, Booth module swapped for a native '*'
#              multiplier (synth/native_multiplier.v) so Yosys can map it
#              to the ECP5 hard MULT18X18D DSP blocks - comparison only.
#
#   For each design:
#     *_stat.txt   resource usage of mac_unit alone
#     *.json       mac_unit inside fmax_wrapper.v (registered I/O), which is
#                  placed and routed with nextpnr-ecp5 to obtain Fmax
# ============================================================================

# ---------------- Design A: Booth multiplier in LUTs ----------------
```

```
read_verilog rtl/booth_multiplier.v rtl/mac_unit.v
synth_ecp5 -top mac_unit
tee -o synth/reports/ecp5_booth_stat.txt stat

design -reset
read_verilog rtl/booth_multiplier.v rtl/mac_unit.v synth/fmax_wrapper.v
synth_ecp5 -top mac_unit_fmax_wrapper -json synth/out/mac_unit_ecp5_booth.json

# ---------------- Design B: native multiplier -> DSP blocks ---------
design -reset
read_verilog synth/native_multiplier.v rtl/mac_unit.v
synth_ecp5 -top mac_unit
tee -o synth/reports/ecp5_dsp_stat.txt stat

design -reset
read_verilog synth/native_multiplier.v rtl/mac_unit.v synth/fmax_wrapper.v
synth_ecp5 -top mac_unit_fmax_wrapper -json synth/out/mac_unit_ecp5_dsp.json
```

C.3 synth/fmax_wrapper.v

```
//=============================================================================
// File        : fmax_wrapper.v
// Module      : mac_unit_fmax_wrapper  (synthesis / timing only)
//
// Description :
//   Registers every input and output of mac_unit so that, after place and
//   route, all timing paths are register-to-register. This models the real
//   environment inside a RISC-V core, where rs1/rs2 come from the register
//   file / ID-EX pipeline register, and excludes FPGA I/O pad delays from
//   the reported maximum frequency. Not part of the verified design.
//=============================================================================
`timescale 1ns/1ps

module mac_unit_fmax_wrapper (
    input  wire        clk_i,
    input  wire        rst_ni,
    input  wire        valid_i,
    input  wire [31:0] instr_i,
    input  wire [31:0] rs1_i,
    input  wire [31:0] rs2_i,
    output reg         valid_o,
    output reg  [4:0]  rd_addr_o,
    output reg         rd_we_o,
    output reg  [31:0] rd_data_o,
    output reg         illegal_o,
    output reg         ovf_o,
    output reg  [63:0] acc_o
);
    // input registers (ID/EX pipeline register of the host core)
    reg        valid_q;
    reg [31:0] instr_q, rs1_q, rs2_q;

    wire        valid_w, rd_we_w, illegal_w, ovf_w, ready_w;
    wire [4:0]  rd_addr_w;
    wire [31:0] rd_data_w;
    wire [63:0] acc_w;

    always @(posedge clk_i) begin
        valid_q <= valid_i;
        instr_q <= instr_i;
        rs1_q   <= rs1_i;
        rs2_q   <= rs2_i;
    end

    mac_unit u_mac (
        .clk_i(clk_i), .rst_ni(rst_ni),
        .valid_i(valid_q), .instr_i(instr_q), .rs1_i(rs1_q), .rs2_i(rs2_q),
        .ready_o(ready_w),
        .valid_o(valid_w), .rd_addr_o(rd_addr_w), .rd_we_o(rd_we_w),
        .rd_data_o(rd_data_w), .illegal_o(illegal_w),
```

```
        .ovf_o(ovf_w), .acc_o(acc_w)
    );

    // output registers (register-file write port of the host core)
    always @(posedge clk_i) begin
        valid_o   <= valid_w;
        rd_addr_o <= rd_addr_w;
        rd_we_o   <= rd_we_w;
        rd_data_o <= rd_data_w;
        illegal_o <= illegal_w;
        ovf_o     <= ovf_w;
        acc_o     <= acc_w;
    end
endmodule
```

C.4 synth/native_multiplier.v (comparison only)

```
//=============================================================================
// File        : native_multiplier.v
// Module      : booth_multiplier  (drop-in replacement, comparison only)
//
// Description :
//   Same module name and ports as rtl/booth_multiplier.v, but written with
//   the Verilog '*' operator. It is used ONLY by synth/synth_ecp5.ys to
//   compare the hand-written radix-4 Booth multiplier (mapped to LUTs)
//   against the FPGA's hard DSP multipliers, which Yosys infers from '*'.
//   It is not part of the verified design.
//=============================================================================
`timescale 1ns/1ps

module booth_multiplier #(
    parameter integer W = 34
) (
    input  wire signed [W-1:0]   a_i,
    input  wire signed [W-1:0]   b_i,
    output wire signed [2*W-1:0] p_o
);
    assign p_o = a_i * b_i;
endmodule
```

C.5 synth/synth_core.ys

```
# ============================================================================
# synth_core.ys - synthesis of the complete processor (rv32_core with the
# integrated MAC unit). Memories are outside the core (ports), as in a real
# SoC where they map to SRAM / FPGA block RAM.
#   Run from the project root:   yosys -s synth/synth_core.ys
# ============================================================================

# ---------------- generic gates, hierarchical breakdown ----------------
read_verilog rtl/booth_multiplier.v rtl/mac_unit.v rtl/rv32_core.v
synth -top rv32_core
abc -g AND,NAND,OR,NOR,XOR,XNOR,ANDNOT,ORNOT,MUX
opt_clean -purge
check -assert
tee -o synth/reports/core_generic_hier_stat.txt stat

# ---------------- Lattice ECP5 ----------------
design -reset
read_verilog rtl/booth_multiplier.v rtl/mac_unit.v rtl/rv32_core.v
synth_ecp5 -top rv32_core -json synth/out/rv32_core_ecp5.json
tee -o synth/reports/core_ecp5_stat.txt stat
```

C.6 synth/schematics.ys

```
# ============================================================================
# schematics.ys - word-level (RTL) netlists for schematic rendering with
#                 netlistsvg (https://github.com/nturley/netlistsvg)
#   Run from the project root:   yosys -s synth/schematics.ys
#   then: netlistsvg synth/out/<name>.json -o <name>.svg
# ============================================================================

# mac_unit top level; the Booth multiplier stays a single block
read_verilog rtl/booth_multiplier.v rtl/mac_unit.v
hierarchy -top mac_unit
proc; opt -full; wreduce; opt_clean -purge
select mac_unit
write_json -selected synth/out/sch_mac_unit.json

# Booth multiplier, small instance (W = 6) after elaboration: shows the
# Booth recoding multiplexers and the partial-product adder chain
design -reset
read_verilog rtl/booth_multiplier.v
chparam -set W 6 booth_multiplier
hierarchy -top booth_multiplier
proc; opt -full; wreduce; alumacc; opt_clean -purge
write_json synth/out/sch_booth_w6.json

# Booth multiplier W = 6 mapped to gates (shows the real gate netlist)
synth -top booth_multiplier
abc -g AND,NAND,OR,NOR,XOR,XNOR,ANDNOT,ORNOT,MUX
opt_clean -purge
tee -o synth/reports/booth_w6_gates_stat.txt stat
write_json synth/out/sch_booth_w6_gates.json
```

D Simulation Logs

D.1 Booth multiplier testbench

```
================================================================
 Radix-4 Booth Multiplier -- Unit Testbench
================================================================

--- Part A: exhaustive 8-bit signed (65,536 cases) ---
  exhaustive 8-bit check done, errors so far = 0

--- Part B: 34-bit directed corner cases ---
  [PASS] 0 * 0 = 0
  [PASS] 1 * -1 = -1
  [PASS] -1 * -1 = 1
  [PASS] 123456 * -654321 = -80779853376
  [PASS] 2147483647 * 2147483647 = 4611686014132420609
  [PASS] -2147483648 * -2147483648 = 4611686018427387904
  [PASS] 4294967295 * 4294967295 = 18446744065119617025
  [PASS] 8589934591 * -8589934592 = -73786976286248271872
  [PASS] -8589934592 * -8589934592 = 73786976294838206464

--- Part B: 34-bit random (20,000 cases) ---
  random 34-bit check done

================================================================
   checks performed : 85545
   errors           : 0
   >>> ALL TESTS PASSED <<<
================================================================
../tb/tb_booth_multiplier.v:115: $finish called at 85545000 (1ps)
```

D.2 MAC unit testbench

```
VCD info: dumpfile mac_unit.vcd opened for output.
================================================================
 RISC-V MAC Unit (custom-0) -- Self-Checking Testbench
================================================================

--- T1: Reset state ---
  [PASS]           rdacc.lo x5          (after reset) | rd=0x00000000 (0)  acc=0x0000000000000000 ovf=0
  [PASS]           rdacc.hi x5          (after reset) | rd=0x00000000 (0)  acc=0x0000000000000000 ovf=0

--- T2: Basic MUL / MAC / MSU (back-to-back issue) ---
  [PASS]                   mul.acc x5, 7, 6     -> 42 | rd=0x0000002a (42)  acc=0x000000000000002a ovf=0
  [PASS]                   mac x5, 3, -4        -> 30 | rd=0x0000001e (30)  acc=0x000000000000001e ovf=0
  [PASS]                   msu x5, -5, 10       -> 80 | rd=0x00000050 (80)  acc=0x0000000000000050 ovf=0
  [PASS]               mac x5, 100, 1000    -> 100080 | rd=0x000186f0 (100080)  acc=0x00000000000186f0 ovf=0
  [PASS]               rdacc.lo x6          -> 100080 | rd=0x000186f0 (100080)  acc=0x00000000000186f0 ovf=0
  [PASS]                    rdacc.hi x7          -> 0 | rd=0x00000000 (0)  acc=0x00000000000186f0 ovf=0

--- T3: Sign combinations ---
  [PASS]                  mul.acc  (+12)*(+13) -> 156 | rd=0x0000009c (156)  acc=0x000000000000009c ovf=0
  [PASS]                mul.acc  (+12)*(-13) -> -156 | rd=0xffffff64 (-156)  acc=0xffffffffffffff64 ovf=0
  [PASS]                  rdacc.hi (sign ext.) -> -1 | rd=0xffffffff (-1)  acc=0xffffffffffffff64 ovf=0
  [PASS]                mul.acc  (-12)*(+13) -> -156 | rd=0xffffff64 (-156)  acc=0xffffffffffffff64 ovf=0
  [PASS]                mul.acc  (-12)*(-13) -> 156 | rd=0x0000009c (156)  acc=0x000000000000009c ovf=0

--- T4: Extreme operands, full 64-bit product ---
  [PASS]         mul.acc MAX*MAX = 0x3FFFFFFF00000001 | rd=0x00000001 (1)  acc=0x3fffffff00000001 ovf=0
  [PASS]               rdacc.hi        -> 0x3FFFFFFF | rd=0x3fffffff (1073741823)  acc=0x3fffffff00000001 ovf=0
  [PASS]                     mul.acc MIN*MIN = 2^62 | rd=0x00000000 (0)  acc=0x4000000000000000 ovf=0
  [PASS]               rdacc.hi        -> 0x40000000 | rd=0x40000000 (1073741824)  acc=0x4000000000000000 ovf=0
  [PASS]         mul.acc MIN*MAX = 0xC000000080000000 | rd=0x80000000 (-2147483648)  acc=0xc000000080000000 ovf=0
  [PASS]               rdacc.hi        -> 0xC0000000 | rd=0xc0000000 (-1073741824)  acc=0xc000000080000000 ovf=0
  [PASS]                    mul.acc (-1)*MIN = +2^31 | rd=0x80000000 (-2147483648)  acc=0x0000000080000000 ovf=0
  [PASS]                       rdacc.hi        -> 0 | rd=0x00000000 (0)  acc=0x0000000080000000 ovf=0

--- T5: 64-bit accumulator saturation and sticky OVF ---
  [PASS]             mul.acc MIN*MIN        acc=2^62 | rd=0x00000000 (0)  acc=0x4000000000000000 ovf=0
  [PASS]          mac +2^62 -> 2^63 SAT to INT64_MAX | rd=0xffffffff (-1)  acc=0x7fffffffffffffff ovf=1
  [PASS]          mac  (stays saturated, OVF sticky) | rd=0xffffffff (-1)  acc=0x7fffffffffffffff ovf=1
  [PASS]         rdacc.hi              -> 0x7FFFFFFF | rd=0x7fffffff (2147483647)  acc=0x7fffffffffffffff ovf=1
  [PASS]                clracc (clears ACC and OVF) | rd=0x00000000 (0)  acc=0x0000000000000000 ovf=0  (no write)
  [PASS]            mul.acc MIN*MAX       acc=-2^62 | rd=0x80000000 (-2147483648)  acc=0xc000000080000000 ovf=0
  [PASS]             msu  -2^62            (no ovf) | rd=0x80000000 (-2147483648)  acc=0x8000000080000000 ovf=0
  [PASS]        mac  -> below -2^63, SAT to INT64_MIN | rd=0x00000000 (0)  acc=0x8000000000000000 ovf=1
  [PASS]         rdacc.hi              -> 0x80000000 | rd=0x80000000 (-2147483648)  acc=0x8000000000000000 ovf=1
  [PASS]          mul.acc (restarts, clears OVF) -> 6 | rd=0x00000006 (6)  acc=0x0000000000000006 ovf=0

--- T6: Unsigned MACU (modulo 2^64) ---
  [PASS]                                  clracc | rd=0x00000000 (0)  acc=0x0000000000000000 ovf=0  (no write)
  [PASS]       macu 0xFFFFFFFF^2 = 0xFFFFFFFE00000001 | rd=0x00000001 (1)  acc=0xfffffffe00000001 ovf=0
  [PASS]               rdacc.hi        -> 0xFFFFFFFE | rd=0xfffffffe (-2)  acc=0xfffffffe00000001 ovf=0
  [PASS]                  macu again (wraps, no OVF) | rd=0x00000002 (2)  acc=0xfffffffc00000002 ovf=0
  [PASS]               rdacc.hi        -> 0xFFFFFFFC | rd=0xfffffffc (-4)  acc=0xfffffffc00000002 ovf=0
```

```
  [PASS]                 rdacc.lo  sum_{i=1..10} i^2 | rd=0x00000181 (385)  acc=0x0000000000000181 ovf=0
  [PASS]          sum of squares 1..10 (analytic 385) | got 385, expected 385

--- T7: Destination x0 (no register write) ---
  [PASS]         mul.acc x0, 9, 9  (acc=81, no write) | rd=0x00000051 (81)  acc=0x0000000000000051 ovf=0  (no write)
  [PASS]        mac     x0, 1, 19 (acc=100, no write) | rd=0x00000064 (100)  acc=0x0000000000000064 ovf=0  (no write)
  [PASS]                    rdacc.lo x31      -> 100 | rd=0x00000064 (100)  acc=0x0000000000000064 ovf=0

--- T8: Illegal instruction detection ---
  [PASS]        RV32M MUL (opcode OP) -> illegal here | rd=0x00000000 (0)  acc=0x0000000000000064 ovf=0     ILLEGAL
  [PASS]      custom-0 with funct7=0100000 -> illegal | rd=0x00000000 (0)  acc=0x0000000000000064 ovf=0     ILLEGAL
  [PASS]          NOP (addi x0,x0,0) -> illegal here | rd=0x00000000 (0)  acc=0x0000000000000064 ovf=0     ILLEGAL
  [PASS]            rdacc.lo  (ACC unchanged) -> 100 | rd=0x00000064 (100)  acc=0x0000000000000064 ovf=0

--- T9: Q15 fixed-point read-out (rdacc.q15) ---
  [PASS]                   mul.acc 0.5 * 0.5   (Q15) | rd=0x10000000 (268435456)  acc=0x0000000010000000 ovf=0  (no write)
  [PASS]                   rdacc.q15 -> 0.25  = 8192 | rd=0x00002000 (8192)  acc=0x0000000010000000 ovf=0
  [PASS]                   mul.acc -0.5 * 0.5  (Q15) | rd=0xf0000000 (-268435456)  acc=0xfffffffff0000000 ovf=0  (no write)
  [PASS]                  rdacc.q15 -> -0.25 = -8192 | rd=0xffffe000 (-8192)  acc=0xfffffffff0000000 ovf=0
  [PASS]                   mul.acc -1.0 * -1.0 (Q15) | rd=0x40000000 (1073741824)  acc=0x0000000040000000 ovf=0  (no write)
  [PASS]              rdacc.q15 -> +1.0 SAT to 32767 | rd=0x00007fff (32767)  acc=0x0000000040000000 ovf=0
  [PASS]                                mac ~ +1.0 | rd=0x7fff0001 (2147418113)  acc=0x000000007fff0001 ovf=0  (no write)
  [PASS]                                mac ~ +1.0 | rd=0xbffe0002 (-1073872894)  acc=0x00000000bffe0002 ovf=0  (no write)
  [PASS]              rdacc.q15 -> -3.0 SAT to 32767 | rd=0x00007fff (32767)  acc=0x00000000bffe0002 ovf=0
  [PASS]                     mul.acc -1.0 * 0.99997 | rd=0xc0008000 (-1073709056)  acc=0xfffffffffc0008000 ovf=0  (no write)
  [PASS]                             mac  -> ~ -2.0 | rd=0x80010000 (-2147418112)  acc=0xffffffff80010000 ovf=0  (no write)
  [PASS]                  rdacc.q15 -> SAT to -32768 | rd=0xffff8000 (-32768)  acc=0xffffffff80010000 ovf=0

--- T10: DSP benchmark - dot product [1..8].[8..1] ---
  [PASS]                rdacc.lo  dot product result | rd=0x00000078 (120)  acc=0x0000000000000078 ovf=0
  [PASS]                  dot product (analytic 120) | got 120, expected 120
  Throughput: 9 instructions issued in 9 cycles (1 instr/cycle)

--- T11: DSP benchmark - 8-tap moving-average FIR, step input ---
  h[k] = 4096 (0.125 in Q15), x[n] = 16384 (0.5) for n >= 0
  Analytic: y[n] = 2048*(n+1) for n < 8, then 16384
    n | x[n]   | y[n] DUT | y[n] analytic
    0 |  16384 |     2048 |     2048 PASS
  [PASS]                      FIR moving average y[n] | got 2048, expected 2048
    1 |  16384 |     4096 |     4096 PASS
  [PASS]                      FIR moving average y[n] | got 4096, expected 4096
    2 |  16384 |     6144 |     6144 PASS
  [PASS]                      FIR moving average y[n] | got 6144, expected 6144
    3 |  16384 |     8192 |     8192 PASS
  [PASS]                      FIR moving average y[n] | got 8192, expected 8192
    4 |  16384 |    10240 |    10240 PASS
  [PASS]                      FIR moving average y[n] | got 10240, expected 10240
    5 |  16384 |    12288 |    12288 PASS
  [PASS]                      FIR moving average y[n] | got 12288, expected 12288
    6 |  16384 |    14336 |    14336 PASS
  [PASS]                      FIR moving average y[n] | got 14336, expected 14336
    7 |  16384 |    16384 |    16384 PASS
  [PASS]                      FIR moving average y[n] | got 16384, expected 16384
    8 |  16384 |    16384 |    16384 PASS
  [PASS]                      FIR moving average y[n] | got 16384, expected 16384
    9 |  16384 |    16384 |    16384 PASS
  [PASS]                      FIR moving average y[n] | got 16384, expected 16384
   10 |  16384 |    16384 |    16384 PASS
  [PASS]                      FIR moving average y[n] | got 16384, expected 16384
   11 |  16384 |    16384 |    16384 PASS
  [PASS]                      FIR moving average y[n] | got 16384, expected 16384
  FIR cost: 9 instructions/sample, 111 cycles for 12 samples (incl. drain)

--- T12: DSP benchmark - 8-tap triangular low-pass FIR ---
  h = {1024,3072,5120,7168,7168,5120,3072,1024}  (sum = 1.0 in Q15)
  x[n] = 8000 + 6000*(-1)^n   (DC + Nyquist-frequency tone)
  Analytic: H(Nyquist) = sum h[k](-1)^k = 0, H(DC) = 1
            => y[n] = 8000 exactly once the 8-tap window is full
    n | x[n]   | y[n] DUT
    0 |  14000 |      437      (warm-up)
    1 |   2000 |     1375      (warm-up)
    2 |  14000 |     2812      (warm-up)
    3 |   2000 |     4750      (warm-up)
    4 |  14000 |     6312      (warm-up)
    5 |   2000 |     7375      (warm-up)
    6 |  14000 |     7937      (warm-up)
    7 |   2000 |     8000 (steady state)
    8 |  14000 |     8000 (steady state)
    9 |   2000 |     8000 (steady state)
   10 |  14000 |     8000 (steady state)
   11 |   2000 |     8000 (steady state)
   12 |  14000 |     8000 (steady state)
   13 |   2000 |     8000 (steady state)
   14 |  14000 |     8000 (steady state)
   15 |   2000 |     8000 (steady state)
  [PASS]                LPF steady-state output = 8000 | got 8000, expected 8000
  [PASS]                LPF steady-state output = 8000 | got 8000, expected 8000
  [PASS]                LPF steady-state output = 8000 | got 8000, expected 8000
  [PASS]                LPF steady-state output = 8000 | got 8000, expected 8000
  [PASS]                LPF steady-state output = 8000 | got 8000, expected 8000
  [PASS]                LPF steady-state output = 8000 | got 8000, expected 8000
  [PASS]                LPF steady-state output = 8000 | got 8000, expected 8000
  [PASS]                LPF steady-state output = 8000 | got 8000, expected 8000
  [PASS]                LPF steady-state output = 8000 | got 8000, expected 8000

--- T13: Constrained-random regression (3000 instructions) ---
  random regression complete

================================================================
```

```
 FUNCTIONAL COVERAGE
   mac=570 msu=338 mul.acc=378 macu=381 rdacc.lo=346 rdacc.hi=386 rdacc.q15=393 clracc=374
   illegal=162  saturation events=7  rd=x0 writes=360  bubble gaps=684
 RESULTS
   instructions issued : 3328
   checks performed    : 3351
   errors              : 0
   >>> ALL TESTS PASSED <<<
================================================================
../tb/tb_mac_unit.v:653: $finish called at 47070000 (1ps)
```

D.3 Processor: `isa_test`

```
================================================================
 RISC-V SoC simulation : isa_test
================================================================
  halted=1  error=0  timeout=0  a0=0x0000600d (PASS code)
  cycles            : 348
  instructions      : 300
  CPI               : 1.160
  load/MAC-use stalls: 8
  taken branch flushes: 18
  MAC instructions  : 22
  >>> PASS <<<
RESULT name=isa_test status=PASS cycles=348 instret=300 stalls=8 flushes=18 macs=22 a0=600d errors=0
tb/tb_riscv_soc.v:186: $finish called at 3520000 (1ps)
```

D.4 Processor: `fir_mac_loop`

```
VCD info: dumpfile sim/cpu/fir_mac_loop.vcd opened for output.
================================================================
 RISC-V SoC simulation : fir_mac_loop
================================================================
  halted=1  error=0  timeout=0  a0=0x0000600d (PASS code)
  cycles            : 19724
  instructions      : 15626
  CPI               : 1.262
  load/MAC-use stalls: 0
  taken branch flushes: 2047
  MAC instructions  : 2304
  --- output check: 128 samples at 0x00010140 ---
     n |  y (CPU) | y (golden)
     0 |        0 |        0
     1 |      -38 |      -38
     2 |      -44 |      -44
     3 |      -29 |      -29
     4 |       60 |       60
     5 |      362 |      362
     6 |      861 |      861
     7 |     1471 |     1471
   127 |   -15621 |   -15621
  output mismatches : 0 / 128
  >>> PASS <<<
RESULT name=fir_mac_loop status=PASS cycles=19724 instret=15626 stalls=0 flushes=2047 macs=2304 a0=600d errors=0
tb/tb_riscv_soc.v:186: $finish called at 197280000 (1ps)
```

D.5 Processor: `fir_rv32im_loop`

```
================================================================
 RISC-V SoC simulation : fir_rv32im_loop
================================================================
  halted=1  error=0  timeout=0  a0=0x0000600d (PASS code)
  cycles            : 31363
  instructions      : 26895
  CPI               : 1.166
  load/MAC-use stalls: 0
  taken branch flushes: 2232
  MAC instructions  : 0
  --- output check: 128 samples at 0x00010140 ---
     n |  y (CPU) | y (golden)
     0 |        0 |        0
     1 |      -38 |      -38
     2 |      -44 |      -44
     3 |      -29 |      -29
     4 |       60 |       60
     5 |      362 |      362
     6 |      861 |      861
     7 |     1471 |     1471
   127 |   -15621 |   -15621
  output mismatches : 0 / 128
  >>> PASS <<<
```

```
RESULT name=fir_rv32im_loop status=PASS cycles=31363 instret=26895 stalls=0 flushes=2232 macs=0 a0=600d errors=0
tb/tb_riscv_soc.v:186: $finish called at 313670000 (1ps)
```